Large Language Models for Software Engineering: A Systematic Literature Review

XINYI HOU and YANJIE ZHAO, Huazhong University of Science and Technology, Wuhan, China
YUE LIU, Monash University, Melbourne, Australia
ZHOU YANG, Singapore Management University, Singapore
KAILONG WANG, Huazhong University of Science and Technology, Wuhan, China
LI LI, Beihang University, Beijing, China
XIAPU LUO, The Hong Kong Polytechnic University, Hong Kong, China
DAVID LO, Singapore Management University, Singapore
JOHN GRUNDY, Monash University, Melbourne, Australia
HAOYU WANG, Huazhong University of Science and Technology, Wuhan, China

Large Language Models (LLMs) have significantly impacted numerous domains, including Software Engineering (SE). Many recent publications have explored LLMs applied to various SE tasks. Nevertheless, a comprehensive understanding of the application, effects, and possible limitations of LLMs on SE is still in its early stages. To bridge this gap, we conducted a Systematic Literature Review (SLR) on LLM4SE, with a particular focus on understanding how LLMs can be exploited to optimize processes and outcomes. We selected and analyzed 395 research articles from January 2017 to January 2024 to answer four key Research Questions (RQs). In RQ1, we categorize different LLMs that have been employed in SE tasks, characterizing their distinctive features and uses. In RQ2, we analyze the methods used in data collection, pre-processing,

Xinyi Hou and Yanjie Zhao contributed equally to this work.

This research/project is supported in part by the National Natural Science Foundation of China (grant No. 62072046), the Key R&D Program of Hubei Province (2023BAB017, 2023BAB079), the Knowledge Innovation Program of Wuhan-Basic Research, HUST CSE-HongXin Joint Institute for Cyber Security, HUST CSE-FiberHome Joint Institute for Cyber Security and the National Research Foundation, under its Investigatorship Grant (NRF-NRFI08-2022-0002). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore. J. Grundy is supported by ARC Laureate Fellowship FL190100035.

Authors' Contact Information: Xinyi Hou, Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China; e-mail: xinyihou@hust.edu.cn; Yanjie Zhao, Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China; e-mail: yanjie_zhao@hust.edu.cn; Yue Liu, Monash University, Melbourne, Australia; e-mail: yue.liu1@monash.edu; Zhou Yang, Singapore Management University, Singapore; e-mail: zyang@smu.edu.sg; Kailong Wang, Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China; e-mail: wangkl@hust.edu.cn; Li Li, Beihang University, Beijing, China; e-mail: lilicoding@ieee.org; Xiapu Luo, The Hong Kong Polytechnic University, Hong Kong, China; e-mail: csxluo@comp.polyu.edu.hk; David Lo, Singapore Management University, Singapore; e-mail: davidlo@smu.edu.sg; John Grundy, Monash University, Melbourne, Australia; e-mail: John.Grundy@monash.edu; Haoyu Wang (corresponding author), Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China; e-mail: haoyuwang@hust.edu.cn.

ACM 1557-7392/2024/12-ART220
https://doi.org/10.1145/3695988

and application, highlighting the role of well-curated datasets for successful LLM for SE implementation. RQ3 investigates the strategies employed to optimize and evaluate the performance of LLMs in SE. Finally, RQ4 examines the specific SE tasks where LLMs have shown success to date, illustrating their practical contributions to the field. From the answers to these RQs, we discuss the current state-of-the-art and trends, identifying gaps in existing research, and highlighting promising areas for future study. Our artifacts are publicly available at https://github.com/security-pride/LLM4SE_SLR.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Software and its engineering** → **Software development techniques**; • **Computing methodologies** → **Artificial intelligence**;

Additional Key Words and Phrases: Software Engineering, Large Language Model, Survey

ACM Reference format:
Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (December 2024), 79 pages.
https://doi.org/10.1145/3695988

1 Introduction

In the field of language processing, traditional **Language Models (LMs)** have been foundational elements, establishing a basis for text generation and understanding [294]. Increased computational power, advanced **Machine Learning (ML)** techniques, and access to very large-scale data have led to a significant transition into the emergence of **Large Language Models (LLMs)** [527, 559]. Trained with expansive and diverse data, these models have demonstrated an impressive ability to simulate human linguistic capabilities, leading to many changes across multiple work domains. With their capacity to learn from massive corpora and to generate plausible text, LLMs are blurring the line between human and machine-produced language. They provide researchers and engineers alike with powerful tools to explore the complexity and richness of human communication, consequently sparking a transformational period in the field of language processing and beyond.

Software Engineering (SE)—a discipline focused on the development, implementation, and maintenance of software systems–is one of those areas reaping the benefits of the LLM revolution [276]. The utilization of LLMs in SE primarily emerges from an innovative perspective where numerous SE challenges can be effectively reframed into code, text, repository, or developer-data analysis tasks [452, 509]. Using LLMs to address these SE tasks has shown a wealth of potential breakthroughs [33, 37, 210, 214, 215, 399, 427, 488, 489, 537, 553]. The use of LLMs is particularly influential for tasks such as requirements analysis [135, 553], which involves analyzing textual requirements from various software stakeholders during software construction or maintenance, program analysis [125, 214], which addresses imprecision and other limitations of traditional static analysis tools, code summarization [143, 443], which involves yielding an abstract **Natural Language (NL)** depiction of a code's functionality, generation of well-structured code [516] and code artifacts like annotations [245], as well as program repair [215, 570]. Codex, an LLM with 12 billion parameters, has demonstrated the ability to solve 72.31% of complex Python programming challenges posed by humans [43]. GPT-4 [320], an LLM from OpenAI, has been used with a strong performance in several SE tasks, encompassing code writing, understanding, execution, and reasoning. It not only handles real-world applications and diverse coding challenges but also shows the ability to explain results in NL and generate code from pseudocode [30].

Simultaneously, researchers have embarked on a series of research activities regarding LLM-related works, where a number of literature reviews or survey articles have been produced [36, 87, 506]. Table 1 summarizes some of the key ones as of the time of publishing this article. However,

Table 1. State-of-the-Art Surveys Related to LLMs for SE

Reference	Year	Scope of models[a]	Scope of SE tasks	SLR[b]	Time frame	# Collected articles
Yang et al. [509]	2022	DL	General SE scope	✓	2015–2020	250
Watson et al. [466]	2022	DL	General SE scope	✓	2009–2019	128
Wang et al. [452]	2022	ML, DL[c]	General SE scope	✓	2009–2020	1,209 (ML) + 358 (DL)
Zhang et al. [546]	2023	Code LLM	General SE scope	✓	2017–2023	185
Zheng et al. [565]	2023	Code LLM	General SE scope	✓	2021–2023	149
Fan et al. [85]	2023	LLM	General SE scope	×	-	Not specified
Zan et al. [527]	2023	LLM (12 M+)	NL2Code	×	2020–2023	Not specified
Wang et al. [448]	2023	LLM (117 M+)	Software testing	✓	2019–2023	52
Our work	2024[d]	LLM	General SE scope	✓	2017–2024	395

[a]"M" means million and "B" means billion. The numbers in parentheses indicate the parameter sizes of LLMs.
[b]SLR stands for Systematic Literature Review. This column denotes whether the article follows an SLR process.
[c]ML and DL refer to Machine Learning and Deep Learning, respectively.
[d]Initial draft was publicly shared in 2023 (https://arxiv.org/abs/2308.10620).

these related studies have limitations. They either focus narrowly on a single SE task, such as the application of LLMs in software testing [448] and **Natural-Language-to-Code (NL2Code)** tasks [527], or they are primarily centered on ML or **Deep Learning (DL)** models [452, 466, 509], overlooking more advanced and recently emerged LLM applications, such as ChatGPT [318], which are increasingly finding applications within the SE field [270, 400, 427, 475]. Alternatively, some merely offer a preliminary exploration of the performance of LLMs in various SE tasks through empirical experiments [74, 276, 400, 493, 522], or analyze existing partially relevant studies to reveal the challenges in this field [85] without conducting a **Systematic Literature Review (SLR)**. Furthermore, some works have investigated the application of Code LLMs in SE [546, 565], yet have not fully considered general LLMs like ChatGPT and LLaMA [431], which are also being widely applied to various SE tasks [144, 325, 381, 497]. The integration of LLMs within SE is a complex endeavor, requiring several key considerations including the choice of the right model, comprehension of the unique features of different LLMs, devising pre-training and fine-tuning strategies, handling of data, evaluation of outcomes, and surmounting implementation challenges [527]. Despite the burgeoning interest and ongoing explorations in the field, *a detailed and systematic review of LLMs' application in SE has been notably absent in the current literature.* This gap signifies a need to more deeply understand the relationship between LLMs and their use in SE. Our research aims to bridge this gap, providing valuable insights to the community.

In this article, we conduct an SLR on the utilization of **Large Language Models for Software Engineering (LLM4SE)**. By mapping the current state-of-the-art, pinpointing the key strengths, weaknesses, and gaps in the existing LLM4SE literature, and proposing potential avenues for future research, our review aims to provide researchers and practitioners with a thorough guide to the convergence of LLMs and SE. We anticipate that our findings will be instrumental in guiding future inquiries and advancements in this rapidly evolving field. This work makes the following key contributions:

—We are the first to present a comprehensive SLR including 395 articles published between January 2017 and January 2024 that focus on the use of LLM-based solutions to address SE challenges. We conducted a detailed analysis of the selected articles based on publication trends, distribution of publication venues, and so on.
—We have classified the LLMs utilized for the reported SE tasks and have provided a summary of the usage and trends of different LLM categories within the SE domain.

—We describe the reported LLM data processing stages, encompassing data collection, categorization, pre-processing, and representation.
—We discuss optimizers used for LLM4SE tasks, including tuning techniques, prevalent prompt engineering techniques, and commonly employed evaluation metrics.
—We describe the key applications of LLM4SE encompassing a diverse range of 85 specific SE tasks, grouped into six core SE activities—**Requirements Engineering (RE)**, software design, software development, software quality assurance, software maintenance, and software management.
—We summarize key challenges of using LLMs for the SE field, and have suggested several key potential research directions for LLM4SE.

Section 2 presents our **Research Questions (RQs)** and elaborates on our SLR methodology. The succeeding Sections 3 to 6 are devoted to answering each of these RQs individually. Section 7 discloses the potential threats to the validity of our study. Section 8 discusses the challenges yet to be overcome when employing LLMs to solve SE tasks and highlights promising opportunities and directions for future research. Section 9 concludes the whole article.

2 Approach

This SLR follows the methodology proposed by Kitchenham et al. [197, 198], used in most other SE-related SLRs [229, 261, 351, 452]. Following the guidelines provided by Kitchenham et al., our methodology included three main steps: planning the review (i.e., Sections 2.1 and 2.2), conducting the review (i.e., Sections 2.3 and 2.4), and analyzing the basic review results (i.e., Section 2.5).

2.1 RQs

To provide a comprehensive overview of the LLM4SE field, it is important to fully comprehend how these models are currently being applied in SE, the challenges they face, and their potential future research directions in SE. Thus, we aim to provide an SLR of the application of LLMs to SE. This study thus aims to answer the following RQs:

RQ1: What LLMs Have Been Employed to Date to Solve SE Tasks? RQ1 is designed to map out the landscape of LLMs applied in the field of SE. It seeks to identify and categorize the various LLM architectures—such as decoder-only, encoder-decoder, and encoder-only models—that have been leveraged to address diverse SE challenges. This RQ aims to provide a comprehensive overview of how these models are being utilized and the implications of their usage in this field.

RQ2: How Are SE-Related Datasets Collected, Pre-Processed, and Used in LLMs? RQ2 examines the methods used for the assembly, refinement, and application of datasets in the realm of LLMs for SE tasks. It aims to uncover the strategies for dataset collection, the criteria for dataset selection, and the pre-processing steps essential for making the data conducive for LLM training and application. Additionally, this question seeks to explore the types of data that are most prevalent in SE-related LLM research and how these data types influence the modeling and outcomes.

RQ3: What Techniques Are Used to Optimize and Evaluate LLM4SE? RQ3 explores the use of different optimization and evaluation techniques specific to LLMs in the context of SE. This includes an investigation into **Parameter Efficient Fine-Tuning (PEFT)** methods and various prompting techniques that are tailored to enhance LLM performance on SE tasks. Furthermore, this RQ aims to assess the range of evaluation metrics and methodologies employed to gauge the effectiveness and impact of LLMs in SE, providing insights into how these models are fine-tuned and assessed for their utility and efficiency.

RQ4: What SE Tasks Have Been Effectively Addressed to Date Using LLM4SE? This RQ identifies the range of SE tasks that have been successfully carried out using LLMs, offering a detailed view

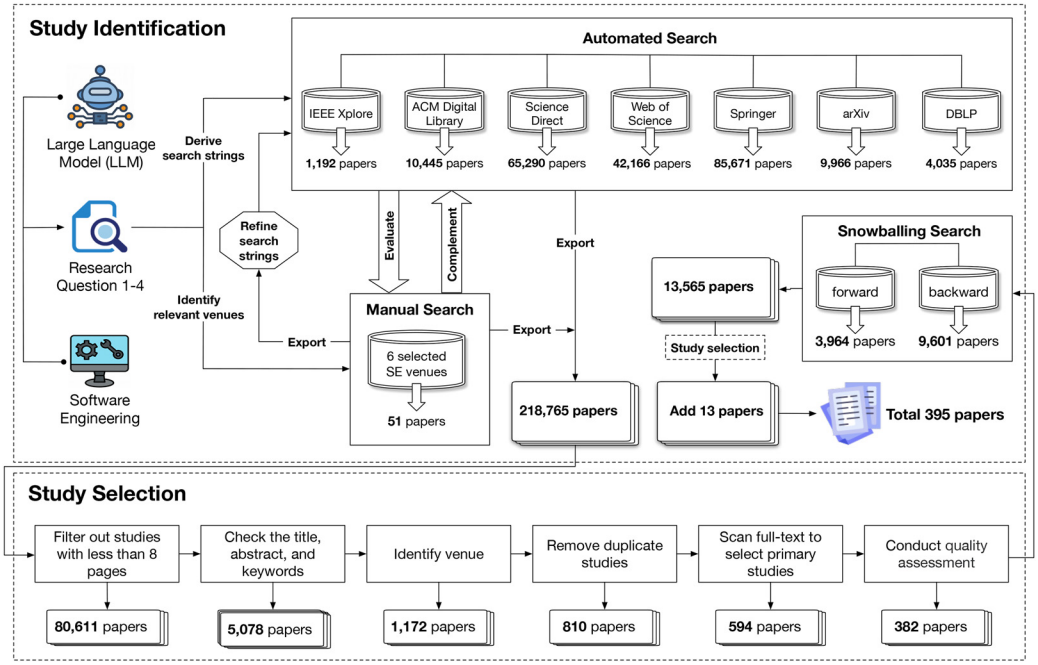


Fig. 1. Study identification and selection process.

of the application spectrum of LLM4SE. It seeks to identify the specific tasks within SE, such as code generation and program repair, where LLMs have shown significant utility, and to explore the nature and scope of these applications.

2.2 Search Strategy

As shown in Figure 1, we employed the **"Quasi-Gold Standard" (QGS)** [532] approach for article search. We conducted a manual search to identify a set of relevant studies and extracted a search string from them. This search string was then used to perform an automated search, and subsequently, a backward and forward snowballing search was employed to further supplement our search results. This approach ensures both search efficiency and maximum coverage, minimizing the risk of omission. Subsequently, we employed a series of relatively strict filtering steps to obtain the most relevant studies. We followed five steps to determine the relevance of the primary studies:

(1) Select publication venues for manual search and select digital databases for automated search to ensure coverage of all the selected venues.
(2) Establish QGS: Screen all articles for manual search and filter by inclusion/exclusion criteria (defined in Table 3).
(3) Subjectively define the search string based on domain knowledge.
(4) Conduct an automated search using the search string defined in Step (3).
(5) Conduct snowballing search after performing study selection on the results of manual search and automated search.

2.2.1 Search Items. During the manual search, we selected six of the top SE conferences and journals (i.e., ICSE, ESEC/FSE, ASE, ISSTA, TOSEM, and TSE, as shown in Table 2) and searched for articles that applied LLM4SE. We systematically crawled a list comprising 4,618 published articles

Table 2. Publication Venues for Manual Search

Acronym	Venues
ASE	International Conference on Automated Software Engineering
ESEC/FSE	Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering
ICSE	International Conference on Software Engineering
ISSTA	International Symposium on Software Testing and Analysis
TOSEM	Transactions on Software Engineering and Methodology
TSE	Transactions on Software Engineering

from the top venues. Following automated scanning via scripts, we manually verified and identified 51 articles that were relevant to our research objectives. These 51 relevant articles formed the basis for constructing the QGS. Our search string needs to combine two sets of keywords: one pertaining to SE tasks and the other related to LLMs. Only if the article contains both types of keywords, there is a higher probability that it is the article we need. The complete set of search keywords is as follows:

— *Keywords related to SE tasks: Software engineering, Software development, Program*,*[1] *Software testing, Software mainten*, SE, Software lifecycle, Software design*, Code representation, Code generation, Code comment generation, Code search, Code localization, Code completion, Code summarization, Method name generation, Bug detection, Bug localization, Vulnerability detection, Testing techniques, Test case generation, Program analysis, Bug classification, Defect prediction, Program repair, Code clone detection, Bug report, Software quality evaluation, SATD detection, Code smell detection, Compiled-related, Code review, Software classification, Code classification, Code change, Incident detection, Requirement extraction, Requirement traceability, Requirement validation, Effort cost prediction, Mining GitHub/Github mining, Mining SO (Stack Overflow)/SO mining, Mining app/App mining, Mining tag/Tag mining, Developer-based mining*
— *Keywords related to LLMs: LLM, Large Language Model*, Language Model*, LM, PLM, Pre-trained, Pre-training, Natural Language Processing, NLP, Machine Learning, ML, Deep Learning, DL, Artificial Intelligence, AI, Transformer, BERT, Codex, GPT, T5, Sequence Model*, Attention Model*, Transfer Learning, Neural Network*, ChatGPT, GPT-**

It is important to note that the list of keywords related to LLMs that we set up includes Machine Learning, Deep Learning, and other such terms that do not seem to be necessarily related to LLMs. The reason for this is that we want to avoid omitting articles related to our research as much as possible, so the process of performing automated searches expands our search scope.

2.2.2 Search Datasets. After determining the search string, we conducted an automated search across seven widely used databases, which cover all published articles and many pre-prints of under review articles. Given that the first article about the Transformer architecture [436], which forms the basis for LLMs, was published in 2017, we focused our search on articles published from that year onward.[2] Two authors independently performed the automated search, and the search results from each database were merged and deduplicated. Specifically, we obtained 1,192 articles from IEEE Xplore, 10,445 articles from the ACM Digital Library, 62,290 articles from ScienceDirect, 42,166 articles from Web of Science, 85,671 articles from Springer, 9,966 articles from arXiv, and 4,035 articles from DBLP.

[1]The * symbol serves as a wildcard, representing any characters or character sequence. For example, "Program*" can match "Program," "Programming," "Programmer," and so on.
[2]The cut-off date for the article collection process of this version is January 31, 2024.

Table 3. Inclusion Criteria and Exclusion Criteria

Inclusion criteria	
(1)	The article claims that an LLM is used.
(2)	The article claims that the study involves an SE task.
(3)	The article with accessible full text.
Exclusion criteria	
(1)	Short articles whose number of pages is less than eight.
(2)	Duplicate articles or similar studies with different versions from the same authors.
(3)	Studies belonging to books, thesis, monographs, keynotes, panels, or venues not executing a full peer-review process.
(4)	Tool demos and editorials.
(5)	The article is published in a workshop or a doctoral symposium.
(6)	The article is a grey publication, e.g., a non-refereed technical report or thesis.
(7)	Non-English written literature.
(8)	The article mentions the use of LLMs without describing the employed techniques.
(9)	The article leverages SE methods to enhance LLMs, rather than focusing on using LLMs for SE tasks.

2.3 Study Selection

2.3.1 Study Inclusion and Exclusion Criteria. Using our search strategy, we initially obtained 218,765 articles that potentially relate to our research. Next, we needed to further evaluate the relevance of these articles based on inclusion and exclusion criteria (to ensure that our inclusion and exclusion criteria were sufficiently objective and rational, we designed these criteria following several state-of-the-art SLR articles [303, 452, 466, 509]), as shown in Table 3, so that the selected articles can directly address our RQs. The article selection process, as illustrated in Figure 1, consists of six phases. In the first phase, we conducted automated filtering to exclude articles with less than eight pages [23, 452] (Exclusion criteria 1), reducing the number of articles to 80,611.

In the second phase, we examined the titles, abstracts, and keywords of the articles to identify those that include relevant LLM-related keywords. We then expanded the search scope to avoid missing relevant articles, including ML, DL, and other related keywords that may not directly correspond to LLM. The purpose of this phase is to narrow down the scope and filter out articles directly related to LLM (Inclusion criteria 1). Articles that are filtered out in this phase are then manually reviewed in the fifth phase. Additionally, we excluded 448 non-English written literature (Exclusion criteria 7). After the second phase, the number of articles was reduced to 5,078.

The third phase involves identifying the venues of the articles (Exclusion criteria 3). We extracted publication information such as "journal," "URL," "DOI," and "series" to determine the publication sources. For articles from arXiv in 2023 and 2024, we chose to retain them, considering that this field is emerging and many works are in the process of submission. Although these articles did not undergo peer review, we have a quality assessment process to eliminate articles with low quality. This step resulted in 1,172 articles.

In the fourth phase, we merged and deduplicated the remaining articles from the seven databases and the manually searched article list (Exclusion criteria 2), resulting in 810 articles. We then reviewed the full texts of the articles and excluded 190 articles that were grey publications or were published in workshops or doctoral symposiums (Exclusion criteria 4, 5, 6). By further assessing the quality of the articles, we identified 382 articles directly relevant to our research. This phase primarily involved excluding articles that mentioned LLMs but did not directly apply them, such as articles that only discussed LLMs in future work or focused on evaluating the performance of LLM-enabled tools [448] (Exclusion criteria 8). For systematic views, survey, and review articles, we have retained them and will assess their content during the quality assessment phase to determine their relevance to our research.

Table 4. Checklist of Quality Assessment Criteria (QAC) for LLM Studies in SE

ID	QAC
QAC1	Is the study relevant to SE tasks?
QAC2	Does the study utilize LLMs?
QAC3	Is the research not a secondary study, such as an SLR, review, or survey?
QAC4	Was the research published in a high-repute venue?
QAC5	Is there a clear motivation for the research?
QAC6	Does the study provide a clear description of the techniques used?
QAC7	Are the experimental setups, including experimental environments and dataset information, described in detail?
QAC8	Does the study clearly confirm the experimental findings?
QAC9	Are the key contributions and limitations of the study discussed?
QAC10	Does the study make a contribution to the academic or industrial community?

2.3.2 Study Quality Assessment. A well-crafted quality assessment can help to prevent biases introduced by low-quality studies and can indicate to readers where caution about conclusions should be drawn [508]. We formulated 10 **Quality Assessment Criteria (QAC)**, as shown in Table 4. These aim to assess the relevance, clarity, validity, and significance of included articles. We used a scoring system of −1, 0, 1 (irrelevant/unmet, partially relevant/met, relevant/fully met). The first three questions were designed for the remaining 382 articles in the fifth stage. If QAC1, QAC2, or QAC3 received a score of −1, there is no need to proceed with QAC4–QAC10, and the article can be excluded directly. QAC4–QAC10 involved assessing the content of the articles using a scoring system of 0, 1, 2, 3 (poor, fair, good, excellent). Finally, we calculated the total score of QAC4–QAC10 for each article. For published articles, the maximum score for QAC4–QAC10 should be 21 (3×7). We retained articles with a score of 16.8 (21×0.8) or above. For unpublished articles on arXiv, the score for QAC4 is always 0, and the maximum score for QAC5–QAC10 should be 18 (3×6). We retained articles with a score of 14.4 (18×0.8) or above. After this quality assessment, we obtained a final set of 382 articles.

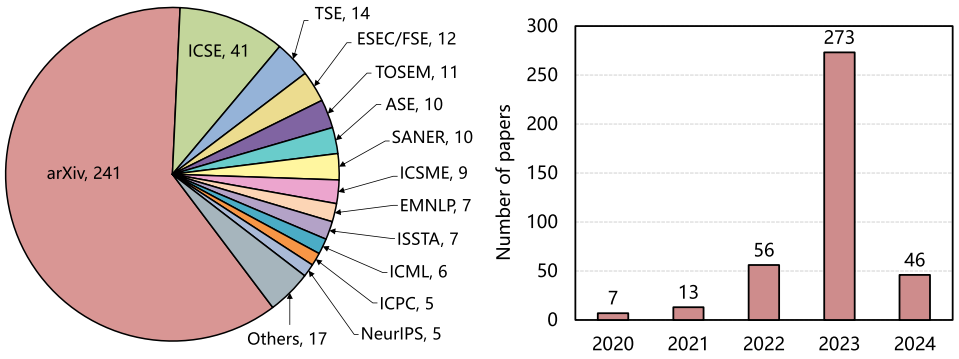
2.4 Snowballing Search

To identify any additional possibly relevant primary studies, we conducted a snowballing search. Snowballing refers to using the reference list of a article or the citations to the article to identify additional articles. Snowballing can benefit us by not only looking at the reference lists and citations but also looking at where articles are actually referenced and where articles are cited. Using the references and the citations, respectively, is referred to as backward and forward snowballing.

Before conducting snowballing, a set of initial articles needs to be prepared. In this study, the initial article list consists of the remaining 382 articles after the quality assessment. We performed forward and backward snowballing, which resulted in the collection of 3,964 and 9,610 articles, respectively. After initial deduplication, we were left with 5,152 articles. We then conducted the full study selection process on these 5,152 articles, including deduplicating them with the 382 articles from performing snowballing on the initial list. As a result, we obtained an additional 13 articles.

2.5 Data Extraction and Analysis

We thus finally obtained 395 relevant primary study articles after searching and snowballing. Figure 2 presents an overview of the distribution of the included articles. As shown in Figure 2(a), 154 articles are published in peer-reviewed venues. ICSE is the most common of these venues, with a contribution of 41 articles. Other venues with noteworthy contributions include TSE, ESEC/FSE, and TOSEM, contributing 14, 12, and 11 articles, respectively. The remaining 241 articles are published on arXiv, an open-access platform that serves as a repository for scholarly articles. This finding is



(a) Distribution of papers across venues. (b) Distribution of papers over years.

Fig. 2. Overview of the selected 395 articles' distribution.

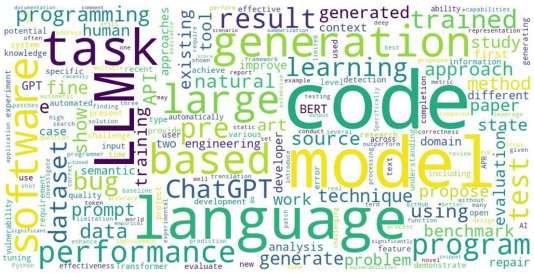


Fig. 3. Topics discussed in the collected articles.

not surprising since much new LLM4SE research is rapidly emerging and thus many works are just completed and are likely in the peer review process. Despite the non-peer-reviewed nature of these articles, we have performed a rigorous quality assessment process on all collected articles, to ensure the quality of validity of our findings. This approach allows us to include all high-quality and relevant publications while maintaining high research standards.

Figure 2(b) shows the temporal distribution of the included articles. The number of publications has seen a rapidly growing trend since 2020. In 2020 and 2021, there are only 7 and 13 relevant articles, respectively. However, by 2022, the number of articles has increased dramatically to 56. What's surprising is that, in 2023 alone, the number of published articles has already reached 273. And within just one month in 2024, 46 relevant articles are published. This rapid growth trend demonstrates that there is a growing research interest in the domain of LLM4SE.

In order to visualize the main content of our collection of articles, we generated a word cloud based on the abstracts of 395 articles as shown in Figure 3. The most frequently occurring words include "code," "LLM," "language," "model," "large," "task," "software," "generation," "performance," and "program," clearly indicating the main themes explored in these articles. The terms "code" and "software" emphasize the core elements of software engineering, while "LLM," "large," "language," and "model" denote the use of large language models in a variety of tasks. The terms "generation." "task." and "program" emphasize the use of the LLM for automatic code generation and other SE tasks. In addition, "performance" reflects the evaluation and assessment of the effectiveness of LLM in SE applications. The word cloud provides further visual evidence that the literature we have collected is closely related to our research topic.

Table 5. Extracted Data Items and Related RQs

RQ	Data item
1,2,3,4	The category of SE task
1,2,3,4	The category of LLM
1,4	Characteristics and applicability of LLMs
2	The adopted data handling techniques
3	The adopted weight training algorithms and optimizer
3	The selected evaluation metrics
4	The SE activity to which the SE task belongs
4	The developed strategies and solutions

We then conducted data extraction during the full-text review. This extraction phase collected all relevant data that would facilitate a comprehensive and insightful response to the RQs outlined in Section 2.1. As depicted in Table 5, we extracted data including the classification of SE tasks, their corresponding activities, as well as the category, characteristics, and applicability of the LLMs. With these collected data, we systematically analyzed the relevant aspects of LLM4SE.

3 RQ1: What LLMs Have Been Employed to Date to Solve SE Tasks?

3.1 LLMs

Pre-Trained Language Models (PLMs) have demonstrated impressive capabilities in solving various **Natural Language Processing (NLP)** tasks [202, 380, 468, 559]. Researchers have observed that scaling up the model sizes significantly enhances their capacity, leading to remarkable performance improvements when the parameter scale surpasses a certain threshold [137, 380, 422]. The term "Large Language Model" was introduced to distinguish LMs based on their parameter size, specifically referring to large-sized PLMs [559]. However, we note that the literature lacks a formal consensus on the minimum parameter scale for LLMs, as the model's capacity is intertwined with both data size and total compute [448]. In this article, we adopt the LLM scope division and taxonomy introduced by Pan et al. [326] and categorize the mainstream LLMs investigated in this study into three groups according to their architectures: encoder-only, encoder-decoder, and decoder-only LLMs. This taxonomy and relevant models are shown in Figure 4. We have included the LLMs used by each work and their parameter sizes (if declared in the article) in our public repository: https://github.com/securitypride/LLM4SE_SLR. Additionally, Table 6 summarizes the LLMs with different architectures suitable for different types of SE tasks.

Encoder-Only LLMs, such as BERT [65] and its variants [92, 118, 211, 260], use only the encoder component to process and encode input into hidden representations, capturing word relationships and context. BERT's bidirectional attention mechanism considers both the left and right context of each word. In SE, models like CodeBERT [92], GraphCodeBERT [118], RoBERTa [260], and ALBERT [211] are widely used. Specialized models such as BERTOverflow [415] and CodeRetriever [234] have been developed for SE applications, leveraging program structure and novel pre-training tasks. For example, CodeBERT uses token prediction to enhance the understanding of programming languages for tasks like code completion and bug detection [92]. GraphCodeBERT incorporates edge-type prediction to understand code structure, aiding in tasks like code summarization and analysis [118]. These models excel in tasks requiring a nuanced understanding of sentences or code snippets, such as code review, bug report comprehension, and named entity recognition related to code entities [19, 231, 298, 343, 379, 502].

Encoder-Decoder LLMs incorporate both encoder and decoder modules [436]. The encoder ingests the input sentence and encodes it into a hidden space, effectively capturing the underlying structure and semantics. This hidden representation serves as an intermediary language, bridging the gap between diverse input and output formats. Conversely, the decoder utilizes this hidden

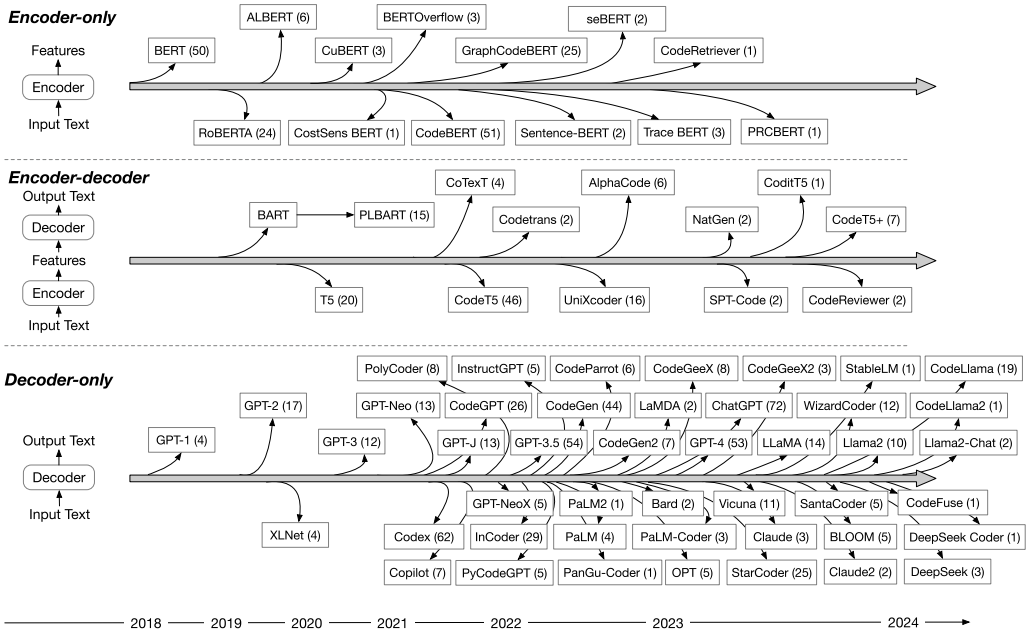


Fig. 4. Distribution of the LLMs (as well as LLM-based applications) discussed in the collected articles. The numbers in parentheses indicate the count of articles in which each LLM has been utilized.

Table 6. Summary of LLMs with Different Architectures Used in SE Tasks

Model	Type	Example of SE tasks
Encoder-only	Understanding	Code understanding Bug localization Vulnerability detection
Encoder-decoder	Understanding and generation	Code summarization Code translation Program repair
Decoder-only	Generation	Code generation Code completion Test case generation

space to generate the target output text, translating the abstract representation into concrete and contextually relevant expressions. Models such as PLBART [5], T5 [349], and CodeT5 [464] embody this architecture. Further advancements are evident in CodeT5+ [461], while AlphaCode [237] and CoTexT [337] showcase the architecture's adaptability to various SE tasks. The encoder-decoder design offers flexible training strategies and is proficient in handling multifaceted tasks such as summarization, translation, and question-answering. Within the field of SE, this ability has been successfully applied to tasks like code summarization [9, 115], code translation [164, 505], and program repair [284, 365]. The encoder module's capacity to understand and represent both the structure and semantics of code is pivotal, allowing the decoder to translate this comprehension into concise, human-readable summaries.

Decoder-Only LLMs, such as the GPT series (GPT-1 [347] to GPT-4 [320]) and their derivatives like ChatGPT [318], generate output text through sequential token prediction without relying on an encoder. They have been used in various SE tasks, with specialized versions like CodeGPT [269],

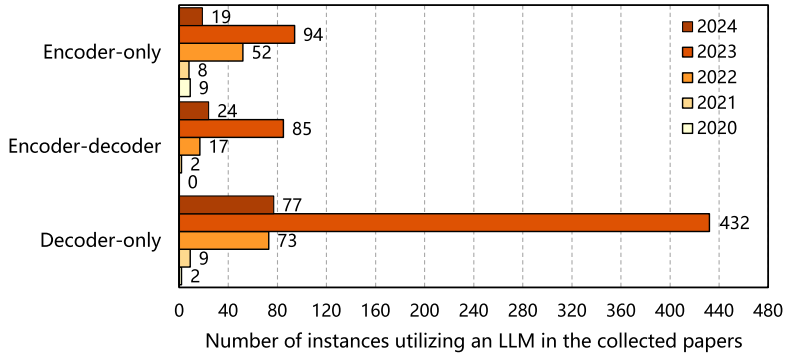


Fig. 5. Trends in the application of LLMs with different architectures in SE tasks over time.

InstructGPT [321], Codex [43], and Copilot [109] being fine-tuned for specific SE applications. Open-source models like GPT-J [444], GPT-Neo [28], GPT-NeoX [27], LLaMA [431], and Vicuna [52] also follow this architecture. Decoder-only LLMs are usually more suitable for various generation tasks, such as code generation and code completion. These models can generally perform downstream tasks from a few examples or simple instructions without adding prediction heads or fine-tuning, making them valuable tools in SE research; *2022 marked a surge in the development of decoder-only LLMs, a trend that gained further momentum in 2023, notably with the launch of commercial products by leading Internet companies.* For example, Google launched Gemini [112], Meta introduced LLaMA [431] and Llama 2 [432], and Anthropic unveiled Claude [18]. Contrary to LLMs such as GPT-4 and its derivative application, ChatGPT, released by OpenAI, which were promptly integrated into SE tasks, these new additions have not yet found widespread application within the SE field. Their potential remains largely unexplored, with opportunities for further assessment and utilization in specific tasks and challenges. The continued advancement of these models emphasizes the active exploration and innovation within decoder-only architectures.

3.2 Trend Analysis

As shown in Figure 5, in the span from 2020 to 2024, the architecture of LLMs has witnessed notable shifts in preference and application within SE tasks. The specific choices between decoder-only, encoder-decoder, and encoder-only structures have shaped the direction of research and solutions in the SE domain [478]. This analysis explores trends in the adoption of these architectures over the years, reflecting the evolving dynamics of LLM for SE tasks.

Evolution of LLM Architectures in 2021. The year 2020 saw research articles predominantly concentrating on encoder-only LLMs for SE tasks, evidenced by a total of eight articles. Decoder-only LLMs or encoder-decoder LLMs were scarcely featured in that year's research. A marked change occurred in 2021. Out of 19 articles in 2021, nine were dedicated to decoder-only LLMs, constituting 47.37% of the research. Additionally, two articles, or 10.53%, focused on encoder-decoder LLMs. Encoder-only LLMs witnessed a slight decline, representing 42.1% of the field with eight articles. This rapid transition can be linked to the generative capability of decoder-only LLMs. Researchers [212, 368, 400] found that these models, e.g., GPT series, requiring minimal fine-tuning, could produce not only syntactically correct but also functionally relevant code snippets. Their proficiency in grasping the context of code quickly made them a preferred choice.

Diversity of LLM Architectures in 2022. The year 2022 experienced a significant increase in diversity, with more varied LLM architectures finding representation. Out of a total of 142 articles, 73 were centered around decoder-only LLMs, comprising 51.41% of the studies. Encoder-decoder

LLMs made their presence known in 17 articles, accounting for 11.97%. Meanwhile, encoder-only LLMs led the field slightly with 52 articles, capturing 36.62% of the research interest. This diverse distribution suggests an exploration phase where researchers were actively assessing and leveraging different architectures to suit varied needs and challenges. The near-equal interest across different architectures underscores the field's richness, indicating that no single approach had become the definitive choice.

Dominance of the Decoder-Only Architecture in 2023. The year 2023 signaled a strong shift toward decoder-only LLMs. A very large 432 instances of utilizing decoder-only LLMs were recorded across 195 unique articles , reflecting that a single article might employ multiple such models. These articles focusing on decoder-only LLMs constituted a significant 70.7% of the total research articles in our selected primary studies this year. In comparison, encoder-decoder LLMs were the subject of 85 articles, contributing 13.91%, while encoder-only LLMs appeared to stabilize, with 94 articles, representing 15.39% of the 2023 research landscape. This trend signifies a shift in focus and resources toward exploring and harnessing the decoder-only architecture as the primary approach in many current and future LLM4SE research and applications.

Exploration of the LLM Architecture in 2024. The initial trends in January 2024 showcase the ongoing evolution of LLM architectures. Among the 120 articles examined, decoder-only LLMs continued to maintain a prominent position, with 77 articles dedicated to this architecture, constituting 64.17% of the research. Encoder-decoder LLMs appeared in 24 articles, representing 20% of the total, while encoder-only LLMs were featured in 19 articles, making up 15.83%. Although there is a slight decrease in the dominance of decoder-only architectures compared to the previous year, they still hold a central role. The persistent exploration of encoder-decoder and encoder-only architectures suggests an enduring interest in diverse configurations within the SE research community.

Criteria for LLM Selection in SE Tasks. The selection of an LLM for SE tasks should involve careful consideration rather than arbitrary choice. Key factors guiding this selection encompass the model's proficiency in understanding the context of code, its ability to generate relevant content, responsiveness to fine-tuning, and demonstrated performance on SE-specific benchmarks [224, 238, 491]. Given the stringent syntactical rules and functional requirements inherent to SE tasks, models capable of seamlessly integrating these complex aspects were typically favored.

Task-Specific Fine-Tuning. A notable trend is the customization of LLMs for precise SE tasks [160, 231, 536]. By fine-tuning models with datasets tailored to specific functions such as bug detection or code review, researchers were able to achieve marked performance improvements [56, 204].

In conclusion, the evolution of LLMs for SE, transitioning from encoder-only to decoder-only architectures, highlights the field's vibrancy and adaptability. This shift has fundamentally altered the approach to SE tasks, reflecting the ongoing innovation within the discipline.

RQ1—Summary

(1) There are more than 70 different LLMs used for SE tasks in our selected primary studies. Based on the underlying architecture or principles of different LLMs, we classified the summarized LLMs into three categories, i.e., decoder-only, encoder-decoder, and encoder-only LLMs.
(2) We observed that each LLM architecture serves a specific purpose in SE tasks, with encoder-only LLMs focusing on comprehensive understanding, encoder-decoder LLMs used for tasks requiring understanding of input information followed by content generation, and decoder-only LLMs being more suitable for generation tasks.
(3) We analyzed the trend of LLM usage for SE tasks. The most widely used LLMs are with decoder-only architectures. There are over 45 LLMs in the decoder-only category and 195 articles have researched the application of decoder-only LLMs to SE tasks.

4 RQ2: How Are SE-Related Datasets Collected, Pre-Processed, and Used in LLMs?

Data play a crucial role in the LLM training phase [413]. First, data are collected to obtain diversity and richness to ensure that the model can cope with different scenarios and situations. Second, data are classified to clarify the training objectives of the model and avoid confusion and misinformation. The pre-processing of data is indispensable to clean and transform the data to improve its quality. Finally, data are formatted into a structure suitable for model processing, allowing the LLM to learn the data's features and patterns effectively. We analyze the reported processes of data collection, data classification, data pre-processing, and data representation in our selected primary studies on LLM4SE.

4.1 How Are the Datasets for Training LLMs Sourced?

Data are an indispensable and critical factor in training LLMs, which determine the generalization ability, effectiveness, and performance of the models [413]. Adequate, high-quality, and diverse data are critical to allow models to fully learn features and patterns, optimize parameters, and ensure reliability in validation and testing. We first investigate the methods used to obtain the dataset. By analyzing the methods of data collection, we divided the data sources into four categories: open-source datasets, collected datasets, constructed datasets, and industrial datasets. *Open-source datasets* [38, 189, 449, 529] refer to publicly accessible collections of data that are often disseminated through open-source platforms or repositories. For example, datasets like HumanEval [43], which consists of 164 manually crafted Python problems, each accompanied by its respective unit tests. The open-source nature of these datasets ensures their credibility and allows for community-driven updates, making them a reliable resource for academic research. *Collected datasets* [149, 286, 379, 427] are those that researchers compile directly from a multitude of sources, including but not limited to, major Web sites, forums, blogs, and social media platforms. For instance, researchers [35, 372, 473, 502] often scrape data from **Stack Overflow (SO)** [323] threads or GitHub [108] issues comments to create a dataset tailored to their specific RQs. *Constructed datasets* [83, 185, 201, 533] are specialized datasets that researchers create by modifying or augmenting collected datasets to better align with their specific research objectives. These modifications can be carried out through manual or semi-automatic methods and may include the generation of domain-specific test sets, annotated datasets, or synthetic data. For example, researchers often take a collected dataset of code snippets and manually annotate them with bug types to create a constructed dataset for studying **Automated Program Repair (APR)** techniques [88, 173, 483]. *Industrial datasets* [11, 291, 462] are those obtained from commercial or industrial entities and often contain proprietary business data, user behavior logs, and other sensitive information. These datasets are particularly valuable for research that aims to address real-world business scenarios. However, the acquisition of such datasets is often complicated by issues related to business confidentiality and data privacy. For example, in a collaborative effort with **China Merchants Bank (CMB)**, Wang et al. [462] were able to access 21 projects from CMB's repositories. Access to such data would likely require non-disclosure agreements and other legal safeguards to protect business interests. Each of these dataset types offers unique advantages and challenges, and the choice between them should be guided by the specific requirements and constraints of the research project at hand.

Figure 6 shows the collection strategies of LLM-related datasets. As can be seen from the data in the figure, *235 studies used open-source datasets for training LLMs.* One of the main reasons for using open-source datasets in LLM training is their authenticity and credibility. Open-source datasets usually contain real-world data collected from various sources (such as relevant studies that have been conducted), which makes them highly reliable and representative of real-world scenarios. This helps LLMs learn from real examples to better understand real-world applications

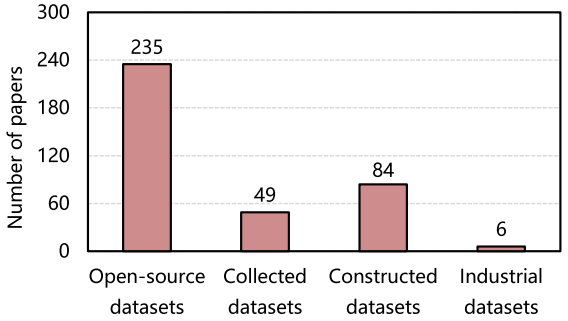


Fig. 6. The collection strategies of LLM-related datasets.

and improve their performance. Second, since LLMs are a topic that has just recently emerged, a lack of suitable training sets does exist. Therefore, researchers often collect data from sites such as SO and GitHub and build datasets to make the data more composite for SE tasks. *Among the 395 articles we studied, we discovered that only six studies utilized industrial datasets.* This suggests a potential misalignment between the properties of datasets used in academic research and those encountered in real-world industrial contexts. This divergence underscores the need for future research to investigate industrial datasets, thereby ensuring that LLMs are applicable and robust across both academic and industrial scenarios.

Note that some studies use multiple datasets that span different categories, e.g., Xu et al. [493] evaluated the performance of Codex, GPT-J, GPT-Neo, and other LLMs on SE tasks, and Mastropaolo et al. [288] investigated the use of T5 in several code-related tasks such as fixing bugs and generating code comments. For different LLMs or different SE tasks, researchers may use different training datasets. On the other hand, some articles focus on exploring how existing LLMs (e.g., ChatGPT) are used in SE tasks [475] and do not specify the dataset used for model training, as these LLMs like ChatGPT often do not require users to prepare training data themselves for general usage scenarios.

4.2 What Types of SE Datasets Have Been Used in Existing LLM4SE Studies?

Data types play a pivotal role in shaping the architecture and selection of LLMs, as they directly influence the extraction of implicit features and subsequent model decisions [35, 106, 390, 504]. The choice of data types can significantly impact the overall performance and generalization ability of the LLMs. We examine and classify the types of SE datasets employed in LLM4SE studies. By investigating the relationship between data types, model architectures, and performance, we seek to shed light on the critical role of data types in the success of LLM4SE applications.

Data Type Categorization. We classified the data types of all datasets into five categories: code-based, text-based, graph-based, software repository-based, and combined data types. Table 7 describes the specific data included in the data types corresponding to the datasets we summarized from the 395 studies. We can find that *most of the studies used text-based datasets, accounting for a total of 151.* The dominance of text-based datasets in training LLMs for SE tasks highlights the models' exceptional NL processing capabilities. These LLMs excel in understanding and processing textual data, making them an ideal choice for tasks that involve code comprehension, bug fixing, code generation, and other text-oriented SE challenges. Their ability to process and learn from vast amounts of text data enables them to provide powerful insights and solutions for various SE applications.

Table 7. Data Types of Datasets Involved in Prior Studies

Category	Data type		Total
Text-based datasets	Programming tasks/problems (42)	Prompts (33)	151
	SO posts (12)	Bug reports (11)	
	Requirements documentation (9)	APIs/API documentation (8)	
	Q&A pairs (6)	Vulnerability descriptions (4)	
	Reviews (4)	Logs (3)	
	Methods (3)	Project issues (3)	
	Code comments (2)	Theorems (2)	
	Buggy text (1)	Dockerfiles (1)	
	Outage descriptions (1)	Semantic merge conflicts (1)	
	Site text (1)	Software development tasks (1)	
	User intents (1)	Software specifications (1)	
	User reviews (1)		
Code-based datasets	Source code (60)	Bugs/Buggy code (16)	103
	Vulnerable source code (8)	Patches (4)	
	Code changes (3)	Test suites/cases (3)	
	Bug-fix pairs (2)	Error code (2)	
	Error-fix pairs (1)	Flaky test cases (1)	
	Identifiers (1)	Labeled clone pairs (1)	
	Packages (1)		
Graph-based datasets	GUI images (1)		1
Software repository-based datasets	Code repository (9)	Android apps (3)	20
	Issues and commits (3)	Pull-requests (2)	
	Industrial projects (1)	Open-source projects (1)	
	Web applications (1)		
Combined datasets	Programming tasks and test suites/cases (17)	Source code and comments (12)	55
	Programming tasks and solutions (8)	Source code and description (3)	
	Code-text pairs (2)	Source code and API usage sequences (2)	
	Source code and test suites/cases (2)	Bug report and test suites/cases (1)	
	Buggy code and comments (1)	Buggy code and solutions (1)	
	Code files and summaries (1)	Binary code and related annotations (1)	
	Failing test code and error messages (1)	Source code and Q&A pairs (1)	
	Source code, methods, and logs (1)	Vulnerable code and description (1)	

See Appendix A for the full table including references.

The most prevalent type of data utilized in training LLMs for SE tasks is programming tasks/problems with 42 instances observed among the surveyed articles. This dominance can be attributed to the diverse and challenging nature of programming problems, which provide LLMs with opportunities to generalize knowledge and skills across various SE challenges, fostering a robust understanding of software concepts and enhancing performance across a wide range of tasks, including code generation, code completion, and code summarization, etc. Prompts follow closely behind programming tasks, with 33 instances observed in the surveyed articles, providing task-specific guidance to LLMs, serving as cues or instructions for the models, and helping them understand the context and requirements of SE tasks. This combination helps the models develop a robust understanding of software concepts and perform well in a wide range of tasks. There are also SO posts (12), bug reports (11), and so on, which are among the more numerous data types in text-based datasets.

The predominance of source code (60) as the most abundant data type in code-based datasets can be attributed to its fundamental role in SE. Source code serves as the foundation of any software project, containing the logic and instructions that define the program's behavior. Therefore, having a large volume of source code data is crucial for training LLMs to understand the intricacies of

Fig. 7. The data pre-processing procedure for text-based datasets.

software development, enabling them to effectively generate, analyze, and comprehend code in various SE tasks. There are also common data types, such as bugs/buggy code (16) and patches (4), for program repair tasks. Additionally, vulnerable source code (8) is used for vulnerability detection tasks. Graph-based datasets are used in some research studies for SE tasks, e.g., Kolthoff et al. [203] used a dataset composed of screenshots from Google Play Android applications to construct a GUI repository in their study on LLM for the rapid prototyping task. These datasets represent code using graph structures, capturing relationships and dependencies between code components.

Software repository-based datasets are compilations of data extracted from version control systems, such as Git repositories, containing code, documentation, and related artifacts. This data includes Code repository (3), issues and commits (3), and so on. The data in software repositories can provide a wealth of information covering all aspects of the software development process, including code evolution history, records of issue fixes and feature improvements, code quality assessments, and so on. These data are valuable for studying behaviors and trends in the software development process, improving software quality and development efficiency, and evaluating the performance of SE techniques. Therefore, many studies have used software repository-based datasets for empirical analysis and model training.

Some studies employed combined datasets containing multiple datatypes. Among them, the most common type is "programming tasks and test suites/cases." Other combinations of data types include "source code and comments," "programming tasks and solutions," "source code and description," "code-text pairs," and so on.

4.3 How Do Data Types Influence the Selection of Data-Pre-Processing Techniques?

For the training and application of LLMs, the raw dataset needs to be subjected to data processing to obtain a clean and suitable dataset for model training. The data processing steps [216, 280] involve operations such as data cleaning, noise removal, normalization. To ensure consistency and quality of the data, different data types may require different processing methods to improve the performance and effectiveness of LLMs in SE tasks. In this section, we aim to detail the data pre-processing procedures for the two most used types of datasets, i.e., text-based datasets and code-based datasets.

The Data Pre-Processing Procedure for Text-Based Datasets. As shown in Figure 7, the steps of text-based dataset pre-processing consist of seven steps in total, yet there are some differences from the code-based dataset pre-processing steps. The process begins with data extraction [55, 56, 83, 504], where relevant text is carefully extracted from SE documentation from a variety of sources, including bug reports [56], requirements documents [203], code comments [342], and API documentation [190]. This step ensures that the dataset captures diverse, task-specific textual information. After data extraction, the text is initially segmented and categorized according to the specific requirements of the research task. For example, the text can be segmented into sentences or further broken down into individual words as needed for analysis [129, 204]. To ensure the quality and relevance of the dataset, substandard data deletion is performed to eliminate any invalid or irrelevant text. For example, the dataset used by Lee et al. [216] was constructed from bug reports, and in the "low-quality data deletion" process, the researchers filtered out bug reports with fewer than 15 words because the text was too short to contain contextual information. Next,

Fig. 8. The data pre-processing procedure for code-based datasets.

pre-processing operations are performed on the text to standardize and clean it. Common pre-processing steps include removing certain symbols, stop words, and special characters [350, 462]. This standardized form of text facilitates the efficient processing of LLMs. To avoid introducing bias and redundancy in the dataset, researchers eliminated duplicate instances by removing any duplicate text samples [129, 204, 493]. This step enhances the diversity of the dataset and helps the model to generalize better to new inputs. "Data tokenization" is a key step in preparing the text for LLMs [272]. Text is labeled into smaller units, such as words or subwords, so that LLMs are easier to manage and process efficiently. Finally, the pre-processed dataset is partitioned into different subsets, usually including a training set, a validation set, and a test set.

The Data Pre-Processing Procedure for Code-Based Datasets. We now summarize the process of pre-processing a code-based dataset, which consists of seven steps. Figure 8 describes the individual data processing steps in detail and gives examples. The first step is data extraction, which involves retrieving relevant code segments from different sources such as software repositories or version control systems [183, 504]. Depending on the requirements of the research task [288, 523], code segments can be extracted at different levels of granularity, ranging from individual methods and functions to entire source code files or even complete software projects. The next step is to remove any code segments that do not meet predefined criteria or quality standards [223, 342, 390]. This filtering process ensures that the extracted code is relevant to the specific SE task under study, thus eliminating incomplete or irrelevant code snippets. To avoid introducing bias and redundancy during model training, the third step involves removing duplicate instances [57, 493, 561]. Any duplicate code instances are identified and removed from the dataset, thus increasing the diversity and uniqueness of the data. After the data extraction and filtering steps, the fourth step, data compilation, comes into play. The extracted and filtered code segments are merged and compiled into a unified code dataset. This compilation process simplifies data storage and access and facilitates subsequent analysis and model training [35, 284]. In the fifth step, the problem of invalid or non-executable code is solved by removing data that cannot be compiled. Any code segments that cannot be compiled or executed are removed from the dataset to ensure that the remaining code instances are valid and usable during model training and evaluation. The sixth step is code representation, which consists of converting the code segments into a suitable representation that can be processed by the LLMs. This conversion can take different forms: token-based representation involves tokenizing the source or binary code into distinct tokens; tree-based representation parses the code into Abstract Syntax Trees; and graph-based representation generates a Program Dependence Graph, encompassing **Control Flow Graphs (CFG)** and Call Graphs. Finally, in the "data segmentation" step, the pre-processed dataset is partitioned into different subsets for training, validation, and testing [57, 473]. The training set is used to train the LLM, the validation set helps to tune the hyper-parameters and optimize the model performance, and the testing set evaluates the model's ability on unseen data. By strictly adhering to these seven pre-processing steps, researchers can create structured and standardized code-based datasets, thus facilitating the effective application of LLMs for a variety of SE tasks such as code completion, error detection, and code summarization.

It is worth emphasizing that the order of these steps is not fixed and can be adjusted based on the specific research task and its associated requirements. Researchers need to carefully consider the

Table 8. The Various Input Forms of LLMs Proposed in Prior Studies

Category	Input forms		Total
Token-based input	Text in tokens (150) Code and text in tokens (78)	Code in tokens (118)	347
Tree/graph-based input	Code in tree structure (2)	Code in graph structure (3)	5
Pixel-based input	Pixel (1)		1
Hybrid-based input	Hybrid input forms (2)		2

See Appendix B for the full table including references.

objectives, characteristics of the dataset, and the desired outcomes when determining the optimal sequence for these pre-processing techniques.

4.4 What Input Formats Are the Datasets for LLM Training Converted to?

Once suitable datasets have been carefully chosen and clean data have been achieved through the pre-processing steps, the next critical aspect is the transformation of the data into appropriate formats that can effectively serve as inputs for LLMs. Table 8 shows four distinct data input types that emerged during the research: Token-based input, Tree/Graph-based input, Pixel-based input, and Hybrid-based input. We now detail each as follows:

Token-Based Input. Token-based input [7, 9, 19] involves representing code and text as sequences of tokens, which are smaller units like words or subwords. Text in tokens refers to the tokenization of textual data, such as documentation, bug reports, or requirements, enabling the LLMs to process and analyze NL descriptions effectively. Code and text in tokens combine both code and its associated textual context, allowing the model to capture the relationships between code elements and their descriptions. Code in tokens refers to the representation of code snippets broken down into meaningful tokens, allowing the LLMs to understand programming language syntax and semantics at a fine-grained level.

Tree/Graph-Based Input. Tree-based input [276, 316, 556] represents code as hierarchical tree structures, capturing the syntactic relationships between code elements. Each node in the tree represents a code element, and the edges represent the hierarchical nesting of control flow statements and other code structures. This form of input allows the LLMs to understand the code's hierarchical structure and perform tasks like code completion and bug fixing. Graph-based input represents code as a graph structure, where nodes represent code elements and edges represent the relationships between them. Unlike trees, graphs allow more flexible and complex relationships between code elements, enabling the model to capture non-linear dependencies in the code. This form of input is used in tasks like code summarization and vulnerability detection by considering the code's intricate relationships.

Pixel-Based Input. Pixel-based input [302] visualizes code as images, where each pixel represents a code element or token. This visual representation allows the LLMs to process and understand code through image-based learning. In this input form, LLMs learn from the visual patterns and structures in the code to perform tasks like code translation or generating code visualizations.

Hybrid-Based Input. Hybrid-based input [314] combines multiple modalities to provide LLMs with diverse perspectives for better code comprehension. For example, a hybrid input may combine code in tokens with visual representations of code, allowing the model to learn both from the fine-grained details in the tokenized code and from the overall visual structure of the code. This approach enhances the model's ability to understand complex code patterns and improve performance in tasks such as code comprehension and code generation.

During our investigation of LLM-based models for SE tasks, we observed distinct trends in the usage of different input forms during the training process. *Token-based input forms, namely code in tokens and text in tokens were the most prevalent, collectively constituting approximately 97.75% of the studies.*[3] Specifically, code in tokens was widely adopted in 118 studies, accounting for approximately 33.24% of the total studies, demonstrating its popularity as a primary choice for representing code snippets. This approach allowed LLMs to grasp programming language syntax and semantics effectively, making it suitable for a wide range of code-related tasks. Similarly, text in tokens was utilized in 150 studies, comprising around 42.25% of the total studies. This input form allowed LLMs to process NL descriptions, bug reports, and documentation with greater efficiency and accuracy. The popularity of token-based input forms underscores their significance in leveraging the power of LLMs for SE applications.

In contrast, *tree/graph-based input forms, such as code in tree-structure, were used in only seven studies, making up approximately 1.4% of the total.* Although less prevalent, this input type emerged as a promising choice to represent the hierarchical structure and syntactic relationships within code. Its adoption indicated an ongoing exploration of tree-based representations in specialized tasks, such as code completion and bug fixing.

Pixel-based input and hybrid-based input were relatively less common, each found in one study, contributing approximately 0.28% of the total studies each. While their adoption rates were lower, these input forms presented intriguing possibilities for specific applications. Pixel-based input offered a unique visual representation of code, potentially advantageous for code translation tasks. Meanwhile, hybrid-based input, combining multiple modalities (e.g., code in tree structure and text in tokens in the work by Niu et al. [314]), showcased the potential for enhancing code comprehension tasks by offering diverse perspectives for the models to learn from.

In summary, the trends in input form usage reveal a strong preference for token-based input, demonstrating its versatility and effectiveness in various SE tasks. However, ongoing exploration of other input forms, such as tree/graph-based, pixel-based, and hybrid-based, suggests a dynamic and evolving landscape in the application of LLMs for SE, with potential for further innovation and improvement in specialized domains. Each of these input forms caters to specific characteristics of the SE tasks being addressed, enabling LLMs to perform effectively across a wide range of code-related applications with a more comprehensive understanding of the input data.

RQ2—Summary

(1) We divided the datasets into four categories based on the source of data: open-source, collected, constructed, and industrial datasets. *The use of open-source datasets is the most prevalent*, constituting approximately 62.83% of the 374 articles that explicitly state the dataset.
(2) We categorized the data types within all datasets into five groups: code-based, text-based, graph-based, software repository-based, and combined. *Text-based and code-based types are the most frequently used in applying LLMs to SE tasks.* This pattern indicates that LLMs are particularly adept at handling text and code-based data in SE tasks, leveraging their NL processing capabilities.
(3) We summarized the data pre-processing procedures for different data types and found several common pre-processing procedures, i.e., *data extraction*, *low-quality data deletion*, *duplicated instance deletion*, and *data segmentation*.

[3]This refers to studies that explicitly state input forms of LLMs, i.e., a total of 355 articles as shown in Table 8.

5 RQ3: What Techniques Are Used to Optimize and Evaluate LLM4SE?

5.1 What Tuning Techniques Are Used to Enhance the Performance of LLMs in SE Tasks?

Through surveying research related to LLM4SE, we found that while many general-purpose LLMs can be directly applied to SE tasks such as code generation [73, 248, 515], code summarization [7, 408, 507], and program repair [37, 102, 489] without fine-tuning, the hidden potential of LLMs often needs to be realized through tuning to be fully exploited. Specifically, this requires training LLMs with task-specific data to learn knowledge relevant to the task context to perform better. We observed that out of 83 studies, LLMs were fine-tuned using *full fine-tuning* techniques to adapt to downstream SE tasks, with the majority being BERT series models [57, 83, 90, 165, 204, 214, 216, 246, 272, 372, 438, 463, 469, 533, 553]. The cost of training these LLMs is expensive, requiring a large amount of computational resources and massive amounts of data. It is also costly to train and deploy the fine-tuned models separately for each downstream task, as the traditional fine-tuning approach would need to copy a model and perform full-parameter fine-tuning for each downstream task [34, 63, 83, 160, 165, 216].

To reduce this computational burden, some researchers have previously used **In-Context Learning (ICL)** [102, 104, 142, 150, 170], which feeds the model with manually designed "prompts" that are overly reliant on human design and do not require updating model parameters at all. However, ICL only operates at the time of inference and does not involve learning task-specific parameters, which experimentally proved to give the model limited improvement in downstream tasks [250]. To address this problem, researchers have begun to apply PEFT [139] techniques to LLMs. PEFT aims to improve the performance of pre-trained models on new tasks by optimizing the subset of parameters fine-tuned, thereby reducing the overall computational complexity. This approach maintains the majority of the pre-trained model's parameters in a fixed state, focusing fine-tuning efforts on a minimal yet impactful set of parameters [472]. Prior code intelligence research has demonstrated the capabilities of PEFT techniques, frequently revealing their superiority over full fine-tuning on a variety of tasks [472]. Four common techniques of PEFT include **Low-Rank Adaptation (LoRA)** [140], prompt tuning [217], prefix tuning [235], and adapter tuning [139]. We now elaborate on each as follows:

LoRA. LoRA injects low-rank trainable matrices into the attention layers of the Transformer architecture to significantly reduce the number of trainable parameters. We observed that eight studies [19, 268, 324, 386, 388, 397, 461, 539] utilized LoRA to enhance the performance of LLMs in SE tasks. For instance, Pan et al. [324] trained SteloCoder, specifically designed for translating multiple programming languages into Python code, which is based on the StarCoder LLM. LoRA technology was employed during the modification of the StarCoder model architecture to adjust the parameter count. Additionally, Silva et al. [397] applied LoRA to LLaMA, resulting in a highly effective "program repair adapter" for fixing bugs through fine-tuning.

Prompt Tuning. Prompt tuning involves appending learnable tokens to the model's input, guiding it toward better task performance. This method keeps the model's architecture unchanged, leveraging adaptable prompts to influence outputs without altering internal parameters. In the surveyed articles, three research works [268, 461, 572] utilized prompt tuning. For instance, Zhu et al. [572] proposed a method named AUMENA, which automates method naming tasks through context-aware prompt tuning.

Prefix Tuning. Prefix tuning adapts PLMs by adding trainable tokens not just to the input but also across internal layers, affecting the model's intermediate representations. This approach modifies the model's processing with minimal changes to its original parameters, allowing for task-specific customization. This technique was utilized in the following two studies: Lu et al. [268] fine-tuned

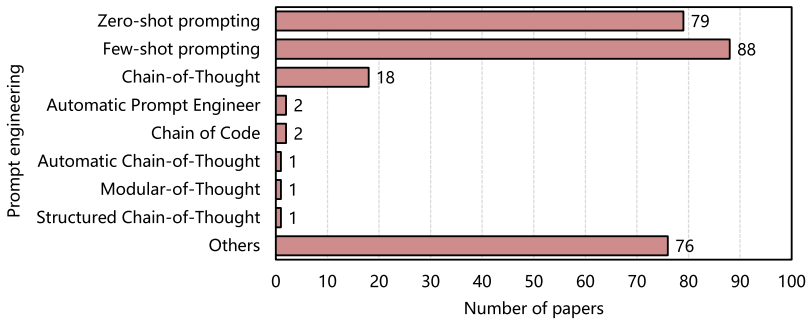


Fig. 9. The prompting engineering techniques used in LLMs for SE tasks. See Appendix C for the full table including references.

LLaMA-Reviewer for automating code review, while Wang et al. [461] fine-tuned CodeT5+ for multiple downstream tasks such as code completion, code generation, and code search.

Adapter Tuning. Adapter tuning adds small neural network modules to the original model, then fine-tuning them on specific tasks without altering the original model's parameters. Agarwal et al. [1] fine-tuned LLMs using adapter tuning techniques to make them suitable for code representation tasks. Wang et al. [447] indicated that LLMs refined through adapter tuning perform exceptionally well in code search and code summarization tasks.

In addition to the above-mentioned tuning methods, other techniques have been used for tuning LLMs in the LLM4SE domain, such as Reinforcement Learning [156, 157, 161, 403, 504], Supervised Fine Tuning [71, 157, 286, 403, 504], an unsupervised data augmentation method called *syntax fine-tuning* [344], *knowledge preservation fine-tuning* [395], and *task-oriented fine-tuning* [408].

5.2 What Prompt Engineering Techniques Are Applied to Improve the Performance of LLMs in SE Tasks?

Prompt engineering is a method of enhancing model performance by using task-specific instructions, known as prompts, without modifying the core model parameters. This approach enables LLMs to seamlessly integrate into downstream tasks solely based on the given prompts, guiding model behavior without the need to update model parameters [369]. Figure 9 presents eight prompt engineering techniques currently applied in the LLM4SE domain.

Zero-Shot Prompting. In zero-shot prompting [348], the model is expected to perform a task without any explicit training on that task. Instead, it relies on the prompt provided during inference to generate the desired output. Following few-shot prompting in terms of usage frequency, 79 studies adopted zero-shot prompting [204, 226, 272, 335, 376, 473, 483, 497]. For example, Li et al. [226] introduced CodeEditor, a pre-trained model specifically designed for code editing, and demonstrated its effectiveness in automatic code editing under zero-shot settings.

Few-Shot Prompting. Few-shot prompting involves providing a limited number of examples or instructions to the model to perform a specific task. The model learns from these examples and generalizes to similar tasks with minimal training data. In the surveyed LLM4SE research, 88 studies utilized few-shot prompting [7, 91, 95, 104, 185, 469, 494, 551]. For instance, Geng et al. [104] adopted an in-context learning paradigm and providing a specific number of prompts simultaneously significantly outperformed state-of-the-art supervised learning methods in generating comments with multiple intents.

Chain-of-Thought (CoT) Prompting. Wei et al. [468] introduced a prompting technique called CoT, which involves each prompt building upon the preceding one, resulting in a coherent chain of

reasoning that enhances the model's ability to generate well-structured and thoughtful responses. Huang et al. [151] proposed a novel method leveraging the fault-tolerance and comprehension capabilities of pre-trained LLMs to generate CFG. This method involves a CoT with four steps: structural hierarchy extraction, nested code block extraction, CFG generation for nested code blocks, and merging of CFGs for all nested code blocks. Tian et al. [429] also introduced the first test case-driven code generation technique, named TCoT, to further enhance LLMs' capabilities in code generation. Including the two studies mentioned earlier, a total of 18 studies applied CoT to improve LLMs' performance in SE tasks [64, 91, 150, 151, 225, 233, 263, 297].

Automatic Prompt Engineer (APE). Inspired by classical program synthesis and human prompt engineering methods, Zhou et al. [571] introduced an APE for automatic instruction generation and selection. APE is a system designed to automatically generate effective prompts for LLMs based on the desired task. It aims to simplify the process of prompt engineering by automating the generation of task-specific instructions. Sharing a similar concept of automated prompts, Sun et al. [408] proposed a new prompt learning framework called PromptCS. PromptCS trains a prompt agent that can generate continuous prompts to fully explore LLMs' potential in code summarization tasks. Continuous prompts, generated under the guidance of LLMs, are easier for LLMs to comprehend compared to manually written discrete prompts.

Chain of Code (CoC) Prompting. CoC prompting [218] is similar to CoT prompting but is specifically tailored for programming tasks. It involves providing a sequence of prompts or code snippets to guide the model's code generation process. Huang et al. [144] proposed CodeCoT, and Le et al. [213] proposed CodeChain, both of which are reasoning frameworks that better guide LLMs in code generation.

Automatic Chain-of-Thought (Auto-CoT) Prompting. Auto-CoT [557] is an automated version of CoT prompting where the sequence of prompts is generated automatically based on the input and desired task. Paranjape et al. [327] introduced a framework, Automatic. ART, for generating intermediate reasoning steps automatically. ART can select multi-step reasoning and tools from a task library based on given tasks at any time and has been experimentally proven effective in code tasks.

Modular-of-Thought (MoT) Prompting. In code generation tasks, LLMs often generate solutions in the form of a single block of code, limiting their effectiveness in handling complex problems. To overcome this limitation, Li et al. [222] proposed the **Modular-of-Thought Coder (MoTCoder)**. They introduced a new MoT prompting optimization framework to facilitate task decomposition into logical subtasks and submodules. Experimental results demonstrate that MoTCoder significantly improves the modularity and correctness of solutions generated by LLMs in programming tasks.

Structured Chain-of-Thought (SCoT) Prompting. Considering that source code contains rich structural information, Li et al. [225] proposed SCoT prompting specifically for code generation tasks. Researchers enable LLMs to use program structure to construct CoTs (i.e., intermediate NL reasoning steps) to obtain SCoTs. Then, LLMs generate the final code based on SCoTs. Compared to CoT prompts, SCoT prompts explicitly constrain LLMs to consider how to address requirements from the source code perspective. Evaluations across multiple benchmarks show that SCoT significantly enhances LLMs' performance in code generation.

Others. In addition to the eight prompting techniques mentioned above, we identified 76 studies where researchers, although not explicitly mentioning the application of any of the aforementioned prompting techniques, carefully designed prompts or proposed new strategies based on prompts to apply LLMs to SE tasks better. For instance, Ren et al. [356] proposed a code generation method based on knowledge-driven prompt chains. Li et al. [233] applied differential prompting to ChaGPT to better identify test cases that cause failures in buggy programs. Ahmed et al. [7] enhanced

Table 9. Evaluation Metrics for Different Types of Tasks

Problem type	Metric		Total
Regression	MAE (1)		1
Classification	Precision (35)	Recall (34)	147
	F1-score (33)	Accuracy (23)	
	AUC (Area Under the ROC Curve) (9)	ROC (Receiver Operating Characteristic) (4)	
	FPR (False-Positive Rate) (4)	FNR (Falser Negative Rate) (3)	
	MCC (Matthews Correlation Coefficient) (2)		
Recommendation	MRR (Mean Reciprocal Rank) (15)	Precision/Precision@k (6)	39
	MAP/MAP@k (6)	F-score/F-score@k (5)	
	Recall/Recall@k (4)	Accuracy (3)	
Generation	BLEU/BLEU-4/BLEU-DC (62)	Pass@k (54)	338
	Accuracy/Accuracy@k (39)	EM (Exact Match) (36)	
	CodeBLEU (29)	ROUGE/ROUGE-L (22)	
	Precision (18)	METEOR (16)	
	Recall (15)	F1-score (15)	
	MRR (6)	ES (Edit Similarity) (6)	
	ED (Edit Distance) (5)	MAR (Mean Average Ranking) (4)	
	ChrF (3)	CrystalBLEU (3)	
	CodeBERTScore (2)	MFR (Mean First Ranking) (1)	
	PP (Perplexity) (1)		

See Appendix D for the full table including references.

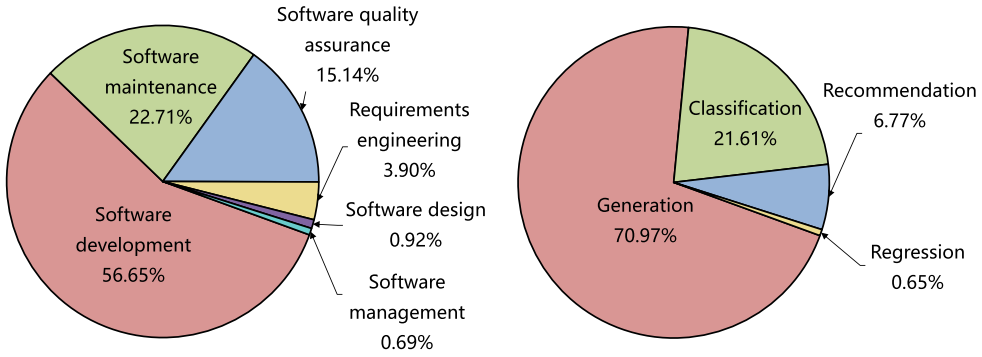
the performance of LLMs in code summarization tasks using automatic semantic augmentation prompts.

5.3 How Are Evaluation Metrics Utilized to Assess the Performance of LLM4SE Tasks?

Evaluating the performance of LLM4SE is a crucial aspect of their development and deployment [185]. Benchmarking against existing datasets and using baselines are common practices to evaluate the effectiveness of LLMs [34]. However, given the diversity of SE tasks, a single evaluation metric may not suffice to capture the model's performance comprehensively. Thus, researchers often employ a range of evaluation metrics tailored to specific problem types [288, 314, 372]. We categorize the SE tasks summarized from 395 articles into four categories according to their addressed problem types, i.e., regression, classification, recommendation, and generation tasks, as displayed in Figure 10(b). The selection of evaluation metrics depends on the target problem types. For example, **Mean Absolute Error (MAE)** has been used for regression tasks [98]. We summarize the most frequently used evaluation metrics for each task type.

For classification tasks, the most commonly used metrics are Precision [26, 48, 83, 90, 130], Recall [26, 48, 83, 90, 130, 135] and F1-score [11, 26, 48, 83, 90, 130], with 35, 34, and 33 studies, respectively, employing these metrics. For example, in the study conducted by Khan et al. [190], F1-score is utilized to evaluate the performance of an automatic bug-fixing model. Similarly, Sharma et al. [382] use Precision and Recall to assess the effectiveness of a transformer-based model for code summarization. These metrics are essential for evaluating the model's ability to correctly classify code snippets [90] or identify specific SE properties [48].

For recommendation tasks, **Mean Reciprocal Rank (MRR)** is the most frequent metric, used in 15 studies [55, 160, 223, 246, 350, 372, 390, 469]. MRR is employed to measure the effectiveness of recommendation systems for code completion, as demonstrated in the study by Ciborowska et al. [55]. Precision@k [55, 129, 246, 572] and F1-score@k [129, 246, 572, 573] are also utilized in recommendation tasks, with six studies each. These metrics are used to evaluate the precision and F1-score of the recommended code snippets or code completions.



(a) Distribution of LLM usages in SE activities. (b) Problem classification based on collected studies.

Fig. 10. Distribution of LLM utilization across different SE activities and problem types.

In generation tasks, metrics like BLEU, along with its variants BLEU-4 and BLEU-DC [7, 9, 19, 40, 57], and Pass@k [31, 34, 38, 43, 67, 70] are the most commonly used, appearing in 62 and 54 studies, respectively. For instance, Wang et al. [461] employed BLEU to evaluate a code-to-code translation model. Pass@k is used in the research by Jiang et al. [170] to assess code generation models, measuring the proportion of generated code snippets that match the reference solutions. Additionally, ROUGE/ROUGE-L [7, 9, 102, 104, 230, 238, 286, 288, 314, 524], METEOR [7, 9, 40, 102, 104, 314], **Exact Match (EM)** [9, 102, 122, 299, 461, 473, 513, 556], and **Edit Similarity (ES)** [256] are used in specific studies to evaluate the quality and accuracy of generated code or NL code descriptions.

RQ3—Summary

(1) We discovered a range of tuning techniques gradually becoming widely adopted in the LLM4SE domain. Among these, PEFT techniques, including LoRA, prompt tuning, prefix tuning, and adapter tuning, are gaining prominence for optimizing LLMs while minimizing computational complexity.
(2) We identified a diverse set of eight prompting techniques, including zero-shot prompting, few-shot prompting, CoT, APE, CoC, Auto-CoT, MoT, and SCoT, applied in the LLM4SE domain to enhance model performance. These techniques leverage task-specific instructions, known as prompts, to guide LLMs without modifying core model parameters, providing a promising avenue for improving LLM capabilities in SE tasks.
(3) We summarized the most widely used evaluation metrics according to four problem types, i.e., regression, classification, recommendation, and generation. Nineteen different evaluation metrics appeared in the generation task, while nine metrics were used for the classification task.

6 RQ4: What SE Tasks Have Been Effectively Addressed to Date Using LLM4SE?

6.1 What Are the Distributions SE Activities and Problem Types Addressed to Date with LLM4SE?

In this section, we provide a detailed analysis of the use of LLMs for different SE tasks. We summarize reported SE tasks [509] addressed with LLMs, following the six phases of the Software Development Lifecycle (i.e., RE, software design, software development, software quality assurance, software maintenance, and software management). Figure 10(a) describes the distribution of LLMs in these

Table 10. Distribution of SE Tasks over Six SE Activities

SE activity	SE task		Total
Requirements engineering	Anaphoric ambiguity treatment (4) Requirements analysis and evaluation (2) Coreference detection (1) Specification formalization (1) Use cases generation (1)	Requirements classification (4) Specification generation (2) Requirements elicitation (1) Traceability automation (1)	17
Software design	GUI retrieval (1) Software specification synthesis (1)	Rapid prototyping (1) System design (1)	4
Software development	Code generation (118) Code summarization (21) Code translation (12) API inference (5) API recommendation (5) Code representation (3) Method name generation (2) Agile story point estimation (1) API documentation smells (1) Data analysis (1) Control flow graph generation (1) Instruction generation (1) Others (14)	Code completion (22) Code search (12) Code understanding (8) Program synthesis (6) Code editing (5) Code comment generation (2) Code recommendation (2) API documentation augment (1) API entity and relation extraction (1) Fuzz driver generation (1) Identifier normalization (1) Type inference (1)	247
Software quality assurance	Vulnerability detection (18) Bug localization (5) Testing automation (4) Defect detection (2) Static analysis (2) Compiler fuzzing (1) Invariant prediction (1) Mobile app crash detection (1) Test prediction (1)	Test generation (17) Verification (5) Fault localization (3) GUI testing (2) Binary taint analysis (1) Decompilation (1) Malicious code localization (1) Resource leak detection (1)	66
Software maintenance	Program repair (35) Code review (7) Bug reproduction (3) Duplicate bug report detection (3) Log parsing (3) Sentiment analysis (3) API misuses repair (1) Bug triage (1) Code review explained (1) Crash bug repair (1) Incivility detection (1) Patch detection (1) Rename Refactoring (1) Technical debt payback (1) Web test repair (1) Others (5)	Code clone detection (8) Debugging (4) Review/commit/code classification (3) Logging (3) Code revision (2) Vulnerability repair (2) Bug prediction (1) Code coverage prediction (1) Code-Review defects repair (1) Dockerfile Repair (1) Patch correctness prediction (1) Program merge conflicts repair (1) Tag recommendation (1) Traceability recovery (1) Type error repair (1)	99
Software management	Effort estimation (2)	Software tool configuration (1)	3

See Appendix E for the full table including references.

six activities. Table 10 shows a detailed count of selected primary studies reporting specific SE tasks addressed with LLMs.

The highest number of studies is observed in the software development domain, constituting approximately 56.65% of the total primary studies. This underscores the focus to date on utilizing

LLMs to enhance coding and development processes. Software maintenance tasks account for about 22.71% of the primary studies, highlighting the significance of LLMs in aiding software updates and improvements. The software quality assurance domain is studied in approximately 15.14% of the primary studies, indicating a growing interest in automating testing procedures. In contrast, RE and software design activities represent approximately 3.9% and 0.92% of the primary studies, respectively, suggesting relatively limited exploration so far in these areas. The software management domain has the least representation, accounting for a tiny 0.69% proportion of the studies. This distribution underscores the vital focus on development and maintenance tasks while also indicating potential avenues for further research in testing, design, and management domains.

We classified our collection of LLM studies for SE tasks based on the type of problems they address (shown in Figure 10(b)). The distribution reveals that *the majority of studies, about 70.97%, center around generation tasks*, showcasing the significance of LLMs in producing code or text. Following this, around 21.61% of the studies fall under classification tasks, indicating the relevance of LLMs in categorizing software elements. Additionally, roughly 6.77% of the studies are related to recommendation tasks, demonstrating the utility of LLMs in suggesting solutions. Lastly, a tiny portion, around 0.65%, is allocated to regression tasks, reflecting the limited exploration of LLMs to date for predictive modeling. *This distribution underscores the broad applicability of LLMs across different SE challenges, with a notable emphasis on code generation and classification tasks.*

6.2 How Are LLMs Used in RE?

This section explores the utilization of LLMs in the domain of RE. It encompasses tasks such as anaphoric ambiguity treatment, requirements classification, coreference detection, requirements elicitation, and software traceability.

Anaphoric Ambiguity Treatment. Ambiguity in software requirements arises when a single reader can interpret a NL requirement in multiple ways, or when different readers have varying understandings of the same requirement. Unclear and ambiguous NL software requirements can lead to suboptimal software artifacts during later development stages. Moharil et al. [292] and Ezzini et al. [83] have empirically demonstrated the significant role of LLMs such as BERT and SpanBERT in effectively addressing anaphoric ambiguity. Sridhara et al. [400] revealed that ChatGPT excels in addressing anaphoric ambiguity in software requirements. Through researchers' analysis of ten English requirements specifications [83] containing anaphora-related challenges, ChatGPT consistently demonstrated its remarkable capability to accurately identify antecedents. This empirical evidence emphasizes the valuable role ChatGPT can play in enhancing the clarity and precision of software requirements, ultimately contributing to more effective software development processes by reducing interpretational uncertainties.

Requirements Classification. Originating in NL documents, requirements demand effective classification, especially for early-stage project discernment, like security-related ones [199, 220]. Automated processing hinges on identifying these requisites. Categorizing into **Functional (FR)** or **Non-Functional (NFR)** requirements, with quality constraints, benefits automated approaches [220]. Hey et al. [135] employ BERT for requirements classification, where it excels in categorizing both FR and NFR requirements using a fine-tuning transfer learning technique, outstripping traditional methods. Luo et al. [272] introduce a BERT-based software requirements classification method, demonstrating remarkable transferability and generalization, especially in zero-shot scenarios.

Requirements Term Identification. Moharil et al. [291] propose a technique for identifying terms used in different contexts within the same domain or in interdisciplinary projects. Using BERT, which reads entire word sequences for deeper language understanding, and K-means clustering,

they create and group vectors for each term in the corpora. The method has been validated on large Computer Science and multi-domain corpora comprising eight different fields.

Coreference Detection. Requirements, authored by diverse stakeholders, continually evolve, leading to terminology differences and inconsistencies across domains. Entity coreference in RE, where various expressions refer to the same real-world entity, can cause confusion and affect comprehensibility. Wang et al. [462] offer a novel application of the BERT model for coreference detection.

Traceability Automation. Software and system traceability refers to the ability to establish and maintain relationships between software artifacts, such as requirements, design definitions, code, and test cases, for product querying and development support [358]. Lin et al. [246] found that T-BERT can effectively migrate knowledge from code search to NLA-PLA (i.e., Natural Language Artifacts to Programming Language Artifacts) traceability, even with limited training instances. It outperforms existing techniques in accuracy and can be adapted to different domains without intermediate training for each project, offering a promising step toward practical, trustworthy traceability.

Others. In addition to the four RE tasks detailed above, LLMs have also been applied to requirements analysis and evaluation [341, 363], specification generation [274, 490], requirements elicitation [475], specification formalization [82], and use case generation [548].

6.3 How Are LLMs Used in Software Design?

GUI Retrieval. Kolthoff et al. [203] describe the application of BERT to the task of GUI retrieval in SE. The authors fine-tune a BERT-based **Learning-to-Rank (LTR)** model for this task. GUIs, which are not standard well-structured text documents, present unique challenges for text-based ranking tasks. The BERT model is prepared by concatenating the NL query and the GUI document text, and then this input is used to train different BERT-LTR models. The models are evaluated based on their performance in NL-based GUI ranking.

Rapid Prototyping. Rapid prototyping enables developers to quickly visualize and iterate on software designs, thereby accelerating the development process and ensuring alignment with user needs. White et al. [475] investigate the role of LLMs in augmenting this process. The study introduces prompt design techniques, organized into patterns, providing a structured methodology to tackle prevalent challenges in LLM4SE. This research indicates that the realm of rapid prototyping stands to benefit from deeper integration with advanced ML techniques, thereby creating opportunities for additional research and refinement aimed at producing more intuitive and user-centric software designs.

Software Specification Synthesis. Software configuration is vital for system behavior, but managing configurations and specifications becomes complex with larger systems. Mandal et al. [279] introduce SpecSyn, a framework using an LLM for automatic software specification synthesis from NL sources. This end-to-end approach treats the task as a sequence-to-sequence learning problem, surpassing the previous state-of-the-art tool by 21% in F1 score, and can find specifications from both single and multiple sentences.

6.4 How Are LLMs Used in Software Development?

Our analysis identifies wide-ranging applications of LLMs for software development, encompassing tasks such as code generation, code completion, and code summarization.

Code Generation. Code generation has long been a task of interest: there is extensive work on program synthesis using symbolic and neural-semiotic approaches [13, 482]. Recently, LLMs trained for text generation have demonstrated the ability to complete programs [27, 29]. Since 2020, several code generation models have been trained or fine-tuned on programming language text [43, 58, 92, 97, 313, 493]. Unlike traditional program synthesis techniques, neurolinguistic models can

Table 11. The State-of-the-Art Applications of LLMs in Code Generation Task

Model	Baseline	Benchmark	Metric	Date	Reference
GPT-3.5	Codex, CodeGen, CodeGeeX, LLaMA, InCoder, PyCodeGPT, CodeParrot, GPT-2	HumanEval, MBPP, MBCPP	Pass@k	11 May 2023	[224]
GPT-4	PaLM Coder, Codex, CodeGen-Mono, Incoder, CodeGeeX, AlphaCode	HumanEval, HumanEval-ET, MBPP, MBPP-ET	Pass@k	24 May 2023	[73]
GPT-4	GPT-3.5, StarCoder, CodeGen, CodeGen2, Vicuna, SantaCoder, Incoder, GPT-J, GPT-Neo, PolyCoder, StableLM	HumanEval, HumanEval+, HumanEval-mini	Pass@k	12 June 2023	[253]
GPT-4	GPT-3.5, WizardCoder, Instruct-StarCoder, SantaCoder, Instruct-CodeGen, CodeGeeX, InCoder, Vicuna, ChatGLM, PolyCoder	ClassEval, HumanEval	Pass@k	3 August 2023	[76]

be conditioned on NL (e.g., code annotations) as well as generate programming language text. Researchers have experimentally demonstrated that LLMs like GPT-4 [22, 107, 169, 253], GPT-2/GPT-3/GPT-3.5 [20, 73, 188, 224, 248, 253, 259, 301, 449, 515], BERT series [209, 529], Codex [22, 43, 67, 122, 207, 278, 519], CodeGen [67, 176, 525], InCoder [205, 253, 299, 454], Copilot [482] and CodeGeeX [563] play a key role in code generation. By pre-training on large-scale text data, these models learn rich linguistic knowledge and semantic representations that enable them to understand the meaning and structure of NL. LLMs can automate code generation by converting NL descriptions into code [170]. These models generate program code from NL descriptions, enhancing code-writing efficiency and accuracy. They show excellent performance in code completion, automatic code generation, and conversion of NL annotations to code, providing software developers with powerful auxiliary tools and promoting further automation and intelligence in the code writing and development process.

Within the domain of LLMs applied to software development tasks, studies centered on code generation distinctly dominate the academic landscape. As reflected in Table 11, *the GPT series, particularly GPT-4, emerged as a key focus, with many more studies using them in the realm of code generation* [73, 76, 224, 253]. Analyzing these studies, several noteworthy findings surface:

— *Programming thinking in LLMs.* Techniques that evoke "programming thinking" within LLMs, such as the Thinking in Programming [224] methodology, have shown promising strides. By guiding LLMs to first craft a high-level code sketch before delving into detailed implementations, the synthesized code exhibits higher accuracy and robustness.
— *Class-level vs. Method-level generation.* LLMs, while adept at method-level code generation, present varied performance metrics when tasked with class-level generation [76]. This divergence underscores the evolving nature of challenges as the granularity of code synthesis shifts.
— *Expanding LLM capabilities.* The next frontier in this discipline seems to lie in harmoniously integrating LLMs with established SE tools and practices. The emergence of frameworks like EvalPlus [253] indicates a trend toward enhancing the evaluation and accuracy of LLM-generated code, possibly ushering in an era where human developers and LLMs collaboratively craft software solutions.

Code Completion. Code completion is an assistive feature provided by many integrated development environments and code editors. Its purpose is to automatically display possible code suggestions or options as developers write code [14]. This innovation has been advanced by LMs, evolving from n-gram and RNN models to transformer-based models like Copilot [109] and CodeGPT [178], pre-trained on extensive code datasets. Recent LLMs equipped with billions of

parameters, excel in generating code snippets. These models are trained on vast amounts of NL text, equipping them with powerful semantic understanding capabilities. In the context of code completion, LLMs such as Codex [43, 70, 244, 333], BERT series [191], GitHub Copilot [70, 244, 343], CodeParrot [244, 493], GPT series [316, 493], T5 [57], InCoder [97], PolyCoder [493], CodeGen [68, 69, 244, 312], and other LLMs [160, 316] can generate accurate and intelligent code suggestions based on code context and syntax structures. They comprehend the developer's intent, predict the next possible code snippet and provide appropriate recommendations based on the context.

With the support of LLMs, code completion achieves significant improvements in efficiency and accuracy. Developers can save time by avoiding manual input of lengthy code and reducing the risk of code errors. LLMs also learn from extensive code repositories, acquiring knowledge and best practices to offer more intelligent and precise suggestions, aiding developers in better understanding and utilizing code [57]. Additionally, these models can provide personalized code recommendations based on developers' coding styles and preferences, further enhancing the effectiveness and user experience of code completion [256].

Code Summarization. Code summarization is a task that attempts to understand the code and automatically generate descriptions directly from the source code. It can also be viewed as an extended form of documentation. Successful code summarization not only facilitates the maintenance of source code [159, 305] but can also be used to improve the performance of code search using NL queries [310, 503] and code classification [305]. LLMs play a significant role in code summarization by analyzing code structures and contexts to generate informative NL summaries. Specifically, LLMs such as Codex [6, 19, 102], CodeBERT [40, 102, 115], and T5 [286, 288] comprehend the functionality and logic of the code, producing easily understandable human language descriptions. For example, Arakelyan et al. [19] rigorously evaluate the efficacy of CodeT5 and Codex across code generation and summarization tasks, shedding light on their performance under distribution shifts. It unveils practical adaptation techniques, underscoring Codex's commendable performance. Additionally, the study demonstrates that while adapted models exhibit proficiency in code generation, their generality can present tradeoffs in the context of code summarization. As a result, code summarization with the support of LLMs enhances code readability, improves software documentation quality, and accelerates code comprehension and collaboration among developers. This advanced approach to code summarization demonstrates great potential for automating and streamlining various aspects of software development in modern SE practices with the employment of LLMs.

Code Search. Code search, or code retrieval, is the task of retrieving source code from a large code base, usually based on a user's NL query. Despite the success of neural models in code search, such models are relatively shallow and are not capable of learning large amounts of data [372]. In recent years, some bimodal pre-training models based on the BERT neural architecture have been proposed to capture semantic links between natural and programming languages [92, 118, 364, 459], such as CodeBERT [92] and GraphCodeBERT [118]. Bimodal pre-training models learn generic representations from large amounts of data in an unsupervised manner by designing pre-training goals. Salza et al. [372] explored the effectiveness of LLMs such as BERT [372] and RoBERTa [40] in understanding NL and code semantics and enhancing code search and retrieval. These studies show that pre-training tasks alone may not be sufficient for code search, which emphasizes the need for a multimodal understanding of data [390], including both NL and code. In addition, research has shown that the use of code generation models such as Codex [219] can enhance code retrieval by generating code snippets from NL documents, thereby improving semantic similarity and obtaining state-of-the-art results on benchmark datasets.

Code Translation. Code translation aims to convert source code from one programming language to another. Researchers have been actively exploring the potential of LLMs in this field. For instance,

study [505] introduces the CoTR approach to enhance the robustness of code translation models. Additionally, SteloCoder [324] focuses on translating multiple programming languages to Python, showcasing improvements in translation functionality. However, as noted in [325], LLMs still face challenges in automating code translation, with high error rates. In summary, while LLMs show promise in code translation tasks, further optimization is needed to improve translation quality and robustness.

Code Understanding. In contrast to code summarization, which focuses on automatically generating human-readable descriptions from source code, code understanding involves a deep analysis of source code to comprehend its logic, structure, functionality, and dependencies, as well as understanding the programming languages, frameworks, and libraries used [384]. LLMs can assist in code understanding by leveraging their powerful NL processing capabilities to interpret code-related text, such as comments and documentation [181, 461]. They aid developers in grasping code functionality, identifying dependencies, and generating relevant code documentation [276, 384]. Through their ability to comprehend both code and NL, LLMs enhance the efficiency and accuracy of code understanding, empowering developers to maintain, optimize, and integrate code effectively [181].

Program Synthesis. Program synthesis is the automated process of generating code that satisfies a given specification or set of constraints, emphasizing the derivation of functional properties of the code [46, 47, 281, 328, 401]. It differs from code generation, which primarily translates higher-level representations into target code without necessarily deriving its functionality from scratch [395, 541, 563]. Several studies have demonstrated that LLMs can be used for program synthesis tasks. LLMs have a significant impact on program synthesis due to their advanced language understanding and generation capabilities. LLMs can effectively interpret NL descriptions, code comments, and requirements and then generate corresponding code snippets that fulfill the given specifications. This helps developers rapidly prototype code and automate repetitive coding tasks [99, 207]. When applied to program synthesis, LLMs enhance productivity and reduce the burden on developers by automating the code-writing process based on high-level input [162]. Their ability to understand the nuances of both NL and programming languages makes them valuable tools in advancing the field of SE and streamlining the development lifecycle.

API Inference. The automated generation of API calls, known as API synthesis, plays a crucial role in bridging human intent with machine execution. In recent studies, Wang et al. [453] and Weyssow et al. [473] have both explored the potential of LLMs in this realm. Utilizing models like GPT-4 and LLaMA-based architectures, these researchers showcase the prowess of LLMs in generating accurate API calls and adapting to real-time documentation changes, effectively addressing challenges like hallucination and inaccurate input arguments. The integration of LLMs in API synthesis signifies a paradigm shift, promising enhanced accuracy, adaptability, and reliability in code generation. As illuminated by these studies, the future of API synthesis may be deeply anchored in advanced ML, heralding new research avenues and refinements for more seamless human-machine interactions.

API Recommendation. Several methods have been proposed to automate API recommendations [117, 149, 257, 304], falling into two orthogonal approaches: information retrieval-based and neural-based. In this context, our focus is on the latter. Wei et al. [469] introduced CLEAR, an API recommendation method that employs the BERT sentence embedding model to represent queries, capturing continuous semantic information. Through contrast training, CLEAR enables BERT to learn precise semantic representations of queries, independent of their lexical content. Recently, Zhang et al. [539] developed ToolCoder, which combines API search tools with existing models to aid in code generation and API selection. This approach involves an automated data annotation method using ChatGPT, adding tool usage information to the source code data, followed by fine-tuning the code generation model. During inference, an API search tool is integrated into

the generation process, allowing the model to utilize the tool for suggestions when selecting APIs automatically.

Code Editing. Code editing involves modifying source code to fix bugs, enhance features, or optimize performance. Recent studies have explored applying LLMs to this task. Tools like CodePlan [21] frame repository-level code editing as a planning problem, synthesizing multi-step edits with LLMs. The Grace method [122] enhances LLMs by conditioning code generation on previous edits to capture developer intent. CodeEditor [226] introduces a specialized pre-training task to improve automatic code editing. COFFEEPOTS [293] leverages open-source LLMs to generate corrective feedback. Additionally, LLMs have been adapted for high-level program optimization, achieving significant speedups [394]. These studies demonstrate LLMs' potential in automating complex code editing tasks.

Code Representation. Code representation learning (also known as code embedding) aims to encode the code semantics into distributed vector representations and plays a key role in recent DL-based models for code intelligence. Code representation can be used to support a variety of downstream tasks, such as code completion [355], code search [116, 440], and code summarization [443, 537]. Niu et al. [314] propose a novel sequence-to-sequence pre-training model that utilizes structural information from source code to enhance its representation learning. The model is trained on a large corpus of source code, which enables it to capture the complex patterns and dependencies inherent in programming languages. Wan et al. [442] show through their research that attention is highly consistent with the syntactic structure of the code, that pre-trained code LMs can preserve the syntactic structure of the code in the intermediate representations of each converter layer, and that pre-trained code models have the ability to induce a syntactic tree of the code. These revelations suggest that incorporating the syntactic structure of the code into the pre-training process results in better code representations.

Code Comment Generation. Code comment generation, the automatic creation of comments for source code, serves to elucidate code functionality, implementation logic, and input-output details, thereby enhancing readability and maintainability [104, 143]. As code complexity grows, manually crafting these comprehensive and accurate comments can become burdensome and prone to errors. Automation in this domain can markedly enhance the efficiency and quality of code documentation. LLMs such as Codex [104] and T5 [283] have been effectively applied to code comment generation. These models are pre-trained on vast amounts of data and possess powerful NL processing and semantic understanding capabilities. During comment generation, LLMs analyze the structure, semantics, and context of the source code to automatically generate high-quality comments that correspond to the code's functionality and logic. Addressing the often observed disconnect between code evolution and its accompanying documentation, Mastropaolo et al. [283] explore the potential of LLMs, particularly the T5 architecture, in assisting developers with code comment completion. Their empirical study juxtaposes the performance of the T5 model against an n-gram model, revealing T5's superior capabilities, though the n-gram model remains a competitive alternative. The research underscores the significance of open-source datasets for training and highlights the scant use of industrial datasets in current studies.

Method Name Generation. Method names significantly affect program comprehensibility, serving as a brief summary of the source code and indicating the developer's intent [200]. The importance of method names in program comprehension is further evidenced by recent studies showing that some programmers even write down important method names to help them figure out the procedures of an application [362]. Zhu et al. [572] present AUMENA, a novel approach using the CodeT5 model for context-aware method naming in SE. AUMENA first learns the contextualized representation of programming and NL, then leverages LLMs with prompt tuning to detect inconsistent method

names and suggest accurate alternatives. This method avoids previous generate-then-compare consistency checking limitations, modeling the task as a two-class classification problem.

Agile Story Point Estimation. Agile story point estimation, representing the total work needed to implement a product backlog item, is a complex task in agility. Story points are typically estimated by team consensus, using methods like plan poker and expert judgment, and considering factors like workload and complexity. However, subjective estimates may introduce uncertainty. Fu et al. [98] present GPT2SP, a Transformer-based approach that overcomes limitations of a previous method called Deep-SE. Unlike Deep-SE, which restricts LMs to known words within a trained project, GPT2SP employs a broader context, making it transferable across projects. GPT2SP's performance is comparable to Deep-SE in within-repository evaluations and surpasses it in 62.5% of cases, with improvements ranging from 3% to 46% across various projects.

API Documentation Smell Detection. APIs, vital for modern software development, are often accompanied by official documentation. Good documentation is key to proper API use, while poor quality can hinder adoption and negatively impact developers' productivity [2, 360, 361]. Khan et al. [190] identified five API documentation smells and presented a benchmark of 1,000 API documentation units containing the five smells found in the official API documentation. The authors developed classifiers to detect these odors, with BERT showing the best performance, demonstrating the potential of LLMs in automatically monitoring and warning about API documentation quality.

API Entity and Relation Extraction. Extracting APIs and their semantic relationships from unstructured text (e.g., data from SO) is a fundamental task in SE, but existing methods require labor-intensive manual rule creation or data labeling. Huang et al. [147] present an innovative approach, AERJE, that leverages LLMs for this task. AERJE consists of a BERT-based dynamic hint generator and a T5-based joint entity-relationship extractor, which together enable efficient extraction of API entities and relationships without manual effort. The approach achieved an F1 score of 96.51% for API entity extraction and 81.2% for API relationship extraction, offering a significant advancement over traditional methods.

Code Recommendation. Zhou et al. [568] pointed out that software developers tend to write similar code examples several times due to the need to implement similar features in different projects. Therefore, during the software development process, recommender systems can provide programmers with the most pertinent and high-quality examples written by other programmers, thus helping them to complete their tasks quickly and efficiently [66]. Open-source projects and informal documentation are the two main sources of information that developers rely on to perform programming tasks. For example, open-source projects on GitHub provide code examples and code resources for various tasks. Rahmani et al. [350] introduce a methodology to improve code example recommendations for Java programming language on SO using BERT and Query-Aware **Locality-Sensitive Hashing (LSH)**. They employ BERT to convert code into numerical vectors and then apply two LSH variants, Random Hyperplane-based, and Query-Aware, to identify Approximate Nearest Neighbors.

CFG Generation. CFGs are a cornerstone of SE that illustrate program behavior by showing sequences of statements and their execution order conditions [12]. As a graphical representation of program behavior, CFGs are critical in many SE tasks, including code search [42, 118], code clone detection [143, 455, 467], and code classification [456, 538]. Huang et al. [151] presented a novel approach for generating behaviorally correct CFGs of statically typed partial code by leveraging the error-tolerant and understanding ability of LLMs. The approach involves a CoT with four steps: structure hierarchy extraction, nested code block extraction, CFG generation of nested code blocks, and fusion of all nested code blocks' CFGs. The CoT is broken down into an AI chain according to the single responsibility principle, along with effective prompt instructions. This results in superior

node and edge coverage compared to traditional program analysis-based methods and the original CoT method.

Identifier Normalization. Identifiers usually consist of multiple words and a certain number of identifiers contain abbreviations [171]. Consequently, the lexical meaning of identifiers and the overall functionality of source code written by one developer may be challenging for other developers to comprehend. In addition, the source code cannot match the vocabulary in other software artifacts described in NL, thus invalidating some automated algorithms. Therefore, there is a strong need to normalize identifiers with the aim of aligning the vocabulary in identifiers with the NL vocabulary in other software artifacts. Zhang et al. [533] addressed this by introducing BEQAIN, an approach for identifier normalization. BEQAIN combines BERT with a **Question and Answering (Q&A)** system and Conditional Random Fields, treating identifier splitting as sequence labeling and abbreviation expansion as a Q&A task. It uses programming context to refine expansion results when multiple expansions are possible, aligning identifier vocabulary with NL and enhancing software development comprehension and automation.

Type Inference. Type inference, the automated process of determining data types in programming, plays a crucial role in enhancing readability, maintainability, and reducing runtime errors [131, 338]. TypeScript, with its unique blend of optional typing, presents a nuanced challenge, especially when navigating the vast landscape of user-defined types. Addressing this complexity, Jesse et al. [165] introduced an approach that leverages the capabilities of a BERT-style pre-trained model. Their solution, DIVERSETYPER, adeptly infers types for user-defined classes and interfaces by uniquely correlating class and interface declarations with their respective usage contexts. Beyond merely filling the gaps of previous methodologies, DIVERSETYPER sets a new benchmark in type inference, especially for user-defined types.

Others. In addition to the 20 software development tasks detailed above, LLMs can also be applied to API documentation augment [501], data analysis [51], fuzz driver generation [530], and instruction generation [571].

6.5 How Are LLMs Used in Software Quality Assurance?

Within the domain of software quality assurance, LLMs have emerged as valuable tools with diverse applications for various tasks, including vulnerability detection, test generation, bug localization, verification, test automation, and so on.

Vulnerability Detection. The number of software vulnerabilities is rapidly increasing, as shown by the vulnerability reports from Common Vulnerabilities and Exposures [17] in recent years. As the number of vulnerabilities increases, there will be more possibilities for cybersecurity attacks, which can cause serious economic and social harm. Therefore, vulnerability detection is crucial to ensure the security of software systems and protect social and economic stability. Traditional static detection methods are based on static analysis and predefined matching rules, which rely on developers' expertise and make it difficult to detect unknown vulnerabilities. With the assistance of LLMs [35, 48, 424], Tang et al. [417] introduced novel approaches using LLMs to enhance vulnerability detection. One of their proposed models, CSGVD, combines sequence and graph embedding for function-level vulnerability detection, outperforming other DL-based models on a real-world benchmark dataset. Their study also explores the application of CodeT5 for vulnerability detection, highlighting the importance of code-specific pre-training tasks.

Test Generation. Test generation involves automating the process of creating test cases to evaluate the correctness and functionality of software applications. It encompasses various aspects, including test case generation [542], unit test generation [375, 396, 419, 491, 523], and so forth. LLM application in test generation offers several advantages, including the ability to automatically generate diverse test cases, improving test coverage [375, 396] and identifying potential defects [491]. LLMs can

also assist in generating test cases based on NL descriptions, fostering better collaboration between developers and testers. Additionally, they help identify areas lacking test coverage and suggest relevant test cases, ensuring comprehensive testing and reducing the risk of undiscovered issues [542]. By enhancing test efficiency and effectiveness, LLMs contribute to producing more reliable and high-quality software products.

Bug Localization. Bug localization refers to the process of identifying the specific source code files, functions, or lines of code that are responsible for a reported bug or software defect. Bug localization typically involves analyzing bug reports or issue descriptions provided by users or testers and correlating them with the relevant portions of the source code. This process can be challenging, especially in large and complex software projects, where codebases can contain thousands or even millions of lines of code. Traditional bug localization methods often rely on heuristics, code metrics, or stack trace analysis, which may not always provide precise results. Ciborowska et al. [56] investigated data augmentation techniques to enhance bug localization models. They introduce a pipeline applying token-level operations such as dictionary replacement, insertion, random swapping, and deletion, along with paragraph-level back-translation to bug reports. By employing augmented data to train BERT-based models for bug localization, they demonstrate that these techniques can substantially expand the training data and boost the models' performance.

Verification. Verification techniques, including prominent methods such as formal verification, hold a pivotal role in the domain of software quality assurance [37, 430]. These techniques validate the correctness of software systems, improving their reliability and security against potential threats. Utilizing mathematical and logical principles in the verification process facilitates thorough error detection and correction before deployment, ensuring stable and secure performance in different operational contexts. Charalambous et al. [37] leverage LLMs, particularly the GPT-3.5, in the realm of formal verification. Their approach combines LLMs with bounded model checking to automatically repair software based on formal methods, showcasing the model's capability to understand intricate software structures and generate accurate repairs.

Test Automation. Automated testing methodologies offer a comprehensive array of tools and strategies designed for the evaluation of software applications' accuracy, reliability, and performance. These methodologies encompass various techniques, such as mutation testing [194] and fuzzing [63, 64]. LLMs have been used for mutation testing, introducing faults to the codebase to assess the effectiveness of test suites in identifying and detecting errors [194]. Furthermore, LLMs can aid in fuzzing, generating valid and diverse input programs that help identify vulnerabilities and bugs, particularly in challenging domains like DL libraries [63]. By incorporating LLMs into test techniques, software engineers benefit from improved test coverage, reduced manual effort, and enhanced bug detection [64], leading to more robust and reliable software systems.

Fault Localization. Test suites typically include two types of test cases: pass-through test cases and fault-inducing test cases [232]. In practice, there are far more pass test cases for faults than fault-inducing test cases, which hinders the effectiveness of program debugging. However, in practice, it is difficult to find fault-inducing test cases. This is because developers first need to find test inputs that trigger program faults, and the search space for such test inputs is huge [96]. Moreover, developers need to build a test oracle to automatically detect program faults and building a test oracle is often an undecidable problem [154]. Li et al. [232] investigated the application of ChatGPT to the task of finding fault-inducing test cases in SE. While recognizing ChatGPT's potential, they initially observed suboptimal performance in pinpointing these cases, particularly when two versions of a program had similar syntax. The authors identified this as a weakness in ChatGPT's ability to discern subtle code differences. To enhance its performance, they devised a novel approach blending ChatGPT with difference testing. Leveraging ChatGPT's strength in inferring expected behavior from erroneous programs, they synthesized programs that amplified

Table 12. The State-of-the-Art Applications of LLMs in Program Repair Task

Model	Baseline	Benchmark	Metric	Date	Reference
Codex	GPT-Neo, GPT-J, GPT-NeoX, CodeT5, InCoder	QuixBugs-Python and Java, Defects4J 1.2 and 2.0, ManyBugs	Correct/plausible patches	20 May 2023	[487]
Codex	CodeT5, CodeGen, PLBART, InCoder	Vul4J, VJBench,	Correct/plausible patches	29 May 2023	[483]
ChatGPT	Codex, CodeGen-16B, CodeGen-6B, CodeGen-2B, CodeGen-350M	QuixBugs-Python and Java	Correct/plausible patches	30 January 2023	[488]
ChatGPT	Codex, CodeBERT, SelfAPR, RewardRepair, Recoder, TBar, CURE, CoCoNuT	QuixBugs-Python and Java, Defects4J 1.2 and 2.0	Correct fixes	1 April 2023	[489]

subtle code differences. The experimental results reveal that this approach greatly increases the probability of finding the correct fault-inducing test case.

Others. In addition to the six software quality assurance tasks detailed above, LLMs can also be applied to defect detection [407, 478], GUI testing [264, 518], static analysis [125, 290], binary taint analysis [254], compiler fuzzing [346], decompilation [495], invariant prediction [335], malicious code localization [407], mobile app crash detection [265], and resource leak detection [445].

6.6 How Are LLMs Used in Software Maintenance?

Within the context of software maintenance, LLMs have been leveraged for bug prediction, program repair, code review, debugging, and an array of other activities.

Program Repair. The goal of APR is to automatically identify and fix bugs or defects in software [556]. It involves leveraging automated techniques to analyze buggy code and generate correct patches to address the identified issues. LLMs, such as BERT [426, 544], CodeBERT [215], CodeT5 [331], Codex [89, 173, 483], PLBART [331, 483], T5 [286, 521] and GPT series [33, 37, 210, 399, 427, 488, 489], have shown effectiveness in generating syntactically correct and contextually relevant code. Leveraging LLMs for program repair can achieve competitive performance in generating patches for various types of bugs and defects [489]. These models can effectively capture the underlying semantics and dependencies in the code [37], leading to the production of accurate and effective patches [488, 544]. Moreover, LLMs can be fine-tuned on specific code repair datasets [286], further improving their ability to generate high-quality patches for real-world software projects. The application of LLMs in program repair not only accelerates the bug-fixing process but also enables software developers to focus on more complex tasks, leading to enhanced software reliability and maintainability.

In recent research, program repair has emerged as a prevalent application. Among the LLMs, as shown in Table 12, Codex [483, 487] and ChatGPT [488] have particularly distinguished themselves in the program repair domain. *ChatGPT edges ahead due to its inherent interactive design, enabling a continuous feedback loop that yields refined and contextually apt patches* [488, 489]. Such conversational dynamics, coupled with rigorous comparisons across diverse baselines, underscore its superior adaptability and efficiency.

Summarizing several key findings from research on LLMs for program repair:

— *Interactive feedback.* Incorporating an interactive feedback loop, as observed with ChatGPT, significantly augments the accuracy of program repair [488]. This dynamic interplay between patch generation and validation fosters a deeper understanding of the software's semantics, leading to more effective repairs.

— *Domain-specific integration.* Merging the capabilities of LLMs with domain-specific knowledge and techniques further enhances their performance. Customized prompts, project-specific fine-tuning, and leveraging SE techniques [448, 487] can dramatically elevate the efficacy of LLM-driven program repairs.
— *Comparative analysis.* Rigorous evaluation against diverse baselines reveals the versatility and adaptability of LLMs, especially ChatGPT. This wide-ranging comparison not only establishes their superiority but also underscores areas for potential improvement [489].

Code Review. Code review is a critical quality assurance practice used to inspect, assess, and validate the quality and consistency of software code [379]. Code review aims to identify potential errors, vulnerabilities, and code quality issues, while also improving code maintainability, readability, and scalability. LLMs like BERT [379], ChatGPT [400, 477], and T5 [230, 435], trained on massive code repositories, possess the ability to understand and learn the semantics, structures, and contextual information of code [536]. In the code review process, LLMs assist reviewers in comprehensively understanding code intent and implementation details, enabling more accurate detection of potential issues and errors. Moreover, these models can generate suggestions for code improvements and optimizations, providing valuable insights and guidance to reviewers. Additionally, Widyasari et al. [477] have demonstrated that with suitable prompts LLMs can generate suitable explanations that help software engineers more readily accept suggested changes. By combining the intelligence of LLMs with the expertise of human reviewers, code review becomes more efficient and precise, further enhancing software quality and reliability.

Sentiment Analysis. Sentiment analysis involves determining emotions in text data related to software products, such as user feedback or comments [123, 155, 177]. The goal of sentiment analysis is to automatically classify the sentiment of the text as positive, negative, or neutral, providing valuable insights into how users perceive and react to software applications. Zhang et al. [553] conducted a study comparing pre-trained Transformer models like BERT, RoBERTa, XLNet, and ALBERT with existing SA4SE tools across six datasets. The results show that the Transformer models outperformed previous tools by 6.5% to 35.6% in macro/micro-averaged F1-scores, albeit with a tradeoff in runtime efficiency. However, this accuracy boost comes with some runtime costs, indicating that while Transformer models are less efficient than existing SA4SE approaches, their runtime cost is not prohibitively high.

Tag Recommendation. Improper tagging in software Q&A sites can lead to redundancy and other issues such as tag explosion. He et al. [130] introduced PTM4Tag, a framework utilizing PLMs with a triplet architecture to recommend tags for posts. By separately modeling the title, description, and code snippets of posts, PTM4Tag was compared using five popular PLMs, including BERT, CodeBERT, and so on. The SE-specialized CodeBERT showed the best performance, notably surpassing CNN-based methods. An ablation study revealed that while the title was crucial in tag prediction, using all post components achieved the optimal result.

Duplicate Bug Report Detection. In large software projects, multiple users may encounter and report the same or similar bugs independently, resulting in a proliferation of duplicate bug reports [158]. Duplicate bug report detection involves analyzing the textual content of bug reports and comparing them to find similarities and redundancies. LLM models, such as BERT [158], ChatGPT [550], and other transformer-based architectures, are well-suited for NL understanding and contextual representation. When applied to this task, LLMs can effectively capture the semantic similarities between bug reports, even in cases with slight variations in language or phrasing. The utilization of LLMs in this context not only enhances efficiency in managing bug reports but also contributes to improving the overall software development and maintenance workflow, reducing redundancy, and ensuring prompt bug resolution [549].

Bug Reproduction. Bug reports are crucial for software maintenance, allowing users to inform developers of problems encountered while using the software. Therefore, researchers have invested significant resources in automating error playback to speed up the software maintenance process. The success of current automated approaches depends heavily on the characteristics and quality of error reports, as they are limited by manually created schemas and predefined vocabularies. Inspired by the success of the LLMs in NL understanding, Feng et al. [91] propose AdbGPT, which utilizes NL understanding and logical reasoning capabilities of the LLM to extract **Steps to Reproduce (S2R)** entities from bug reports and guide the bug replay process based on the current GUI state. The researchers describe how cue engineering, a small amount of learning, and thought chain reasoning can be utilized to leverage the knowledge of the LLM for automated error replay. This approach is significantly lightweight compared to traditional approaches, which utilize a single LLM to address both phases of S2R entity extraction and guided replay through novel hint engineering.

Logging. Logging involves the systematic recording of events, messages, or information during the operation of a software application. It provides valuable information for understanding the behavior, performance, and potential problems of an application. Developers strategically insert logging statements throughout the code base to capture relevant data such as variable values, function calls, and error messages. These logs are an important tool for testing [39, 41], debugging [373], monitoring [127, 128], and analyzing the behavior of software operations, helping developers identify and diagnose bugs, performance bottlenecks, and other critical issues. Mastropaolo et al. [286] introduce LANCE, a system for automatically generating and injecting full log statements into Java code using the T5 model. Sridhara et al. [400] present that ChatGPT performs well in the log summarization task, generating aggregated results that are better than the current state of the art.

Debugging. Debugging targets identifying, locating, and resolving software defects or errors, commonly known as bugs. The debugging process involves scrutinizing the code, tracing the execution flow, and isolating the root cause of the problem to correct the error effectively. LLMs, such as BERT and other transformer-based architectures, excel at utilizing contextual information and NL understanding. In terms of debugging, LLMs can be used to simulate the scientific debugging process, such as AutoSD proposed by Kang et al. [183]. This model generates hypotheses about code problems and extracts relevant values to identify potential problems. In addition, the SELF-DEBUGGING method proposed by Chen et al. [45] enables LLM to debug its own generated code by learning a small number of presentations and explanations, which effectively improves the accuracy and sampling efficiency of code generation. Using LLMs in debugging not only improves fixing performance by generating competitive fixes but also provides insights into and explanations of the model's decision-making process, making it an important tool for improving software quality and developer productivity.

Vulnerability Repair. Vulnerability repair is the process of identifying and fixing security holes or weaknesses in software applications. Pearce et al. [333] investigate how to use LLMs for software zero-point vulnerability remediation. The authors explore the challenges faced in designing hints to induce LLMs to generate fixed versions of insecure code. It shows that while the approach is promising, with LLMs capable of fixing 100% of synthetic and hand-created scenarios, a qualitative assessment of the model's performance on a corpus of historical real-life examples reveals challenges in generating functionally correct code. It is concluded that despite the potential for future targeted LLM applications in this area, challenges remain. For a complete end-to-end system, the full system needs to be evaluated in conjunction with error localization and an improved testbed.

Code Clone Detection. Code clones are code samples that are identical to each other [24, 187]. These code samples can have structural or semantic equivalence [414]. Sharma et al. [382] investigate BERT's application in code clone detection through an exploratory study. Analyzing BERT's attention to code markers, they found that identifiers received higher attention, advocating their

use in clone detection. This insight enhanced clone detection across all layers, and the implications extended beyond BERT. The researchers suggest that these findings could lead to the development of smaller models with performance akin to larger ones, thus mitigating computational accessibility issues.

Bug Prediction. Gomes et al. [110] conduct a BERT and **Term Frequency-Inverted Document Frequency (TF-IDF)** application for long-lived bug prediction in **Free/Libre Open-Source Software (FLOSS)** study to compare their accuracy in predicting long-lived errors. The results show that BERT-based feature extraction consistently outperforms TF-IDF, demonstrating BERT's ability to capture the semantic context in error reports. In addition, smaller BERT architectures also show competitive results, highlighting the effectiveness of LLMs in bug prediction. This approach promises to enable more accurate error detection in FLOSS projects and improve software quality and maintenance.

Bug Triage. Bug triage is pivotal for effective issue management in large projects. It entails prioritizing bugs and assigning appropriate developers for resolution. While bug triage is straightforward for smaller projects, scalability brings complexity. Finding the right developers with the needed skills becomes intricate as bugs vary in expertise requirements. Some even demand combined skills, amplifying the intricacy. Lee et al. [216] introduce the Light Bug Triage framework. This innovative approach employs BERT to extract semantic information from bug reports. To surmount challenges with LLMs in bug triage, the researchers employ techniques like model compression, knowledge preservation fine-tuning, and a new loss function.

Program Merge Conflicts Repair. Program merge conflicts repair addresses the challenges faced when integrating individual code changes, which can lead to textual or semantic inconsistencies. Zhang et al. [534] explored the potential of using k-shot learning with LLMs like GPT-3 to automate this repair process. While these models showed promise in resolving semantic conflicts for Microsoft Edge, they didn't fully replace the benefits of domain-specific languages for certain synthesis patterns.

Traceability Recovery. Traceability recovery focuses on re-establishing lost or unclear connections between related software artifacts, thereby facilitating coherent software evolution and maintenance [105]. While traditional methods have offered some solutions, the integration of LLMs has recently emerged as a promising avenue for enhancing the accuracy and efficiency of this task. Zhu et al. [573] present TRACEFUN, a traceability link recovery framework enhanced with unlabeled data, serves as a testament to this potential, leveraging LLMs to bridge the gap between labeled and unlabeled data, thereby refining traceability link predictions.

Others. In addition to the 14 software maintenance tasks detailed above, LLMs can also be applied to review/commit/code classification [106, 204, 502], log parsing [263, 275, 520], code revision [180, 439], API misuses repair [552], code coverage prediction [434], code review explained [477], Code-Review defects repair [562], crash bug repair [75], dockerfile Repair [134], incivility detection [94], patch correctness prediction [545], patch detection [418], rename Refactoring [251], technical debt payback [285], web test repair [496], type error repair [54], and so on.

6.7 How Are LLMs Used in Software Management?

Research articles describing the utilization of LLMs in software management are still limited.

Effort Estimation. Effort estimation refers to the process of predicting the amount of time, resources, and manpower required to complete a software development project. Alhamed et al. [11] conduct an evaluation of the application of BERT in the task of effort estimation for software maintenance. Their study underscores BERT's potential to offer valuable insights and aid in the decision-making process while also highlighting the associated challenges and need for further investigation.

RQ4—Summary

(1) We categorized SE tasks into six activities: RE, software design, software development, software quality assurance, software maintenance, and software management. Subsequently, we summarized the specific applications of LLMs in these SE activities.
(2) We identified a total of 85 SE tasks and found that LLMs are most widely used in software development, with 229 articles mentioning over 24 SE tasks. The least applied area, software management, was mentioned in only three studies.
(3) *Code generation and program repair are the most prevalent tasks for employing LLMs in software development and maintenance activities.* We analyze the top-performing LLMs repeatedly validated in these tasks and summarize novel findings.

7 Threats to Validity

Article Search Omission. One key limitation is the possibility of omitting relevant articles during the search process. When gathering articles related to LLM4SE tasks from various publishers, it is possible to miss some articles due to incomplete summarization of keywords for SE tasks or LLMs. To address this concern, we adopted a comprehensive approach, combining manual search, automated search, and snowballing techniques, to minimize the risk of missing relevant articles. For manual search, we systematically searched for LLM articles related to SE tasks in six top-tier SE venues and extracted authoritative and comprehensive SE tasks and LLM keywords from these sources. Using these constructed search strings, we conducted automated searches on seven widely used publisher platforms. Additionally, to further augment our search results, we employed both forward and backward snowballing.

Study Selection Bias. Another limitation is the potential study selection bias. We established inclusion and exclusion criteria to perform the initial selection of articles, followed by manual verification based on QAC. This process involves a combination of automated and manual procedures. The automated selection process may result in mislabeling of articles due to incomplete or ambiguous information in their corresponding BibTeX records. To mitigate this issue, any articles that cannot be confidently excluded are temporarily retained for manual verification. However, the manual verification stage could be influenced by the subjective judgment and biases of the researchers, affecting the accuracy of the quality assessment of articles. To address these concerns, we invited two experienced reviewers in the fields of SE and LLM research to conduct a secondary review of the study selection results. The two reviewers raised objections to some of our results. Following a thorough discussion among all authors and reviewers regarding these issues, a consensus was ultimately reached. This step aims to enhance the accuracy of our article selection and minimize the likelihood of omission or misclassification. By using these measures, we strive to ensure that the selected articles are accurate and comprehensive, minimizing the impact of study selection bias and enhancing the reliability of our SLR. We additionally provide a replication package[4] for others to view.

Empirical Knowledge Bias. This SLR, along with 395 relevant studies in the LLM4SE field, answers four RQs. This implies the need for manual analysis and understanding of each study. In this process, there may be biases introduced by subjective judgments and experiential knowledge. To minimize potential errors in this regard, we have made the following efforts. First, in determining the RQs, as the first comprehensive overview of the LLM4SE field, we aim to provide a comprehensive

[4] https://github.com/security-pride/LLM4SE_SLR

interpretation of the current state and trends in this domain. Considering the commonality in AI4SE research, we referred to Yang et al.'s survey on DL4SE [509] during our RQ formulation. We finally decided to focus on LLM types, datasets, tuning, evaluation, and targeted SE tasks. Second, for the understanding and analysis of each study, to ensure accurate comprehension of article details, before addressing each RQ, we extensively reviewed relevant literature to predefine the approximate categories and details for each RQ. For example, in RQ3, based on prior work [369, 472, 559], we identified differences between tuning techniques for LLMs and those commonly used in traditional ML, such as prompt engineering and PEFT.

8 Challenges and Opportunities

8.1 Challenges

8.1.1 *Challenges in LLM Applicability.*

Model Size and Deployment. The size of LLMs has seen a marked increase over time, moving from GPT-1's 117M parameters to GPT-2's 1.5B, and further to GPT-3's 175B parameters [506]. The billions and even trillions [295] of parameters pose significant storage, memory, and computational challenges, which can hinder LLMs in resource-limited and real-time scenarios, especially when developers lack access to powerful GPUs or TPUs. CodeBERT [92], a pre-trained model proposed in 2019, has a total of 125 M parameters, resulting in a large model size of 476 MB. Recently proposed models like Codex [43] and CodeGen [312], have over 100 billion parameters and over 100 GB in size. The large sizes also require more computational resources. As pointed out by Hugging Face team [25], training a 176B model (i.e., BLOOM [374]) on 1.5 TB datasets consumes an estimated 1,082,880 GPU hours. Similarly, the training of the GPT-NeoX-20B model [27] on the Pile dataset [100], encompassing over 825 GiB of raw text data, requires the deployment of eight NVIDIA A100-SXM4-40GB GPUs. Each of these GPUs comes with a price tag of over 6,000 dollars [16], and the training extends to 1,830 hours or approximately 76 days. Moreover, even training a relatively smaller model like the PolyCoder (2.7B) [493], employing eight NVIDIA RTX 8000 GPUs on a single machine, demands a commitment of around 6 weeks. These examples illustrate the significant computational costs associated with training LLMs. These also have significant energy costs with predictions of massively increased energy usage by LLM-based platforms [359]. Fortunately, there are preliminary studies on reducing code models' size and improving their efficiency. Shi et al. [389] use a genetic algorithm to compress CodeBERT into only 3 MB and reduce its response latency by more than 70%. Overall, the challenge of increasing model sizes and efficient deployment requires further attention from the communities.

Data Dependency. In Section 4, we provide a detailed analysis of the datasets used in 395 studies and the data pre-processing process, finding that LLMs rely heavily on a large number of different datasets for training and fine-tuning, posing the data dependency challenge. The quality, diversity, and quantity of data directly affect the performance and generalizability of the models. Given their size, LLMs often require large amounts of data to capture nuances, but obtaining such data can be challenging. Relying on limited or biased datasets may cause the model to inherit these biases, resulting in biased or inaccurate predictions. In addition, the domain-specific data required for fine-tuning can be a bottleneck. Due to the relatively short period of time since the emergence of LLM, such large-scale datasets are still relatively rare, especially in the SE domain. Another issue is the risk of benchmark data contamination, where training and test data overlaps could lead to inflated performance metrics [561]. For instance, Brown et al. [29] discovered a code bug that prevented them from fully removing all overlapping data. They were unable to afford retraining and resorted to using "cleaned" variants of the benchmarks to mitigate the issue. Moreover, there are grave concerns around the inclusion of **Personally Identifiable Information (PII)** in pre-training

corpora. Instances of PII, such as phone numbers and e-mail addresses, have led to privacy leaks during the prompting process [80, 206].

Ambiguity in Code Generation. Ambiguity in code generation poses a significant challenge for LLMs in SE tasks. When code intent is unclear (e.g., multiple valid solutions exist), LLMs may struggle to produce accurate and contextually appropriate code. This can lead to syntactically correct but functionally incorrect code, impacting the reliability and effectiveness of LLM-based code generation. Addressing this issue requires exploring techniques to incorporate additional context, domain-specific knowledge, or multi-model ensembles to improve LLMs' ability to handle ambiguity and generate precise code, ensuring their successful integration into real-world software development processes.

8.1.2 Challenges in LLM Generalizability.

The generalizability of LLMs refers to the ability of these models to consistently and accurately perform tasks in different tasks, datasets, or domains outside their training environment. While LLMs are trained on massive amounts of data, ensuring extensive knowledge capture, their performance is sometimes problematic when confronted with specific or idiosyncratic tasks outside the scope of their training. This challenge is particularly evident in the SE domain, where we present the application of LLMs to 85 SE tasks in Section 6. We observed that the context and semantics of code or documents vary greatly across projects, languages, or domains. Ensuring that the LLM generalizes well requires careful fine-tuning, validation on different datasets, and continuous feedback loops. Without these measures, models run the risk of over-adapting their training data, thus limiting their usefulness in a variety of real-world applications. Recent studies have shown that the LLMs cannot generalize their good performance to inputs after semantic-preserving transformations. For example, Yang et al. [510] show that the performance of CodeBERT on different tasks decreases significantly after substituting the variables' names in the input.

8.1.3 Challenges in LLM Evaluation.

We summarized key evaluation metrics used in different types of SE tasks according to four task types: regression, classification, recommendation, and generation (Section 6). We found that when applying LLMs in the SE domain, the methodology for evaluating the performance of the models is usually based on a set of predefined metrics. Unfortunately, these metrics (e.g., Accuracy, Recall, or F1-score), while useful in some cases, may not fully capture all the effects and impacts of a model in a given SE task. For example, a model may perform well in terms of accuracy but may fail in processing specific types of inputs or in some specific situations. In addition, these metrics may not capture certain qualitative aspects of the model, such as its interpretability, robustness, or sensitivity to specific types of errors. Some of the most recent studies on LLM4SE tasks [3, 141, 398, 495, 522, 541], in which researchers customized some evaluation metrics to assess the performance of models, also further illustrate the limitations of some of the widely used evaluation metrics in the field of LLM.

8.1.4 Challenges in LLM Interpretability, Trustworthiness, and Ethical Usage.

Interpretability and trustworthiness are crucial aspects in the adoption of LLMs for SE tasks. The challenge lies in understanding the decision-making process of these models, as their black-box nature often makes it difficult to explain why or how a particular code snippet or recommendation is generated. Recent studies [228, 441, 512] also show that LLM of code trained on low-quality datasets can have vulnerabilities (e.g., generating insecure code). The lack of interpretability and trustworthiness can lead to uncertainty and hesitation among developers, who may be hesitant to rely on LLM-generated code without a clear understanding of how it was derived. Establishing trust in LLMs requires efforts to develop techniques and tools that provide insights into the model's

internal workings and enable developers to comprehend the reasoning behind the generated outputs. Enhancing interpretability and trustworthiness can ultimately promote the widespread adoption of LLMs in SE, leading to more efficient and effective development practices. Many LLMs are not open and it is unclear what data they have been trained on, both quality and representativeness but also ownership of the source training data. This brings into question ownership of the derivative data, e.g., generated designs, code, or test cases. There is also potential for various adversarial attacks, e.g., deliberately seeding LLMs with code vulnerabilities so that automatically generated code snippets have subtle but vulnerable aspects.

8.2 Opportunities

8.2.1 Optimization of LLM4SE.

The Advent of Code-Specialized LLMs in SE. The recent emergence of code-specialized LLMs, such as GitHub Copilot [109], Amazon's CodeWhisperer [15], OpenAI Code Interpreter [319] integrated into ChatGPT, and Code Llama [289] from Meta's Llama family, signals a transformative phase in LLM4SE. These specialized LLMs, fine-tuned on code-specific datasets, are not merely incremental improvements but paradigm shifts in code understanding, generation, and efficiency. They offer new avenues for automated coding, personalized developer assistance, enhanced code review, and quality assurance, among other tasks, setting the stage for groundbreaking advancements in the SE domain.

Influence and Applications of ChatGPT. ChatGPT's popularity in recent academic research, as evidenced by its large presence in our 395 analyzed articles, emphasizes its escalating influence and acceptance within academia. Researchers' preference for ChatGPT over other LLMs and LLM-based applications since its release can be attributed to its computational efficiency, adaptability to various tasks, and potential cost-effectiveness [212, 224, 488]. Its applications extend beyond mere code efficiency and debugging, fostering a collaborative era in development. This paradigm shift signifies a broader move toward integrating advanced NL understanding into conventional coding practices [212, 276, 368]. By thoughtfully analyzing these dynamics and trends, we can foresee the potential pathways for LLMs and LLM applications like ChatGPT in shaping more robust, efficient, and collaborative software development procedures. Such insights stand as a promising indication of the future revolutionary impact of LLMs on SE.

Performance Enhancement from Task-Specific Model Training. The choice between leveraging commercially available pre-trained models like GPT-4 and building upon open-source frameworks such as Llama 2 [432], Gemma [113], and Mistral [8] provides a nuanced set of options for individual or organizational customization in specialized tasks. The distinction between these two approaches lies in the degree of control and customization. Pre-trained models like GPT-4 are generally not designed for large-scale retraining due to their proprietary nature, but they allow quick task-specific adaptations with limited data, thereby minimizing computational overhead. On the other hand, frameworks like LLaMA offer an open-source foundation for more extensive customization. While they come pre-trained, organizations often modify the source code and retrain these models on their own large-scale datasets to meet specialized requirements [136, 517]. This process is computationally intensive, leading to greater resource allocation and cost, but affords the advantage of creating highly domain-specific models. Hence, the primary tradeoff is between the ease of use and quick deployment offered by models like GPT-4, and the deep customization capabilities but higher computational demands associated with open-source frameworks like LLaMA.

Collaborative LLMs. From our review, it is evident that LLMs have made significant strides in addressing various SE challenges. However, as the complexity of SE tasks continues to grow, there's an emerging need for more sophisticated and tailored solutions. One promising direction is the concept of Collaborative LLMs. This approach involves integrating multiple LLMs [73, 560, 570] or combining LLMs with specialized ML models [83, 533] to enhance their efficacy for SE tasks.

By harnessing the collective strengths of different models, we believe that the SE community can achieve more precise and efficient outcomes, from code completion to bug detection.

8.2.2 *Expanding LLM's NLP Capabilities in More SE Phases.*

Integration of New Input Forms. In our analysis, we observed that the predominant input forms were code-based datasets and text-based datasets. However, there was a noticeable scarcity of graph-based datasets [203] (Section 4). Leveraging new input forms of NL, such as spoken language, diagrams, and multimodal inputs, presents an opportunity to enhance the LLMs' ability to understand and process diverse user requirements. Integrating spoken language could improve interactions between developers and models, enabling more natural and context-rich communication. Diagrams can facilitate visual representations of code and requirements, offering a complementary perspective for code generation. Furthermore, multimodal inputs that combine text, audio, and visual cues could offer a more comprehensive context understanding, leading to more accurate and contextually appropriate code generation. Additionally, exploring graph-based datasets could be crucial for addressing complex code scenarios, as graphs capture the structural relationships and dependencies in code, allowing LLMs to better comprehend code interactions and dependencies.

Widening LLM Applications across SE Phases. We observed a pronounced emphasis on the application of LLMs in software development, quality assurance, and maintenance. These areas have undoubtedly benefited from the capabilities of LLMs, leading to enhanced code completion [160, 256], decompilation [383, 495], bug detection [91, 183], and other related tasks. The current application of LLMs in RE, software design, and software management remains relatively sparse. This presents a significant opportunity: by expanding the use of LLMs to these under-explored areas, we can potentially improve how requirements are elicited, how software designs are conceptualized, and how projects are managed.

8.2.3 *Enhancing LLM's Performance in Existing SE Tasks.*

Tackling Domain-Specific Challenges. Many SE domains, including safety-critical systems and specific industries, suffer from a scarcity of open-source datasets, hindering the application of LLMs in these specialized areas. Future research can focus on creating domain-specific datasets and fine-tuning LLMs to cater to the unique challenges and intricacies of these fields [26, 411]. Collaboration with domain experts and practitioners is vital to curate relevant data, and fine-tuning LLMs on this data can enhance their effectiveness and ensure better alignment with the specific requirements of each domain, paving the way for LLMs to address real-world challenges [30] in diverse SE domains [232].

Establishing a Comprehensive Evaluation Framework for LLM4SE. The necessity for a universal, yet adaptable, evaluation framework for LLM4SE is pressing for both academic and industrial sectors. In academia, such a framework enables streamlined assessments of LLM performance, efficacy, and limitations, serving as a benchmark to verify the models' practical readiness. On the industrial side, collaborations with real-world development teams using this framework yield empirical insights into LLMs' utility, including their impacts on productivity, code quality, and team collaboration, while also revealing challenges like model biases, misinterpretation of code semantics, and context-specific limitations. Establishing this framework is critical for standardizing assessments and facilitating responsible LLM adoption in both academic research and practical applications [26, 111].

8.3 Roadmap

We provide a roadmap for future development in leveraging LLM4SE, with an additional high-level perspective that acknowledges the reciprocal relationship and emerging exploration of **Software Engineering for Large Language Models (SE4LLM)**.

Automated Coding, Development, and Personalized Developer Assistance. The pursuit of automation in coding encompasses the auto-generation of code snippets, bug fixes, system optimization, and the creation of intelligent, personalized assistance for developers that is context-aware and adaptable to individual needs. LLM's generative capabilities can be leveraged to help developers better understand requirements and generate syntactically and semantically correct code, thereby accelerating development cycles and improving software quality. Leveraging LLM's NL processing to develop context-aware tools allows for interaction with developers in a more intuitive and responsive manner. Additionally, fine-tuning LLMs for specific coding tasks and developer assistance can further enhance their accuracy and efficiency, customizing the automation process to suit the unique demands of different projects and individuals. Still, important concerns of trust (how developers can put trust in LLM-powered agents) and synergy (how developers can work well hand-in-hand with LLM-powered agents) need to be addressed for such assistance to be widely adopted [267].

Advancing Testing and Analysis. The inclusion of LLMs in software testing methods opens up avenues for enhanced test case generation, bug classification, and defect prediction, thereby improving the precision and efficiency of the software testing process. For instance, LLMs show potential to be fine-tuned to a project's specific requirements to generate customized test cases, which elevates the likelihood of early detection of subtle bugs or security vulnerabilities. Furthermore, the integration of LLMs with traditional SE techniques, including both static and dynamic program analysis presents a compelling direction for more rigorous code analysis [50]. The potential for utilizing LLMs in formal analysis methodologies, including formal verification, is another area that merits investigation [37]. These advancements not only facilitate the early discovery of complex errors but also lead to reduced development costs and quicker time-to-market, ultimately contributing to the robustness and reliability of the software products.

Integrating Programming Knowledge into LLMs. One critical future direction lies in the integration of specialized code representation methods and programming domain knowledge into LLM4SE [277, 442]. This integration aims to enhance the capability of LLMs to generate code that is not only functionally accurate but also secure and compliant with programming standards. Leveraging advanced techniques in code embedding, syntax tree parsing, and semantic analysis could significantly refine the generation capabilities of LLMs. Moreover, embedding domain-specific rules and best practices into these models would enable them to auto-generate code that adheres to industry or language-specific guidelines for security and style.

Enhanced Code Review and Quality Assurance. The transformation of the code review process can be supported by employing LLMs to analyze code context, perform intelligent comparisons, and offer insights that go beyond traditional automated review systems. The application of fine-tuned LLMs for code review can allow for more precise error detection and tailored feedback, offering a more nuanced understanding of code quality and potential improvements.

Extracting Insights from Data Mining. LLMs can play a critical role in mining insights from platforms like GitHub, StackOverflow, and app stores. Through the application in tasks such as requirement extraction, traceability, validation, and various types of mining (tag, app, developer-based), LLMs can provide valuable insights that inform development strategies and decision-making. By automating and enhancing these mining tasks, LLMs contribute to a deeper understanding of user needs, emerging trends, and the efficiency of development practices.

Empowering Predictive Analytics and Decision Support. Leveraging LLMs for effort cost prediction, software classification, code classification, incident detection, and software quality evaluation may support better data-driven insights and predictive analytics. This empowers organizations to make informed decisions throughout the development lifecycle. LLMs' ability to model and analyze vast

amounts of data enables more accurate forecasts of project timelines, resource needs, and potential risks.

LLMs in Software Security. The growing impact of LLM4SE offers both unparalleled opportunities and challenges in the domain of software security. On the one hand, LLMs offer promising solutions for automated security audits, compliance verifications, and vulnerability detection. These models can potentially be leveraged for automated code reviews to ensure compliance with industry standards and legal regulations, while also identifying potential security vulnerabilities [4, 62, 91, 93, 126, 333]. For instance, Ferrag et al. [93] showcased the efficacy of LLMs in cyber reasoning tasks related to software security. On the other hand, the usage of LLMs introduces novel security concerns. Their complexity makes them susceptible to attacks, demanding novel strategies to fortify the models themselves [61, 81, 258, 352, 353, 481]. As an example, Wu et al. [481] delve into methods to secure LLMs against jailbreak attacks. An intriguing direction for future research lies in enabling LLMs to automatically identify and rectify their own vulnerabilities. Specifically, the focus could be on equipping LLMs to generate self-applied patches to their underlying code, thereby enhancing their inherent security, as opposed to merely implementing application-layer restrictions. Given this landscape, future research should adopt a balanced approach, aiming to exploit LLMs for automating and enhancing existing software security protocols while concurrently developing techniques to secure the LLMs themselves. This dual focus is crucial for fully realizing the potential of LLMs in enhancing the security and compliance assurance of software systems.

SE4LLM. As the capabilities and complexities of LLMs continue to expand, there arises a reciprocal need for specialized SE practices tailored for the development, optimization, and maintenance of these models. These include LLMs used for SE purposes (i.e., SE4LLM4SE). SE4LLM encompasses a range of challenges and opportunities, including the design of scalable and maintainable architectures, the creation of efficient training algorithms, the development of rigorous testing frameworks for model robustness and fairness, and the implementation of ethical guidelines and compliance mechanisms. The convergence of SE with LLMs not only facilitates the growth of more sophisticated and adaptable models but also opens up new avenues for interdisciplinary research and innovation, bringing together the expertise of both the AI and SE communities. This aligns with a broader vision where SE practices become an integral part of the lifecycle of LLMs, ensuring their robustness, efficiency, and ethical alignment with societal values. While some works exist in assessing and improving the robustness and efficiency of LLMs, including LLMs for SE, such as work by Yang et al. [510] and Shi et al. [389], much more work is needed [511].

9 Conclusion

LLMs are bringing significant changes to the field of SE. The potential of these models to handle complex tasks can fundamentally reshape many SE practices and tools. In this SLR, we analyzed the emerging utilization of LLMs for software engineering, encompassing articles published since the inception of the first LLM (BERT). We examined the diverse LLMs that have been employed in SE tasks and explored their distinct features and applications (RQ1). We then investigated the processes involved in data collection, pre-processing, and usage, emphasizing the significant role well-curated datasets play in the successful application of LLMs to solve SE tasks (RQ2). Following this, we investigated the various strategies utilized to optimize and assess the performance of LLMs for SE tasks (RQ3). Lastly, we reviewed the wide range of SE tasks where LLMs have been applied to date, shedding light on the practical contributions LLMs have made (RQ4). We summarized some key existing challenges of LLM4SE and provided a research roadmap, outlining promising future research directions.

References

[1] Mayank Agarwal, Yikang Shen, Bailin Wang, Yoon Kim, and Jie Chen. 2024. Structured code representations enable data-efficient adaptation of code language models (2024). arXiv:2401.10716. Retrieved from https://arxiv.org/abs/2401.10716

[2] Emad Aghajani, Csaba Nagy, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, Michele Lanza, and David C. Shepherd. 2020. Software documentation: The practitioners' perspective. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 590–601.

[3] Lakshya Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram Rajamani. 2023. Monitor-guided decoding of code lms with static analysis of repository context. In *Advances in Neural Information Processing Systems*. A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 32270–32298. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2023/file/662b1774ba8845fc1fa3d1fc0177ceeb-Paper-Conference.pdf

[4] Baleegh Ahmad, Shailja Thakur, Benjamin Tan, Ramesh Karri, and Hammond Pearce. 2023. Fixing hardware security bugs with large language models. arXiv:2302.01215.

[5] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. arXiv:2103.06333.

[6] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl T. Barr. 2023. Improving few-shot prompts with relevant static analysis products. arXiv:2304.06815.

[7] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl T. Barr. 2024. Automatic semantic augmentation of language model prompts (for code summarization). arXiv:2304.06815.

[8] Mistral AI. 2023. Mistral. Retrieved from https://mistral.ai/

[9] Ali Al-Kaswan, Toufique Ahmed, Maliheh Izadi, Anand Ashok Sawant, Premkumar Devanbu, and Arie van Deursen. 2023. Extending source code pre-trained language models to summarise decompiled binarie. In *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '23)*. IEEE, 260–271.

[10] Ajmain I. Alam, Palash R. Roy, Farouq Al-Omari, Chanchal K. Roy, Banani Roy, and Kevin A. Schneider. 2023. Gptclonebench: A comprehensive benchmark of semantic clones and cross-language clones using GPT-3 model and semanticclonebench. In *Proceedings of the 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME '23)*. IEEE, 1–13.

[11] Mohammed Alhamed and Tim Storer. 2022. Evaluation of context-aware language models and experts for effort estimation of software maintenance issues. In *Proceedings of the 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME '22)*. IEEE, 129–138.

[12] Frances E. Allen. 1970. Control flow analysis. *ACM SIGPLAN Notices* 5, 7 (1970), 1–19.

[13] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-guided synthesis. In *2013 Formal Methods in Computer-Aided Design*. 1–8. DOI: https://doi.org/10.1109/FMCAD.2013.6679385

[14] Sven Amann, Sebastian Proksch, Sarah Nadi, and Mira Mezini. 2016. A study of visual studio usage in practice. In *Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER '16)*, Vol. 1. IEEE, 124–134.

[15] Amazon. 2023. Amazon codewhisperer. Retrieved from https://aws.amazon.com/cn/codewhisperer/

[16] Amazon. 2023. Nvidia tesla a100 ampere 40 gb graphics card - pcie 4.0 - dual slot. Retrieved from https://www.amazon.com/NVIDIA-Tesla-A100-Ampere-Graphics/dp/B0BGZJ27SL

[17] M. Anon. 2022. National vulnerability database. Retrieved from https://www.nist.gov/programs-projects/national-vulnerability-database-nvd

[18] Anthropic. 2023. Claude. Retrieved from https://www.anthropic.com/claude

[19] Shushan Arakelyan, Rocktim Jyoti Das, Yi Mao, and Xiang Ren. 2023. Exploring distributional shifts in large language models for code analysis. arXiv:2303.09128.

[20] Amos Azaria, Rina Azoulay, and Shulamit Reches. 2023. CHATGPT is a remarkable tool–for experts. arXiv:2306.03102.

[21] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2023. Codeplan: Repository-level coding using llms and planning. arXiv:2309.12499.

[22] Patrick Bareiß, Beatriz Souza, Marcelo d'Amorim, and Michael Pradel. 2022. Code generation tools (almost) for free? A study of few-shot, pre-trained language models on code. arXiv:2206.01335.

[23] Rabih Bashroush, Muhammad Garba, Rick Rabiser, Iris Groher, and Goetz Botterweck. 2017. Case tool support for variability management in software product lines. *ACM Computing Surveys* 50, 1 (2017), 1–45.

[24] Ira D. Baxter, Andrew Yahin, Leonardo Moura, Marcelo Sant'Anna, and Lorraine Bier. 1998. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (Cat. No. 98cb36272)*. IEEE, 368–377.

[25] Stas Bekman. 2022. The technology behind bloom training. Retrieved from https://huggingface.co/blog/bloom-megatron-deepspeed
[26] Eeshita Biswas, Mehmet Efruz Karabulut, Lori Pollock, and K. Vijay-Shanker. 2020. Achieving reliable sentiment analysis in the software engineering domain using bert. In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME '20)*. IEEE, 162–173.
[27] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. 2022. Gpt-neox-20b: An open-source autoregressive language model. arXiv:2204.06745. Retrieved from https://arxiv.org/abs/2204.06745
[28] Sid Black, Gao Leo, Phil Wang, Connor Leahy, and Stella Biderman. 2021. GPT-Neo: Large scale autoregressive language modeling with mesh-tensorflow. Retrieved from https://doi.org/10.5281/zenodo.5297715
[29] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems,* Vol. 33, 1877–1901.
[30] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of artificial general intelligence: Early experiments with GPT-4. arXiv:2303.12712.
[31] Nghi D. Q. Bui, Hung Le, Yue Wang, Junnan Li, Akhilesh Deepak Gotmare, and Steven C. H. Hoi. 2023. Codetf: One-stop transformer library for state-of-the-art code LLM. arXiv:2306.00029.
[32] Alessio Buscemi. 2023. A comparative study of code generation using chatgpt 3.5 across 10 programming languages. arXiv:2308.04477.
[33] Jialun Cao, Meiziniu Li, Ming Wen, and Shing-chi Cheung. 2023. A study on prompt design, advantages and limitations of chatgpt for deep learning program repair. arXiv:2304.08191.
[34] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. 2023. MultiPLe: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering* 49, 7 (July 2023), 3675–3691. DOI: https://doi.org/10.1109/TSE.2023.3267446
[35] Aaron Chan, Anant Kharkar, Roshanak Zilouchian Moghaddam, Yevhen Mohylevskyy, Alec Helyar, Eslam Kamal, Mohamed Elkamhawy, and Neel Sundaresan. 2023. Transformer-based vulnerability detection in code at edittime: Zero-shot, few-shot, or fine-tuning? arXiv:2306.01754.
[36] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Kaijie Zhu, Hao Chen, Linyi Yang, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. 2023. A survey on evaluation of large language models. arXiv:2307.03109.
[37] Yiannis Charalambous, Norbert Tihanyi, Ridhi Jain, Youcheng Sun, Mohamed Amine Ferrag, and Lucas C. Cordeiro. 2023. A new era in software security: Towards self-healing software via large language models and formal verification. arXiv:2305.14752.
[38] Angelica Chen, Jérémy Scheurer, Tomasz Korbak, Jon Ander Campos, Jun Shern Chan, Samuel R. Bowman, Kyunghyun Cho, and Ethan Perez. 2023. Improving code generation by training with natural language feedback. arXiv:2303.16749.
[39] Boyuan Chen, Jian Song, Peng Xu, Xing Hu, and Zhen Ming Jiang. 2018. An automated approach to estimating code coverage measures via execution logs. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 305–316.
[40] Fuxiang Chen, Fatemeh H. Fard, David Lo, and Timofey Bryksin. 2022. On the transferability of pre-trained language models for low-resource programming languages. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, 401–412.
[41] Jinfu Chen, Weiyi Shang, Ahmed E. Hassan, Yong Wang, and Jiangbin Lin. 2019. An experience report of generating load tests using log-recovered workloads at varying granularities of user behaviour. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE '19)*. IEEE, 669–681.
[42] Long Chen, Wei Ye, and Shikun Zhang. 2019. Capturing source code semantics via tree-based convolution over api-enhanced ast. In *Proceedings of the 16th ACM International Conference on Computing Frontiers*, 174–182.
[43] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert,

Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. arXiv:2107.03374.

[44] Meng Chen, Hongyu Zhang, Chengcheng Wan, Zhao Wei, Yong Xu, Juhong Wang, and Xiaodong Gu. 2023. On the effectiveness of large language models in domain-specific code generation. arXiv:2312.01639.

[45] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. arXiv:2304.05128.

[46] Xinyun Chen, Chang Liu, and Dawn Song. 2017. Towards synthesizing complex programs from input-output examples. arXiv:1706.01284.

[47] Xinyun Chen, Dawn Song, and Yuandong Tian. 2021. Latent execution for neural program synthesis beyond domain-specific languages. In *Advances in Neural Information Processing Systems*, Vol. 34, 22196–22208.

[48] Yizheng Chen, Zhoujie Ding, Xinyun Chen, and David Wagner. 2023. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. arXiv:2304.00409.

[49] Yujia Chen, Cuiyun Gao, Muyijie Zhu, Qing Liao, Yong Wang, and Guoai Xu. 2024. APIGen: Generative API method recommendation. arXiv:2401.15843.

[50] Yiming Liu, Cen Zhang, Feng Li, Yeting Li, Jianhua Zhou, Jian Wang, Lanlan Zhan, Yang Liu, and Wei Huo. 2024. Semantic-enhanced static vulnerability detection in baseband firmware. In *Proceedings of the 46th International Conference on Software Engineering (ICSE 2024)*. ACM, New York, NY, 12 pages. DOI: https://doi.org/10.1145/3597503.3639158

[51] Liying Cheng, Xingxuan Li, and Lidong Bing. 2023. Is GPT-4 a good data analyst? arXiv:2305.15038.

[52] The Vicuna Team. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. Retrieved from https://lmsys.org/blog/2023-03-30-vicuna/

[53] Muslim Chochlov, Gul Aftab Ahmed, James Vincent Patten, Guoxian Lu, Wei Hou, David Gregg, and Jim Buckley. 2022. Using a nearest-neighbour, bert-based approach for scalable clone detection. In *Proceedings of the 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME '22)*. IEEE, 582–591.

[54] Yiu Wai Chow, Luca Di Grazia, and Michael Pradel. 2024. Pyty: Repairing static type errors in python. arXiv:2401.06619.

[55] Agnieszka Ciborowska and Kostadin Damevski. 2022. Fast changeset-based bug localization with bert. In *Proceedings of the 44th International Conference on Software Engineering*, 946–957.

[56] Agnieszka Ciborowska and Kostadin Damevski. 2023. Too few bug reports? Exploring data augmentation for improved changeset-based bug localization. arXiv:2305.16430.

[57] Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, and Gabriele Bavota. 2021. An empirical study on the usage of transformer models for code completion. *IEEE Transactions on Software Engineering* 48, 12 (2021), 4818–4837.

[58] Colin B. Clement, Dawn Drain, Jonathan Timcheck, Alexey Svyatkovskiy, and Neel Sundaresan. 2020. Pymt5: Multi-mode translation of natural language and python code with transformers. arXiv:2010.03150.

[59] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2023. Effective test generation using pre-trained large language models and mutation testing. arXiv:2308.16557.

[60] Pantazis Deligiannis, Akash Lal, Nikita Mehrotra, and Aseem Rastogi. 2023. Fixing rust compilation errors using llms. arXiv:2308.05177.

[61] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2023. Jailbreaker: Automated jailbreak across multiple large language model chatbots. arXiv:2307.08715.

[62] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2023. Pentestgpt: An llm-empowered automatic penetration testing tool. arXiv:2308.06782.

[63] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. Association for Computing Machinery, New York, NY, 423–435. DOI: https://doi.org/10.1145/3597926.3598067

[64] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt. arXiv:2304.02014.

[65] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805.

[66] Juri Di Rocco, Davide Di Ruscio, Claudio Di Sipio, Phuong T Nguyen, and Riccardo Rubei. 2021. Development of recommendation systems for software engineering: The crossminer experience. *Empirical Software Engineering* 26, 4 (2021), 69.

[67] Victor Dibia, Adam Fourney, Gagan Bansal, Forough Poursabzi-Sangdeh, Han Liu, and Saleema Amershi. 2022. Aligning offline metrics and human judgments of value of ai-pair programmers. arXiv:2210.16494.
[68] Hantian Ding, Varun Kumar, Yuchen Tian, Zijian Wang, Rob Kwiatkowski, Xiaopeng Li, Murali Krishna Ramanathan, Baishakhi Ray, Parminder Bhatia, Sudipta Sengupta, Dan Roth, and Bing Xiang. 2023. A static evaluation of code completion by large language models. arXiv:2306.03203.
[69] Tuan Dinh, Jinman Zhao, Samson Tan, Renato Negrinho, Leonard Lausen, Sheng Zha, and George Karypis. 2024. Large language models of code fail at completing code with potential bugs. In *Advances in Neural Information Processing Systems*. A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 41386–41412. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2023/file/819cebb05f993840e8a52d7564c5c282-Paper-Conference.pdf
[70] Jean-Baptiste Döderlein, Mathieu Acher, Djamel Eddine Khelladi, and Benoit Combemale. 2022. Piloting copilot and codex: Hot temperature, cold prompts, or black magic? arXiv:2210.14699.
[71] Guanting Dong, Hongyi Yuan, Keming Lu, Chengpeng Li, Mingfeng Xue, Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, and Jingren Zhou. 2023. How abilities in large language models are affected by supervised fine-tuning data composition. arXiv:2310.05492.
[72] Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2023. Codescore: Evaluating code generation by learning code execution. arXiv:2301.09043.
[73] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration code generation via ChatGPT. arXiv:2304.07590.
[74] Shihan Dou, Junjie Shan, Haoxiang Jia, Wenhao Deng, Zhiheng Xi, Wei He, Yueming Wu, Tao Gui, Yang Liu, and Xuanjing Huang. 2023. Towards understanding the capability of large language models on code clone detection: A survey. arXiv:2308.01191.
[75] Xueying Du, Mingwei Liu, Juntao Li, Hanlin Wang, Xin Peng, and Yiling Lou. 2023. Resolving crash bugs via large language models: An empirical study. arXiv:2312.10448.
[76] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. arXiv:2308.01861. Retrieved from https://arxiv.org/abs/2308.01861
[77] Yali Du and Zhongxing Yu. 2023. Pre-training code representation with semantic flow graph for effective bug localization. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 579–591.
[78] Aryaz Eghbali and Michael Pradel. 2024. De-hallucinator: Iterative grounding for llm-based code completion. arXiv:2401.01701.
[79] Abdelkarim El-Hajjami, Nicolas Fafin, and Camille Salinesi. 2023. Which ai technique is better to classify requirements? An experiment with SVM, LSTM, and ChatGPT. arXiv:2311.11547.
[80] El-Mahdi El-Mhamdi, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Lê-Nguyên Hoang, Rafael Pinot, Sébastien Rouault, and John Stephan. 2023. On the impossible safety of large ai models. arXiv:2209.15259.
[81] Andre Elizondo. 2023. Langkit: Making large language models safe and responsible. Retrieved from https://whylabs.ai/blog/posts/langkit-making-large-language-models-safe-and-responsible
[82] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2023. Formalizing natural language intent into program specifications via large language models. arXiv:2310.01831. Retrieved from https://arxiv.org/abs/2310.01831
[83] Saad Ezzini, Sallam Abualhaija, Chetan Arora, and Mehrdad Sabetzadeh. 2022. Automated handling of anaphoric ambiguity in requirements: A multi-solution study. In *Proceedings of the 44th International Conference on Software Engineering*, 187–199.
[84] Sarah Fakhoury, Saikat Chakraborty, Madan Musuvathi, and Shuvendu K. Lahiri. 2023. Towards generating functionally correct code edits from natural language issue descriptions. arXiv:2304.03816. Retrieved from https://arxiv.org.abs/2304.03816
[85] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large language models for software engineering: Survey and open problems. arXiv:2310.03533. Retrieved from https://arxiv.org/abs/2310.03533
[86] Guodong Fan, Shizhan Chen, Cuiyun Gao, Jianmao Xiao, Tao Zhang, and Zhiyong Feng. 2024. Rapid: Zero-shot domain adaptation for code search with pre-trained models. *ACM Transactions on Software Engineering and Methodology* 33, 5, (June 2024), Article 128, 35 pages. DOI: https://doi.org/10.1145/3641542
[87] Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, and Qing Li. 2023. Recommender systems in the era of large language models (LLMS). arXiv:2307.02046. Retrieved from https://arxiv.org/abs/2307.02046
[88] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*. IEEE, 1469–1481.

[89] Zhiyu Fan, Xiang Gao, Abhik Roychoudhury, and Shin Hwei Tan. 2022. Automated repair of programs from large language models. arXiv:2205.10583. Retrieved from https://arxiv.org/abs/2205.10583

[90] Sakina Fatima, Taher A. Ghaleb, and Lionel Briand. 2023. Flakify: A black-box, language model-based predictor for flaky tests. *IEEE Transactions on Software Engineering* 49, 4 (April 2023), 1912–1927. DOI: https://doi.org/10.1109/TSE.2022.3201209

[91] Sidong Feng and Chunyang Chen. 2023. Prompting is all your need: Automated android bug replay with large language models. arXiv:2306.01987.

[92] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. Codebert: A pre-trained model for programming and natural languages. arXiv:2002.08155.

[93] Mohamed Amine Ferrag, Ammar Battah, Norbert Tihanyi, Merouane Debbah, Thierry Lestable, and Lucas C. Cordeiro. 2023. Securefalcon: The next cyber reasoning system for cyber security. arXiv:2307.06616. Retrieved from https://arxiv.org/abs/2307.06616

[94] Isabella Ferreira, Ahlaam Rafiq, and Jinghui Cheng. 2024. Incivility detection in open source code review and issue discussions. *Journal of Systems and Software* 209 (2024), 111935.

[95] Emily First, Markus Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1229–1241.

[96] Gordon Fraser, Matt Staats, Phil McMinn, Andrea Arcuri, and Frank Padberg. 2015. Does automated unit test generation really help software testers? A controlled empirical study. *ACM Transactions on Software Engineering and Methodology* 24, 4 (2015), 1–49.

[97] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. arXiv:2204.05999. Retrieved from https://arxiv.org/abs/2204.05999

[98] Michael Fu and Chakkrit Tantithamthavorn. 2022. Gpt2sp: A transformer-based agile story point estimation approach. *IEEE Transactions on Software Engineering* 49, 2 (2022), 611–625.

[99] Apurva Gandhi, Thong Q. Nguyen, Huitian Jiao, Robert Steen, and Ameya Bhatawdekar. 2023. Natural language commanding via program synthesis. arXiv:2306.03460. Retrieved from https://arxiv.org/abs/2306.03460

[100] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The pile: An 800gb dataset of diverse text for language modeling. arXiv:2101.00027. Retrieved from https://arxiv.org/abs/2101.00027

[101] Shuzheng Gao, Wenxin Mao, Cuiyun Gao, Li Li, Xing Hu, Xin Xia, and Michael R. Lyu. 2024. Learning in the wild: Towards leveraging unlabeled data for effectively tuning pre-trained code models. arXiv:2401.01060. Retrieved from https://arxiv.org/abs/2401.01060

[102] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, and Michael R. Lyu. 2023. Constructing effective in-context demonstration for code intelligence tasks: An empirical study. arXiv:2304.07575. Retrieved from https://arxiv.org/abs/2304.07575

[103] Zeyu Gao, Hao Wang, Yuchen Zhou, Wenyu Zhu, and Chao Zhang. 2023. How far have we gone in vulnerability detection using large language models. arXiv:2311.12420. Retrieved from https://arxiv.org/abs/2311.12420

[104] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024)*. ACM, New York, NY, 13 pages. DOI: https://doi.org/10.1145/3597503.3608134

[105] Malcom Gethers, Rocco Oliveto, Denys Poshyvanyk, and Andrea De Lucia. 2011. On integrating orthogonal information retrieval methods to improve traceability recovery. In *Proceedings of the 2011 27th IEEE International Conference on Software Maintenance (ICSM '11)*. IEEE, 133–142.

[106] Lobna Ghadhab, Ilyes Jenhani, Mohamed Wiem Mkaouer, and Montassar Ben Messaoud. 2021. Augmenting commit classification by using fine-grained source code changes and a pre-trained deep neural language model. *Information and Software Technology* 135 (2021), 106566.

[107] Henry Gilbert, Michael Sandborn, Douglas C Schmidt, Jesse Spencer-Smith, and Jules White. 2023. Semantic compression with large language models. arXiv:2304.12512. Retrieved from https://arxiv.org/abs/2304.12512

[108] Github. 2023. Github. Retrieved from https://github.com/

[109] GitHub. 2023. Github copilot. Retrieved from https://copilot.github.com

[110] Luiz Gomes, Ricardo da Silva Torres, and Mario Lúcio Côrtes. 2023. Bert-and TF-IDF-based feature extraction for long-lived bug prediction in floss: A comparative study. *Information and Software Technology* 160 (2023), 107217.

[111] Lina Gong, Jingxuan Zhang, Mingqiang Wei, Haoxiang Zhang, and Zhiqiu Huang. 2023. What is the intended usage context of this model? an exploratory study of pre-trained models on various model repositories. *ACM Transactions on Software Engineering and Methodology* 32, 3 (2023), 1–57.

[112] Google. 2023. Gemini. Retrieved from https://gemini.google.com/
[113] Google. 2024. Gemma. Retrieved from https://blog.google/technology/developers/gemma-open-models/
[114] Anastasiia Grishina, Max Hort, and Leon Moonen. 2023. The earlybird catches the bug: On exploiting early layers of encoder models for more efficient code classification. arXiv:2305.04940. Retrieved from https://arxiv.org/abs/2305.04940
[115] Jian Gu, Pasquale Salza, and Harald C. Gall. 2022. Assemble foundation models for automatic code summarization. In *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '22)*. IEEE, 935–946.
[116] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*, 933–944.
[117] Xiaodong Gu, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. 2016. Deep API learning. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 631–642.
[118] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2020. Graphcodebert: Pre-training code representations with data flow. arXiv:2009.08366. Retrieved from https://arxiv.org/abs/2009.08366
[119] Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. 2023. Longcoder: A long-range pre-trained language model for code completion. arXiv:2306.14893. Retrieved from https://arxiv.org/abs/2306.14893
[120] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. arXiv:2401.14196. Retrieved from https://arxiv.org/abs/2401.14196
[121] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
[122] Priyanshu Gupta, Avishree Khare, Yasharth Bajpai, Saikat Chakraborty, Sumit Gulwani, Aditya Kanade, Arjun Radhakrishna, Gustavo Soares, and Ashish Tiwari. 2023. Grace: Generation using associated code edits. arXiv:2305.14129. Retrieved from https://arxiv.org/abs/2305.14129
[123] Emitza Guzman, David Azócar, and Yang Li. 2014. Sentiment analysis of commit comments in github: An empirical study. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, 352–355.
[124] Patrick Hajali and Ignas Budvytis. 2023. Function-constrained program synthesis. arXiv:2311.15500. Retrieved from https://arxiv.org/abs/2311.15500
[125] Yu Hao, Weiteng Chen, Ziqiao Zhou, and Weidong Cui. 2023. E & v: Prompting large language models to perform static analysis by pseudo-code execution and verification. arXiv:2312.08477. Retrieved from https://arxiv.org/abs/2312.08477
[126] Andreas Happe and Jürgen Cito. 2023. Getting pwn'd by ai: Penetration testing with large language models. arXiv:2308.00121. Retrieved from https://arxiv.org/abs/2308.00121
[127] Julian Harty, Haonan Zhang, Lili Wei, Luca Pascarella, Mauricio Aniche, and Weiyi Shang. 2021. Logging practices with mobile analytics: An empirical study on firebase. In *Proceedings of the 2021 IEEE/ACM 8th International Conference on Mobile Software Engineering and Systems (MOBILESOFT '21)*. IEEE, 56–60.
[128] Wilhelm Hasselbring and André van Hoorn. 2020. Kieker: A monitoring framework for software engineering research. *Software Impacts* 5 (2020), 100019.
[129] Junda He, Zhou Xin, Bowen Xu, Ting Zhang, Kisub Kim, Zhou Yang, Ferdian Thung, Ivana Irsan, and David Lo. 2023. Representation learning for stack overflow posts: How far are we? arXiv:2303.06853. Retrieved from https://arxiv.org/abs/2303.06853
[130] Junda He, Bowen Xu, Zhou Yang, DongGyun Han, Chengran Yang, and David Lo. 2022. Ptm4tag: Sharpening tag recommendation of stack overflow posts with pre-trained models. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, 1–11.
[131] Vincent J. Hellendoorn, Christian Bird, Earl T. Barr, and Miltiadis Allamanis. 2018. Deep learning type inference. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 152–162.
[132] Robert Kraig Helmeczi, Mucahit Cevik, and Savas Yildirim. 2023. Few-shot learning for sentence pair classification and its applications in software engineering. arXiv:2306.08058. Retrieved from https://arxiv.org/abs/2306.08058
[133] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with apps. arXiv:2105.09938. Retrieved from https://arxiv.org/abs/2105.09938
[134] Jordan Henkel, Denini Silva, Leopoldo Teixeira, Marcelo d'Amorim, and Thomas Reps. 2021. Shipwright: A human-in-the-loop system for dockerfile repair. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*. IEEE, 1148–1160.

[135] Tobias Hey, Jan Keim, Anne Koziolek, and Walter F. Tichy. 2020. Norbert: Transfer learning for requirements classification. In *Proceedings of the 2020 IEEE 28th International Requirements Engineering Conference (RE '20)*. IEEE, 169–179.

[136] hiyouga. 2023. Llama efficient tuning. Retrieved from https://github.com/hiyouga/LLaMA-Efficient-Tuning

[137] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training compute-optimal large language models. arXiv:2203.15556. Retrieved from https://arxiv.org/abs/2203.15556

[138] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. METAGPT: Meta programming for multi-agent collaborative framework. arXiv:2308.00352. Retrieved from https://arxiv.org/abs/2308.00352

[139] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2790–2799.

[140] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. arXiv:2106.09685. Retrieved from https://arxiv.org/abs/2106.09685

[141] Jie Hu, Qian Zhang, and Heng Yin. 2023. Augmenting greybox fuzzing with generative AI. arXiv:2306.06782. Retrieved from https://arxiv.org/abs/2306.06782

[142] Xueyu Hu, Kun Kuang, Jiankai Sun, Hongxia Yang, and Fei Wu. 2024. Leveraging print debugging to improve code generation in large language models. arXiv:2401.05319. Retrieved from https://arxiv.org/abs/2401.05319

[143] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2018. Deep code comment generation. In *Proceedings of the 26th Conference on Program Comprehension*, 200–210.

[144] Dong Huang, Qingwen Bu, and Heming Cui. 2023. Codecot and beyond: Learning to program and test like a developer. arXiv:2308.08784. Retrieved from https://arxiv.org/abs/2308.08784

[145] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. 2023. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. arXiv:2312.13010. Retrieved from https://arxiv.org/abs/2312.13010

[146] Di Huang, Ziyuan Nan, Xing Hu, Pengwei Jin, Shaohui Peng, Yuanbo Wen, Rui Zhang, Zidong Du, Qi Guo, Yewen Pu, and Yunji Chen. 2023. anpl: Compiling natural programs with interactive decomposition. arXiv:2305.18498. Retrieved from https://arxiv.org/abs/2305.18498

[147] Qing Huang, Yanbang Sun, Zhenchang Xing, Min Yu, Xiwei Xu, and Qinghua Lu. 2023. Api entity and relation joint extraction from text via dynamic prompt-tuned language model. arXiv:2301.03987. Retrieved from https://arxiv.org/abs/2301.03987

[148] Qing Huang, Yishun Wu, Zhenchang Xing, He Jiang, Yu Cheng, and Huan Jin. 2023. Adaptive intellect unleashed: The feasibility of knowledge transfer in large language models. arXiv:2308.04788. Retrieved from https://arxiv.org/abs/2308.04788

[149] Qiao Huang, Xin Xia, Zhenchang Xing, David Lo, and Xinyu Wang. 2018. Api method recommendation without worrying about the task-api knowledge gap. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 293–304.

[150] Qing Huang, Jiahui Zhu, Zhenchang Xing, Huan Jin, Changjing Wang, and Xiwei Xu. 2023. A chain of ai-based solutions for resolving fqns and fixing syntax errors in partial code. arXiv:2306.11981. Retrieved from https://arxiv.org/abs/2306.11981

[151] Qing Huang, Zhou Zou, Zhenchang Xing, Zhenkang Zuo, Xiwei Xu, and Qinghua Lu. 2023. Ai chain on large language model for unsupervised control flow graph generation for statically-typed partial code. arXiv:2306.00757. Retrieved from https://arxiv.org/abs/2306.00757

[152] Yuchao Huang, Junjie Wang, Zhe Liu, Yawen Wang, Song Wang, Chunyang Chen, Yuanzhe Hu, and Qing Wang. 2024. Crashtranslator: Automatically reproducing mobile application crashes directly from stack trace. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.

[153] Ali Reza Ibrahimzada, Yang Chen, Ryan Rong, and Reyhaneh Jabbarvand. 2023. Automated bug generation in the era of large language models. arXiv:2310.02407. Retrieved from https://arxiv.org/abs/2310.02407

[154] Ali Reza Ibrahimzada, Yigit Varli, Dilara Tekinoglu, and Reyhaneh Jabbarvand. 2022. Perfect is the enemy of test oracle. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 70–81.

[155] Md Rakibul Islam and Minhaz F. Zibran. 2017. Leveraging automated sentiment analysis in software engineering. In *Proceedings of the 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR '17)*. IEEE, 203–214.

[156] Nafis Tanveer Islam, Joseph Khoury, Andrew Seong, Gonzalo De La Torre Parra, Elias Bou-Harb, and Peyman Najafirad. 2024. LLM-powered code vulnerability repair with reinforcement learning and semantic reward. arXiv:2401.03374. Retrieved from https://arxiv.org/abs/2401.03374

[157] Nafis Tanveer Islam and Peyman Najafirad. 2024. Code security vulnerability repair using reinforcement learning with large language models. arXiv:2401.07031. Retrieved from https://arxiv.org/abs/2401.07031

[158] Haruna Isotani, Hironori Washizaki, Yoshiaki Fukazawa, Tsutomu Nomoto, Saori Ouji, and Shinobu Saito. 2021. Duplicate bug report detection by using sentence embedding and fine-tuning. In *Proceedings of the 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME '21)*. IEEE, 535–544.

[159] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2016. Summarizing source code using a neural attention model. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics 2016*. Association for Computational Linguistics, 2073–2083.

[160] Maliheh Izadi, Roberta Gismondi, and Georgios Gousios. 2022. Codefill: Multi-token code completion by jointly learning from structure and naming sequences. In *Proceedings of the 44th International Conference on Software Engineering*, 401–412.

[161] Abhinav Jain, Chima Adiole, Thomas Reps, Swarat Chaudhuri, and Chris Jermaine. 2024. Coarse-tuning models of code with reinforcement learning feedback. Retrieved from https://openreview.net/forum?id=vLqkCvjHRD

[162] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering*, 1219–1231.

[163] Naman Jain, Tianjun Zhang, Wei-Lin Chiang, Joseph E Gonzalez, Koushik Sen, and Ion Stoica. 2023. LLM-assisted code cleaning for training accurate code generators. arXiv:2311.14904. Retrieved from https://arxiv.org/abs/2311.14904

[164] Prithwish Jana, Piyush Jha, Haoyang Ju, Gautham Kishore, Aryan Mahajan, and Vijay Ganesh. 2023. Attention, compilation, and solver-based symbolic analysis are all you need. arXiv:2306.06755. Retrieved from https://arxiv.org/abs/2306.06755

[165] Kevin Jesse, Premkumar T. Devanbu, and Anand Sawant. 2022. Learning to predict user-defined types. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1508–1522.

[166] Zhenlan Ji, Pingchuan Ma, Zongjie Li, and Shuai Wang. 2023. Benchmarking and explaining large language model-based code generation: A causality-centric approach. arXiv:2310.06680. Retrieved from https://arxiv.org/abs/2310.06680

[167] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. arXiv:2302.05020. Retrieved from https://arxiv.org/abs/2302.05020

[168] Nan Jiang, Chengxiao Wang, Kevin Liu, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. 2023. Nova$^+$: Generative language models for binaries. arXiv:2311.13721. Retrieved from https://arxiv.org/abs/2311.13721

[169] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. Selfevolve: A code evolution framework via large language models. arXiv:2306.02907. Retrieved from https://arxiv.org/abs/2306.02907

[170] Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 2023. Self-planning code generation with large language model. arXiv:2303.06689. Retrieved from https://arxiv.org/abs/2303.06689

[171] Yanjie Jiang, Hui Liu, Jiahao Jin, and Lu Zhang. 2020. Automated expansion of abbreviations based on semantic relation and transfer expansion. *IEEE Transactions on Software Engineering* 48, 2 (2020), 519–537.

[172] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world GitHub issues? arXiv:2310.06770. Retrieved from https://arxiv.org/abs/2310.06770

[173] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Inferfix: End-to-end program repair with LLMS. arXiv:2303.07263. Retrieved from https://arxiv.org/abs/2303.07263

[174] Pengxiang Jin, Shenglin Zhang, Minghua Ma, Haozhe Li, Yu Kang, Liqun Li, Yudong Liu, Bo Qiao, Chaoyun Zhang, Pu Zhao, Shilin He, Federica Sarro, Yingnong Dang, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. Assess and summarize: Improve outage understanding with large language models. arXiv:2305.18084. Retrieved from https://arxiv.org/abs/2305.18084

[175] Xin Jin, Jonathan Larson, Weiwei Yang, and Zhiqiang Lin. 2023. Binary code summarization: Benchmarking CHATGPT/GPT-4 and other large language models. arXiv:2312.09601. Retrieved from https://arxiv.org/abs/2312.09601

[176] Erik Jones and Jacob Steinhardt. 2022. Capturing failures of large language models via human cognitive biases. In *Advances in Neural Information Processing Systems*, Vol. 35, 11785–11799.

[177] Robbert Jongeling, Subhajit Datta, and Alexander Serebrenik. 2015. Choosing your weapons: On sentiment analysis tools for software engineering research. In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME '15)*. IEEE, 531–535.

[178] Judini. 2023. The future of software development powered by AI. Retrieved from https://codegpt.co/

[179] Azmain Kabir, Shaowei Wang, Yuan Tian, Tse-Hsun (Peter)Chen, Muhammad Asaduzzaman, Wenbin Zhang. 2024. ZS4C: Zero-shot synthesis of compilable code for incomplete code snippets using ChatGPT. arXiv:2401.14279. Retrieved from https://arxiv.org/abs/2401.14279

[180] Md Mahir Asef Kabir, Sk Adnan Hassan, Xiaoyin Wang, Ying Wang, Hai Yu, and Na Meng. 2023. An empirical study of ChatGPT-3.5 on question answering and code maintenance. arXiv:2310.02104. Retrieved from https://arxiv.org/abs/2310.02104

[181] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and evaluating contextual embedding of source code. In *Proceedings of the International Conference on Machine Learning*. PMLR, 5110–5121.

[182] Sungmin Kang, Gabin An, and Shin Yoo. 2023. A preliminary evaluation of LLM-based fault localization. arXiv:2308.05487. Retrieved from https://arxiv.org/abs/2308.05487

[183] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2023. Explainable automated debugging via large language model-driven scientific debugging. arXiv:2304.02195. Retrieved from https://arxiv.org/abs/2304.02195

[184] Sungmin Kang, Juyeon Yoon, Nargiz Askarbekkyzy, and Shin Yoo. 2023. Evaluating diverse large language models for automatic and general bug reproduction. arXiv:2311.04532. Retrieved from https://arxiv.org/abs/2311.04532

[185] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2022. Large language models are few-shot testers: Exploring LLM-based general bug reproduction. arXiv:2209.11515. Retrieved from https://arxiv.org/abs/2209.11515

[186] Jai Kannan. 2023. Can llms configure software tools. arXiv:2312.06121. Retrieved from https://arxiv.org/abs/2312.06121

[187] Rafael-Michael Karampatsis and Charles Sutton. 2020. Scelmo: Source code embeddings from language models. arXiv:2004.13214. Retrieved from https://arxiv.org/abs/2004.13214

[188] Li Ke, Hong Sheng, Fu Cai, Zhang Yunhe, and Liu Ming. 2023. Discriminating human-authored from chatgpt-generated code via discernable feature analysis. arXiv:2306.14397. Retrieved from https://arxiv.org/abs/2306.14397

[189] Adam Khakhar, Stephen Mell, and Osbert Bastani. 2023. PAC prediction sets for large language models of code. arXiv:2302.08703. Retrieved from https://arxiv.org/abs/2302.08703

[190] Junaed Younus Khan, Md Tawkat Islam Khondaker, Gias Uddin, and Anindya Iqbal. 2021. Automatic detection of five API documentation smells: Practitioners' perspectives. In *Proceedings of the 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '21)*. IEEE, 318–329.

[191] Junaed Younus Khan and Gias Uddin. 2022. Automatic detection and analysis of technical debts in peer-review documentation of r packages. In *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '22)*. IEEE, 765–776.

[192] Mohammad Abdullah Matin Khan, M. Saiful Bari, Xuan Long Do, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. 2023. Xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. arXiv:2303.03004. Retrieved from https://arxiv.org/abs/2303.03004

[193] Muhammad Fawad Akbar Khan, Max Ramsdell, Erik Falor, and Hamid Karimi. 2023. Assessing the promise and pitfalls of chatgpt for automated code generation. arXiv:2311.02640. Retrieved from https://arxiv.org/abs/2311.02640

[194] Ahmed Khanfir, Renzo Degiovanni, Mike Papadakis, and Yves Le Traon. 2023. Efficient mutation testing via pre-trained language models. arXiv:2301.03543. Retrieved from https://arxiv.org/abs/2301.03543

[195] Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, and Mayur Naik. 2023. Understanding the effectiveness of large language models in detecting security vulnerabilities. arXiv:2311.16169. Retrieved from https://arxiv.org/abs/2311.16169

[196] Hiroyuki Kirinuki and Haruto Tanno. 2024. Chatgpt and human synergy in black-box testing: A comparative analysis. arXiv:2401.13924. Retrieved from https://arxiv.org/abs/2401.13924

[197] Barbara Ann Kitchenham and Stuart Charters. 2007. Guidelines for performing systematic literature reviews in software engineering. In *Technical Report EBSE 2007-001*. Keele University and Durham University Joint Report. Retrieved from https://www.elsevier.com/__data/promis_misc/525444systematicreviewsguide.pdf

[198] Barbara Kitchenham, Lech Madeyski, and David Budgen. 2022. Segress: Software engineering guidelines for reporting secondary studies. *IEEE Transactions on Software Engineering* 49, 3 (2022), 1273–1298.

[199] Eric Knauss, Siv Houmb, Kurt Schneider, Shareeful Islam, and Jan Jürjens. 2011. Supporting requirements engineers in recognising security issues. In *Requirements Engineering: Foundation for Software Quality: 17th International Working Conference (REFSQ '11)*. Springer, 4–18.

[200] Amy J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006), 971–987.

[201] Takashi Koide, Naoki Fukushi, Hiroki Nakano, and Daiki Chiba. 2023. Detecting phishing sites using chatgpt. arXiv:2306.05816. Retrieved from https://arxiv.org/abs/2306.05816

[202] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, Vol. 35, 22199–22213.

[203] Kristian Kolthoff, Christian Bartelt, and Simone Paolo Ponzetto. 2023. Data-driven prototyping via natural-language-based GUI retrieval. *Automated Software Engineering* 30, 1 (2023), 13.
[204] Bonan Kou, Muhao Chen, and Tianyi Zhang. 2023. Automated summarization of stack overflow posts. arXiv:2305.16680. Retrieved from https://arxiv.org/abs/2305.16680
[205] Bonan Kou, Shengmai Chen, Zhijie Wang, Lei Ma, and Tianyi Zhang. 2023. Is model attention aligned with human attention? An empirical study on large language models for code generation. arXiv:2306.01220. Retrieved from https://arxiv.org/abs/2306.01220
[206] Amit Kulkarni. 2021. GitHub copilot ai is leaking functional API keys. Retrieved from https://analyticsdrift.com/github-copilot-ai-is-leaking-functional-api-keys/
[207] Kirby Kuznia, Swaroop Mishra, Mihir Parmar, and Chitta Baral. 2022. Less is more: Summary of long instructions is better for program synthesis. arXiv:2203.08597. Retrieved from https://arxiv.org/abs/2203.08597
[208] Shuvendu K Lahiri, Aaditya Naik, Georgios Sakkas, Piali Choudhury, Curtis von Veh, Madanlal Musuvathi, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. 2022. Interactive code generation via test-driven user-intent formalization. arXiv:2208.05950. Retrieved from https://arxiv.org/abs/2208.05950
[209] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. Ds-1000: A natural and reliable benchmark for data science code generation. In *Proceedings of the International Conference on Machine Learning*. PMLR, 18319–18345.
[210] Márk Lajkó, Viktor Csuvik, and László Vidács. 2022. Towards javascript program repair with generative pre-trained transformer (GPT-2). In *Proceedings of the 3rd International Workshop on Automated Program Repair*, 61–68.
[211] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. Albert: A lite bert for self-supervised learning of language representations. arXiv:1909.11942. Retrieved from https://arxiv.org/abs/1909.11942
[212] Md Tahmid Rahman Laskar, M. Saiful Bari, Mizanur Rahman, Md Amran Hossen Bhuiyan, Shafiq Joty, and Jimmy Xiangji Huang. 2023. A systematic study and comprehensive evaluation of ChatGPT on benchmark datasets. arXiv:2305.18486. Retrieved from https://arxiv.org/abs/2305.18486
[213] Hung Le, Hailin Chen, Amrita Saha, Akash Gokul, Doyen Sahoo, and Shafiq Joty. 2023. Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules. arXiv:2310.08992. Retrieved from https://arxiv.org/abs/2310.08992
[214] Thanh Le-Cong, Hong Jin Kang, Truong Giang Nguyen, Stefanus Agus Haryono, David Lo, Xuan-Bach D. Le, and Quyet Thang Huynh. 2022. Autopruner: Transformer-based call graph pruning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 520–532.
[215] Thanh Le-Cong, Duc-Minh Luong, Xuan Bach D. Le, David Lo, Nhat-Hoa Tran, Bui Quang-Huy, and Quyet-Thang Huynh. 2023. Invalidator: Automated patch correctness assessment via semantic and syntactic reasoning. *IEEE Transactions on Software Engineering* 49, 6 (June 2023), 3411–3429. DOI: https://doi.org/10.1109/TSE.2023.3255177
[216] Jaehyung Lee, Kisun Han, and Hwanjo Yu. 2022. A light bug triage framework for applying large pre-trained language model. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–11.
[217] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. arXiv:2104.08691. Retrieved from https://arxiv.org/abs/2104.08691
[218] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2023. Chain of code: Reasoning with a language model-augmented code emulator. arXiv:2312.04474. Retrieved from https://arxiv.org/abs/2312.04474
[219] Dong Li, Yelong Shen, Ruoming Jin, Yi Mao, Kuan Wang, and Weizhu Chen. 2022. Generation-augmented query expansion for code retrieval. arXiv:2212.10692. Retrieved from https://arxiv.org/abs/2212.10692
[220] Feng-Lin Li, Jennifer Horkoff, John Mylopoulos, Renata S. S. Guizzardi, Giancarlo Guizzardi, Alexander Borgida, and Lin Liu. 2014. Non-functional requirements as qualities, with a spice of ontology. In *Proceedings of the 2014 IEEE 22nd International Requirements Engineering Conference (RE '14)*. IEEE, 293–302.
[221] Haochen Li, Xin Zhou, and Zhiqi Shen. 2024. Rewriting the code: A simple method for large language model augmented code search. arXiv:2401.04514. Retrieved from https://arxiv.org/abs/2401.04514
[222] Jingyao Li, Pengguang Chen, and Jiaya Jia. 2023. Motcoder: Elevating large language models with modular of thought for challenging programming tasks. arXiv:2312.15960. Retrieved from https://arxiv.org/abs/2312.15960
[223] Jingxuan Li, Rui Huang, Wei Li, Kai Yao, and Weiguo Tan. 2021. Toward less hidden cost of code completion with acceptance and ranking models. In *Proceedings of the 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME '21)*. IEEE, 195–205.
[224] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Enabling programming thinking in large language models toward code generation. arXiv:2305.06599. Retrieved from https://arxiv.org/abs/2305.06599
[225] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Structured chain-of-thought prompting for code generation. arXiv:2305.06599. Retrieved from https://arxiv.org/abs/2305.06599

[226] Jia Li, Ge Li, Zhuo Li, Zhi Jin, Xing Hu, Kechi Zhang, and Zhiyi Fu. 2023. Codeeditor: Learning to edit source code with pre-trained models. *ACM Transactions on Software Engineering and Methodology* 32, 6 (2023), 1–22.

[227] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Zhi Jin, Hao Zhu, Huanyu Liu, Kaibo Liu, Lecheng Wang, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yihong Dong, Yuqi Zhu, Bin Gu, and Mengfei Yang. 2024. Deveval: Evaluating code generation in practical software projects. arXiv:2401.06401. Retrieved from https://arxiv.org/abs/2401.06401

[228] Jia Li, Zhuo Li, Huangzhao Zhang, Ge Li, Zhi Jin, Xing Hu, and Xin Xia. 2022. Poison attack and defense on deep source code processing models. DOI: https://doi.org/10.48550/ARXIV.2210.17029

[229] Li Li, Tegawendé F. Bissyandé, Mike Papadakis, Siegfried Rasthofer, Alexandre Bartel, Damien Octeau, Jacques Klein, and Le Traon. 2017. Static analysis of android apps: A systematic literature review. *Information and Software Technology* 88 (2017), 67–95.

[230] Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, and Chun Zuo. 2022. Auger: Automatically generating review comments with pre-training models. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1009–1021.

[231] Peng Li, Tianxiang Sun, Qiong Tang, Hang Yan, Yuanbin Wu, Xuanjing Huang, and Xipeng Qiu. 2023. Codeie: Large code generation models are better few-shot information extractors. arXiv:2305.05711. Retrieved from https://arxiv.org/abs/2305.05711

[232] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, and Shing-Chi Cheung. 2023. Finding failure-inducing test cases with ChatGPT. arXiv:2304.11686. Retrieved from https://arxiv.org/abs/2304.11686

[233] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 14–26.

[234] Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. 2022. Coderetriever: A large scale contrastive pre-training method for code search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2898–2910.

[235] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. arXiv:2101.00190. Retrieved from https://arxiv.org/abs/2101.00190

[236] Xin-Ye Li, Jiang-Tian Xue, Zheng Xie, and Ming Li. 2023. Think outside the code: Brainstorming boosts large language models in code generation. arXiv:2305.10679. Retrieved from https://arxiv.org/abs/2305.10679

[237] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

[238] Yichen Li, Yintong Huo, Zhihan Jiang, Renyi Zhong, Pinjia He, Yuxin Su, and Michael R. Lyu. 2023. Exploring the effectiveness of LLMS in automated logging generation: An empirical study. arXiv:2307.05950. Retrieved from https://arxiv.org/abs/2307.05950

[239] Yue Li, Zhong Ren, Zhiqi Wang, Lanxin Yang, Liming Dong, Chenxing Zhong, and He Zhang. 2024. Fine-SE: Integrating semantic features and expert features for software effort estimation. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–12.

[240] Youjia Li, Jianjun Shi, and Zheng Zhang. 2023. A novel approach for rapid development based on chatgpt and prompt engineering. arXiv:2312.13115. Retrieved from https://arxiv.org/abs/2312.13115

[241] Yao Li, Tao Zhang, Xiapu Luo, Haipeng Cai, Sen Fang, and Dawei Yuan. 2022. Do pre-trained language models indeed understand software engineering tasks? arXiv:2211.10623. Retrieved from https://arxiv.org/abs/2211.10623

[242] Zhihao Li, Chuanyi Li, Ze Tang, Wanhong Huang, Jidong Ge, Bin Luo, Vincent Ng, Ting Wang, Yucheng Hu, and Xiaopeng Zhang. 2024. ptm-apirec: Leveraging pre-trained models of source code in API recommendation. *ACM Transactions on Software Engineering and Methodology* 3, 3 (March 2024), Article 72, 30 pages. DOI: https://doi.org/10.1145/3632745

[243] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Dong Chen, Shuai Wang, and Cuiyun Gao. 2023. Cctest: Testing and repairing code completion systems. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1238–1250.

[244] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Shuai Wang, and Cuiyun Gao. 2022. Cctest: Testing and repairing code completion systems. arXiv:2208.08289. Retrieved from https://arxiv.org/abs/2208.08289

[245] Yuding Liang and Kenny Zhu. 2018. Automatic generation of text descriptive comments for code blocks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32, No. 1. DOI: https://doi.org/10.1609/aaai.v32i1.11963

[246] Jinfeng Lin, Yalin Liu, Qingkai Zeng, Meng Jiang, and Jane Cleland-Huang. 2021. Traceability transformed: Generating more accurate links with pre-trained bert models. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*. IEEE, 324–335.

[247] Yu-Chen Lin, Akhilesh Kumar, Wen-Liang Zhang, Norman Chang, Muhammad Zakir, Rucha Apte, Chao Wang, and Jyh-Shing Roger Jang. 2023. Applications of large language models in data processing: Innovative approaches to segmenting and renewing information. arXiv:2311.16267. Retrieved from https://arxiv.org/abs/2311.16267

[248] Chao Liu, Xuanlin Bao, Hongyu Zhang, Neng Zhang, Haibo Hu, Xiaohong Zhang, and Meng Yan. 2023. Improving ChatGPT prompt for code generation. arXiv:2305.08360. Retrieved from https://arxiv.org/abs/2305.08360

[249] Fang Liu, Ge Li, Yunfei Zhao, and Zhi Jin. 2020. Multi-task learning based pre-trained language model for code completion. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 473–485.

[250] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin A. Raffel. 2022. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems,* Vol. 35, 1950–1965.

[251] Hao Liu, Yanlin Wang, Zhao Wei, Yong Xu, Juhong Wang, Hui Li, and Rongrong Ji. 2023. RefBERT: A two-stage pre-trained framework for automatic rename refactoring. arXiv:2305.17708. Retrieved from https://arxiv.org/abs/2305.17708

[252] Jinrun Liu, Xinyu Tang, Linlin Li, Panpan Chen, and Yepang Liu. 2023. Which is a better programming assistant? A comparative study between chatgpt and stack overflow. arXiv:2308.13851. Retrieved from https://arxiv.org/abs/2308.13851

[253] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? Rigorous evaluation of large language models for code generation. arXiv:2305.01210. Retrieved from https://arxiv.org/abs/2305.01210

[254] Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhi Li, and Limin Sun. 2023. Harnessing the power of llm to support binary taint analysis. arXiv:2310.08275. Retrieved from https://arxiv.org/abs/2310.08275

[255] Shangqing Liu, Bozhi Wu, Xiaofei Xie, Guozhu Meng, and Yang Liu. 2023. Contrabert: Enhancing code pre-trained models via contrastive learning. arXiv:2301.09072. Retrieved from https://arxiv.org/abs/2301.09072

[256] Tianyang Liu, Canwen Xu, and Julian McAuley. 2023. Repobench: Benchmarking repository-level code auto-completion systems. arXiv:2306.03091. Retrieved from https://arxiv.org/abs/2306.03091

[257] Xiaoyu Liu, LiGuo Huang, and Vincent Ng. 2018. Effective API recommendation without historical software repositories. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 282–292.

[258] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2023. Prompt injection attack against LLM-integrated applications. arXiv:2306.05499. Retrieved from https://arxiv.org/abs/2306.05499

[259] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2023. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. arXiv:2307.12596. Retrieved from https://arxiv.org/abs/2307.12596

[260] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. arXiv:1907.11692. Retrieved from https://arxiv.org/abs/1907.11692

[261] Yue Liu, Chakkrit Tantithamthavorn, Li Li, and Yepang Liu. 2022. Deep learning for android malware defenses: A systematic literature review. *ACM Computing Surveys* 55, 8 (2022), 1–36.

[262] Yue Liu, Chakkrit Tantithamthavorn, Yonghui Liu, and Li Li. 2024. On the reliability and explainability of language models for program generation. *ACM Transactions on Software Engineering and Methodology* 33, 5 (June 2024), Article 126, 26 pages. DOI: https://doi.org/10.1145/3641540

[263] Yilun Liu, Shimin Tao, Weibin Meng, Jingyu Wang, Wenbing Ma, Yanqing Zhao, Yuhang Chen, Hao Yang, Yanfei Jiang, and Xun Chen. 2024. Interpretable online log analysis using large language models with prompt strategies. arXiv:2308.07610. Retrieved from https://arxiv.org/abs/2308.07610

[264] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Fill in the blank: Context-aware automated text input generation for mobile GUI testing. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*. IEEE, 1355–1367.

[265] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model. arXiv:2310.15657. Retrieved from https://arxiv.org/abs/2310.15657

[266] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. 2023. No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. arXiv:2308.04838. Retrieved from https://arxiv.org/abs/2308.04838

[267] David Lo. 2023. Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps. In *Proceedings of the IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE '23)*. IEEE, 69–85. DOI: https://doi.org/10.1109/ICSE-FOSE59343.2023.00010

[268] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE '23)*. IEEE, 647–658.

[269] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. arXiv:2102.04664. Retrieved from https://arxiv.org/abs/2102.04664

[270] James H. Lubowitz. 2023. ChatGPT, an artificial intelligence ChatBot, is impacting medical literature. *Arthroscopy* 39, 5 (2023), 1121–1122.

[271] Dipeeka Luitel, Shabnam Hassani, and Mehrdad Sabetzadeh. 2023. Improving requirements completeness: Automated assistance through large language models. arXiv:2308.03784. Retrieved from https://arxiv.org/abs/2308.03784

[272] Xianchang Luo, Yinxing Xue, Zhenchang Xing, and Jiamou Sun. 2022. Prcbert: Prompt learning for requirement classification using bert-based pretrained language models. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–13.

[273] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. arXiv:2306.08568. Retrieved from https://arxiv.org/abs/2306.08568

[274] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2024. SpecGen: Automated generation of formal program specifications via large language models. arXiv:2401.08807. Retrieved from https://arxiv.org/abs/2401.08807

[275] Lipeng Ma, Weidong Yang, Bo Xu, Sihang Jiang, Ben Fei, Jiaqing Liang, Mingjie Zhou, and Yanghua Xiao. 2024. Knowlog: Knowledge enhanced pre-trained language model for log understanding. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.

[276] Wei Ma, Shangqing Liu, Wenhan Wang, Qiang Hu, Ye Liu, Cen Zhang, Liming Nie, and Yang Liu. 2023. The scope of chatgpt in software engineering: A thorough investigation. arXiv:2305.12138. Retrieved from https://arxiv.org/abs/2305.12138

[277] Wei Ma, Shangqing Liu, Mengjie Zhao, Xiaofei Xie, Wenhan Wang, Qiang Hu, Jie Zhang, and Yang Liu. 2024. Unveiling code pre-trained models: Investigating syntax and semantics capacities. *ACM Transactions on Software Engineering and Methodology* 33, 7 (August 2024), Article 169, 29 pages. DOI: https://doi.org/10.1145/3664606

[278] Aman Madaan, Shuyan Zhou, Uri Alon, Yiming Yang, and Graham Neubig. 2022. Language models of code are few-shot commonsense learners. arXiv:2210.07128. Retrieved from https://arxiv.org/abs/2210.07128

[279] Shantanu Mandal, Adhrik Chethan, Vahid Janfaza, S. M. Mahmud, Todd A Anderson, Javier Turek, Jesmin Jahan Tithi, and Abdullah Muzahid. 2023. Large language models based automatic synthesis of software specifications. arXiv:2304.09181. Retrieved from https://arxiv.org/abs/2304.09181

[280] Dung Nguyen Manh, Nam Le Hai, Anh T. V. Dau, Anh Minh Nguyen, Khanh Nghiem, Jin Guo, and Nghi D. Q. Bui. 2023. The vault: A comprehensive multilingual dataset for advancing code understanding and generation. arXiv:2305.06156. Retrieved from https://arxiv.org/abs/2305.06156

[281] Zohar Manna and Richard Waldinger. 1980. A deductive approach to program synthesis. *ACM Transactions on Programming Languages and Systems* 2, 1 (1980), 90–121.

[282] Yuetian Mao, Chengcheng Wan, Yuze Jiang, and Xiaodong Gu. 2023. Self-supervised query reformulation for code search. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 363–374.

[283] Antonio Mastropaolo, Emad Aghajani, Luca Pascarella, and Gabriele Bavota. 2021. An empirical study on code comment completion. In *Proceedings of the 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME '21)*. IEEE, 159–170.

[284] Antonio Mastropaolo, Nathan Cooper, David Nader Palacio, Simone Scalabrino, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2022. Using transfer learning for code-related tasks. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1580–1598.

[285] Antonio Mastropaolo, Massimiliano Di Penta, and Gabriele Bavota. 2023. Towards automatically addressing self-admitted technical debt: How far are we?. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 585–597.

[286] Antonio Mastropaolo, Luca Pascarella, and Gabriele Bavota. 2022. Using deep learning to generate complete log statements. In *Proceedings of the 44th International Conference* on Software Engineering, 2279–2290.

[287] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. 2023. On the robustness of code generation techniques: An empirical study on GitHub copilot. arXiv:2302.00438. Retrieved from https://arxiv.org/abs/2302.00438

[288] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the usage of text-to-text transfer transformer to support code-related tasks. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE '21)*. IEEE, 336–347.

[289] Meta. 2023. Code llama: Open foundation models for code. Retrieved from https://ai.meta.com/research/publications/code-llama-open-foundation-models-for-code/

[290] Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Boaye Belle, Hung Viet Pham, and Song Wang. 2023. Skipanalyzer: An embodied agent for code analysis with large language models. arXiv:2310.18532. Retrieved from https://arxiv.org/abs/2310.18532

[291] Ambarish Moharil and Arpit Sharma. 2022. Identification of intra-domain ambiguity using transformer-based machine learning. In *Proceedings of the 1st international workshop on natural language-based software engineering*, 51–58.

[292] Ambarish Moharil and Arpit Sharma. 2023. Tabasco: A transformer based contextualization toolkit. *Science of Computer Programming* 230, C (Aug 2023), 16 pages. DOI: https://doi.org/10.1016/j.scico.2023.102994

[293] Seungjun Moon, Yongho Song, Hyungjoo Chae, Dongjin Kang, Taeyoon Kwon, Kai Tzu-iunn Ong, Seung-won Hwang, and Jinyoung Yeo. 2023. Coffee: Boost your code LLMS by fixing bugs with feedback. arXiv:2311.07215. Retrieved from https://arxiv.org/abs/2311.07215

[294] Robert C Moore and William Lewis. 2010. Intelligent selection of language model training data. In *Proceedings of the ACL 2010 Conference Short Papers*, 220–224.

[295] Sebastian Moss. 2021. Google brain unveils trillion-parameter ai language model, the largest yet. Retrieved from https://aibusiness.com/nlp/google-brain-unveils-trillion-parameter-ai-language-model-the-largest-yet

[296] Quim Motger, Alessio Miaschi, Felice Dell'Orletta, Xavier Franch, and Jordi Marco. 2024. T-FREX: A transformer-based feature extraction method from mobile app reviews. arXiv:2401.03833. Retrieved from https://arxiv.org/abs/2401.03833

[297] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. ClarifyGPT: Empowering LLM-based code generation with intention clarification. arXiv:2310.10996. Retrieved from https://arxiv.org/abs/2310.10996

[298] Manisha Mukherjee and Vincent J Hellendoorn. 2023. Stack over-flowing with results: The case for domain-specific pre-training over one-size-fits-all models. arXiv:2306.03268. Retrieved from https://arxiv.org/abs/2306.03268

[299] Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, and Nachiappan Nagappan. 2023. Codecompose: A large-scale industrial deployment of AI-assisted code authoring. arXiv:2305.12050. Retrieved from https://arxiv.org/abs/2305.12050

[300] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2023. In-IDE generation-based information support with a large language model. arXiv:2307.08177. Retrieved from https://arxiv.org/abs/2307.08177

[301] Nathalia Nascimento, Paulo Alencar, and Donald Cowan. 2023. Comparing software developers with ChatGPT: An empirical investigation. arXiv:2305.11837. Retrieved from https://arxiv.org/abs/2305.11837

[302] Muhammad U. Nasir, Sam Earle, Julian Togelius, Steven James, and Christopher Cleghorn. 2023. Llmatic: Neural architecture search via large language models and quality-diversity optimization. arXiv:2306.01102. Retrieved from https://arxiv.org/abs/2306.01102

[303] Hira Naveed, Chetan Arora, Hourieh Khalajzadeh, John Grundy, and Omar Haggag. 2024. Model driven engineering for machine learning components: A systematic literature review. *Information and Software Technology* 169 (2024), 107423. DOI: https://doi.org/10.1016/J.INFSOF.2024.107423

[304] Anh Tuan Nguyen, Michael Hilton, Mihai Codoban, Hoan Anh Nguyen, Lily Mast, Eli Rademacher, Tien N. Nguyen, and Danny Dig. 2016. API code recommendation using statistical learning from fine-grained changes. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 511–522.

[305] Anh Tuan Nguyen and Tien N. Nguyen. 2017. Automatic categorization with deep neural network for open-source JAVA projects. In *Proceedings of the 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C '17)*. IEEE, 164–166.

[306] Phuong T. Nguyen, Juri Di Rocco, Claudio Di Sipio, Riccardo Rubei, Davide Di Ruscio, and Massimiliano Di Penta. 2023. Is this snippet written by chatgpt? An empirical study with a codebert-based classifier. arXiv:2307.09381. Retrieved from https://arxiv.org/abs/2307.09381

[307] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. 2023. Lever: Learning to verify language-to-code generation with execution. In *Proceedings of the International Conference on Machine Learning*. PMLR, 26106–26128.

[308] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, Shafiq Joty, Yingbo Zhou, Dragomir Radev, and Arman Cohan. 2023. L2ceval: Evaluating language-to-code generation capabilities of large language models. arXiv:2309.17446. Retrieved from https://arxiv.org/abs/2309.17446

[309] Daniel Nichols, Joshua H. Davis, Zhaojun Xie, Arjun Rajaram, and Abhinav Bhatele. 2024. Can large language models write parallel code? arXiv:2401.12554. Retrieved from https://arxiv.org/abs/2401.12554

[310] Liming Nie, He Jiang, Zhilei Ren, Zeyi Sun, and Xiaochen Li. 2016. Query expansion based on crowd knowledge for code search. *IEEE Transactions on Services Computing* 9, 5 (2016), 771–783.

[311] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for training llms on programming and natural languages. arXiv:2305.02309. Retrieved from https://arxiv.org/abs/2305.02309
[312] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. CodeGen: An open large language model for code with multi-turn program synthesis. arXiv:2203.13474. Retrieved from https://arxiv.org/abs/2203.13474
[313] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. CodeGen: An open large language model for code with multi-turn program synthesis. In *Proceedings of the International Conference on Learning Representations*. Retrieved from https://api.semanticscholar.org/CorpusID:252668917
[314] Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, Liguo Huang, and Bin Luo. 2022. SPT-Code: Sequence-to-sequence pre-training for learning source code representations. In *Proceedings of the 44th International Conference on Software Engineering*, 2006–2018.
[315] David Noever. 2023. Can large language models find and fix vulnerable software? arXiv:2308.10345. Retrieved from https://arxiv.org/abs/2308.10345
[316] Marcel Ochs, Krishna Narasimhan, and Mira Mezini. 2023. Evaluating and improving transformers pre-trained on ASTS for code completion. In *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '16)*. IEEE, 834–844.
[317] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2023. Demystifying GPT self-repair for code generation. arXiv:2306.09896. Retrieved from https://arxiv.org/abs/2306.09896
[318] OpenAI. 2022. Chatgpt: Optimizing language models for dialogue. Retrieved from https://chat.openai.com
[319] OpenAI. 2023. Code interpreter. Retrieved from https://openai.com/blog/chatgpt-plugins#code-interpreter
[320] OpenAI. 2023. GPT-4 technical report. arXiv:2303.08774. Retrieved from https://arxiv.org/abs/2303.08774
[321] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder Paul Christiano, Jan Leike and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, Vol. 35, 27730–27744.
[322] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2023. Llm is like a box of chocolates: The non-determinism of chatgpt in code generation. arXiv:2308.02828. Retrieved from https://arxiv.org/abs/2308.02828
[323] Stack Overflow. 2023. Stack overflow. Retrieved from https://stackoverflow.com/
[324] Jialing Pan, Adrien Sadé, Jin Kim, Eric Soriano, Guillem Sole, and Sylvain Flamant. 2023. Stelocoder: A decoder-only LLM for multi-language to python code translation. arXiv:2310.15539. Retrieved from https://arxiv.org/abs/2310.15539
[325] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2023. Understanding the effectiveness of large language models in code translation. arXiv:2308.03109. Retrieved from https://arxiv.org/abs/2308.03109
[326] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2023. Unifying large language models and knowledge graphs: A roadmap. arXiv:2306.08302. Retrieved from https://arxiv.org/abs/2306.08302
[327] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. 2023. Art: Automatic multi-step reasoning and tool-use for large language models. arXiv:2303.09014. Retrieved from https://arxiv.org/abs/2303.09014
[328] Emilio Parisotto, Abdel-rahman Mohamed, Rishabh Singh, Lihong Li, Dengyong Zhou, and Pushmeet Kohli. 2016. Neuro-symbolic program synthesis. arXiv:1611.01855. Retrieved from https://arxiv.org/abs/1611.01855
[329] Arkil Patel, Siva Reddy, Dzmitry Bahdanau, and Pradeep Dasigi. 2023. Evaluating in-context learning of libraries for code generation. arXiv:2311.09635. Retrieved from https://arxiv.org/abs/2311.09635
[330] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large language model connected with massive APIS. arXiv:2305.15334. Retrieved from https://arxiv.org/abs/2305.15334
[331] Rishov Paul, Md Mohib Hossain, Masum Hasan, and Anindya Iqbal. 2023. Automated program repair based on code review: How do pre-trained transformer models perform? arXiv:2304.07840. Retrieved from https://arxiv.org/abs/2304.07840
[332] Rishov Paul, Md. Mohib Hossain, Mohammed Latif Siddiq, Masum Hasan, Anindya Iqbal, and Joanna C. S. Santos. 2023. Enhancing automated program repair through fine-tuning and prompt engineering. arXiv:2304.07840. Retrieved from https://arxiv.org/abs/2304.07840
[333] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining zero-shot vulnerability repair with large language models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP '23)*. IEEE, 2339–2356.
[334] Tommaso Pegolotti, Elias Frantar, Dan Alistarh, and Markus Püschel. 2023. QIGen: Generating efficient kernels for quantized inference on large language models. arXiv:2307.03738. Retrieved from https://arxiv.org/abs/2307.03738

[335] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can large language models reason about program invariants? In *Proceedings of the 40th International Conference on Machine Learning (ICML'23)*. Vol. 202, 27496–27520.
[336] Yun Peng, Shuzheng Gao, Cuiyun Gao, Yintong Huo, and Michael Lyu. 2024. Domain knowledge matters: Improving prompts with fix templates for repairing python type errors. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
[337] Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Anibal, Alec Peltekian, and Yanfang Ye. 2021. Cotext: Multi-task learning with code-text transformer. arXiv:2105.08645. Retrieved from https://arxiv.org/abs/2105.08645
[338] Benjamin C. Pierce and David N. Turner. 2000. Local type inference. *ACM Transactions on Programming Languages and Systems* 22, 1 (2000), 1–44.
[339] Sanyogita Piya and Allison Sullivan. 2023. LLM4TDD: Best practices for test driven development using large language models. arXiv:2312.04687. Retrieved from https://arxiv.org/abs/2312.04687
[340] Laura Plein, Wendkûuni C. Ouédraogo, Jacques Klein, and Tegawendé F. Bissyandé. 2023. Automatic generation of test cases based on bug reports: A feasibility study with large language models. arXiv:2310.06320. Retrieved from https://arxiv.org/abs/2310.06320
[341] Amrit Poudel, Jinfeng Lin, and Jane Cleland-Huang. 2023. Leveraging transformer-based language models to automate requirements satisfaction assessment. arXiv:2312.04463. Retrieved from https://arxiv.org/abs/2312.04463
[342] Julian Aron Prenner and Romain Robbes. 2021. Making the most of small software engineering datasets with modern machine learning. *IEEE Transactions on Software Engineering* 48, 12 (2021), 5050–5067.
[343] Rohith Pudari and Neil A. Ernst. 2023. From copilot to pilot: Towards AI supported software development. arXiv:2303.04142. Retrieved from https://arxiv.org/abs/2303.04142
[344] Mengnan Qi, Yufan Huang, Maoquan Wang, Yongqiang Yao, Zihan Liu, Bin Gu, Colin Clement, and Neel Sundaresan. 2023. Sut: Active defects probing for transcompiler models. arXiv:2310.14209. Retrieved from https://arxiv.org/abs/2310.14209
[345] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative agents for software development. arXiv:2307.07924. Retrieved from https://arxiv.org/abs/2307.07924
[346] Vu Le Anh Quan, Chau Thuan Phat, Kiet Van Nguyen, Phan The Duy, and Van-Hau Pham. 2023. XGV-BERT: Leveraging contextualized language model and graph neural network for efficient software vulnerability detection. arXiv:2309.14677. Retrieved from https://arxiv.org/abs/2309.14677
[347] Alec Radford and Karthik Narasimhan. 2018. Improving language understanding by generative pre-training. Retrieved from https://api.semanticscholar.org/CorpusID:49313245
[348] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.
[349] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.
[350] Sajjad Rahmani, AmirHossein Naghshzan, and Latifa Guerrouj. 2023. Improving code example recommendations on informal documentation using bert and query-aware LSH: A comparative study. arXiv:2305.03017. Retrieved from https://arxiv.org/abs/2305.03017
[351] Aurora Ramirez, Jose Raul Romero, and Christopher L. Simons. 2018. A systematic review of interaction in search-based software engineering. *IEEE Transactions on Software Engineering* 45, 8 (2018), 760–781.
[352] Sami Ramly. 2023. Preventing abuse of LLMS' alignment deficit by injection neutralization (Paladin). Retrieved from https://medium.com/@SamiRamly/prompt-attacks-are-llm-jailbreaks-inevitable-f7848cc11122
[353] Abhinav Rao, Sachin Vashistha, Atharva Naik, Somak Aditya, and Monojit Choudhury. 2023. Tricking llms into disobedience: Understanding, analyzing, and preventing jailbreaks. arXiv:2305.14965. Retrieved from https://arxiv.org/abs/2305.14965
[354] Nikitha Rao, Jason Tsay, Kiran Kate, Vincent J. Hellendoorn, and Martin Hirzel. 2023. AI for low-code for AI. arXiv:2305.20015. Retrieved from https://arxiv.org/abs/2305.20015
[355] Veselin Raychev, Martin Vechev, and Eran Yahav. 2014. Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 419–428.
[356] Xiaoxue Ren, Xinyuan Ye, Dehai Zhao, Zhenchang Xing, and Xiaohu Yang. 2023. From misuse to mastery: Enhancing code generation with knowledge-driven ai chaining. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 976–987.
[357] Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code generation with alphacodium: From prompt engineering to flow engineering. arXiv:2401.08500. Retrieved from https://arxiv.org/abs/2401.08500
[358] Leanna Rierson. 2017. *Developing Safety-Critical Software: A Practical Guide for Aviation Software and Do-178c Compliance*. CRC Press.

[359] Matthias C. Rillig, Marlene Ågerstrand, Mohan Bi, Kenneth A. Gould, and Uli Sauerland. 2023. Risks and benefits of large language models for the environment. *Environmental Science & Technology* 57, 9 (2023), 3464–3466.
[360] Martin P. Robillard. 2009. What makes apis hard to learn? Answers from developers. *IEEE Software* 26, 6 (2009), 27–34.
[361] Martin P. Robillard and Robert DeLine. 2011. A field study of api learning obstacles. *Empirical Software Engineering* 16 (2011), 703–732.
[362] Tobias Roehm, Rebecca Tiarks, Rainer Koschke, and Walid Maalej. 2012. How do professional developers comprehend software?. In *Proceedings of the 2012 34th International Conference on Software Engineering (ICSE '12)*. IEEE, 255–265.
[363] Krishna Ronanki, Beatriz Cabrero-Daniel, and Christian Berger. 2023. Chatgpt as a tool for user story quality evaluation: Trustworthy out of the box? arXiv:2306.12132. Retrieved from https://arxiv.org/abs/2306.12132
[364] Baptiste Roziere, Marie-Anne Lachaux, Marc Szafraniec, and Guillaume Lample. 2021. DOBF: A deobfuscation pre-training objective for programming languages. arXiv:2102.07492. Retrieved from https://arxiv.org/abs/2102.07492
[365] Fernando Vallecillos Ruiz, Anastasiia Grishina, Max Hort, and Leon Moonen. 2024. A novel approach for automatic program repair using round-trip translation with large language models. arXiv:2401.07994. Retrieved from https://arxiv.org/abs/2401.07994
[366] Iman Saberi, Fatemeh Fard, and Fuxiang Chen. 2023. Multilingual adapter-based knowledge aggregation on code summarization for low-resource languages. arXiv:2307.07854. Retrieved from https://arxiv.org/abs/2307.07854
[367] Iman Saberi, Fatemeh Fard, and Fuxiang Chen. 2023. Utilization of pre-trained language model for adapter-based knowledge transfer in software engineering. arXiv:2307.08540. Retrieved from https://arxiv.org/abs/2307.08540
[368] Ahmed Sadik, Antonello Ceravola, Frank Joublin, and Jibesh Patra. 2023. Analysis of ChatGPT on source code. arXiv:2306.00597. Retrieved from https://arxiv.org/abs/2306.00597
[369] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A systematic survey of prompt engineering in large language models: Techniques and applications. arXiv:2402.07927. Retrieved from https://arxiv.org/abs/2402.07927
[370] Anthony Saieva, Saikat Chakraborty, and Gail Kaiser. 2023. On contrastive learning of semantic similarity forcode to code search. arXiv:2305.03843. Retrieved from https://arxiv.org/abs/2305.03843
[371] Fardin Ahsan Sakib, Saadat Hasan Khan, and A. H. M. Karim. 2023. Extending the frontier of ChatGPT: Code generation and debugging. arXiv:2307.08260. Retrieved from https://arxiv.org/abs/2307.08260
[372] Pasquale Salza, Christoph Schwizer, Jian Gu, and Harald C. Gall. 2023. On the effectiveness of transfer learning for code search. *IEEE Transactions on Software Engineering* 49, 4 (April 2023), 1804–1822. DOI: https://doi.org/10.1109/TSE.2022.3192755
[373] Mahadev Satyanarayanan, David C. Steere, Masashi Kudo, and Hank Mashburn. 1992. Transparent logging as a technique for debugging complex distributed systems. In *Proceedings of the 5th Workshop on ACM SIGOPS European Workshop: Models and Paradigms for Distributed Systems Structuring*, 1–3.
[374] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Villanova del Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, L ucile Saulnier, Samson Tan, Pedro Ortiz Suarez, Victor Sanh, Hugo Laurençon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa, Alham Fikri Aji, Amit Alfassy, Anna Rogers, Ariel Kreisberg Nitzav, Canwen Xu, Chenghao Mou, Chris Emezue, Christopher Klamm, Colin Leong, Daniel van Strien, David Ifeoluwa Adelani, Dragomir Radev, Eduardo González Ponferrada, Efrat Levkovizh, Ethan Kim, Eyal Bar Natan, Francesco De Toni, Gérard Dupont, Germán Kruszewski, Giada Pistilli, Hady Elsahar, Hamza Benyamina, Hieu Tran, Ian Yu, Idris Abdulmumin, Isaac Johnson, Itziar Gonzalez-Dios, Javier de la Rosa, Jenny Chim, Jesse Dodge, Jian Zhu, Jonathan Chang, Jörg Frohberg, Joseph Tobing, Joydeep Bhattacharjee, Khalid Almubarak, Kimbo Chen, Kyle Lo, Leandro Von Werra, Leon Weber, Long Phan, Loubna Ben allal, Ludovic Tanguy, Manan Dey, Manuel Romero Muñoz, Maraim Masoud, María Grandury, Mario Šaško, Max Huang, Maximin Coavoux, Mayank Singh, Mike Tian-Jian Jiang, Minh Chien Vu, Mohammad A. Jauhar, Mustafa Ghaleb, Nishant Subramani, Nora Kassner, Nurulaqilla Khamis, Olivier Nguyen, Omar Espejel, Ona de Gibert, Paulo Villegas, et al. 2022. BLOOM: A 176B-parameter open-access multilingual language model. arXiv:2211.05100. Retrieved from https://arxiv.org/abs/2211.05100
[375] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. Adaptive test generation using a large language model. arXiv:2302.06527. Retrieved from https://arxiv.org/abs/2302.06527
[376] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. DOI: https://doi.org/10.1109/TSE.2023.3334955
[377] Imanol Schlag, Sainbayar Sukhbaatar, Asli Celikyilmaz, Wen tau Yih, Jason Weston, Jürgen Schmidhuber, and Xian Li. 2023. Large language model programs. arXiv:2305.05364 . Retrieved from https://arxiv.org/abs/2305.05364

[378] Martin Schroder. 2023. AutoScrum: Automating project planning using large language models. arXiv:2306.03197. Retrieved from https://arxiv.org/abs/2306.03197

[379] Oussama Ben Sghaier and Houari Sahraoui. 2023. A multi-step learning approach to assist code review. In *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '23)*. IEEE, 450–460.

[380] Murray Shanahan. 2022. Talking about large language models. arXiv:2212.03551. Retrieved from https://arxiv.org/abs/2212.03551

[381] Anton Shapkin, Denis Litvinov, and Timofey Bryksin. 2023. Entity-augmented code generation. arXiv:2312.08976. Retrieved from https://arxiv.org/abs/2312.08976

[382] Rishab Sharma, Fuxiang Chen, Fatemeh Fard, and David Lo. 2022. An exploratory study on code attention in bert. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, 437–448.

[383] Xinyu She, Yanjie Zhao, and Haoyu Wang. 2024. WaDec: Decompile webassembly using large language model. arXiv:2406.11346. Retrieved from https://arxiv.org/abs/2406.11346

[384] Da Shen, Xinyun Chen, Chenguang Wang, Koushik Sen, and Dawn Song. 2022. Benchmarking language models for code syntax understanding. arXiv:2210.14473 . Retrieved from https://arxiv.org/abs/2210.14473

[385] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML'23)*. JMLR.org, Article 1288, 23 pages.

[386] Alexey Shestov, Anton Cheshkov, Rodion Levichev, Ravil Mussabayev, Pavel Zadorozhny, Evgeny Maslov, Chibirev Vadim, and Egor Bulychev. 2024. Finetuning large language models for vulnerability detection. arXiv:2401.17010. Retrieved from https://arxiv.org/abs/2401.17010

[387] Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond. arXiv:2304.05216. Retrieved from https://arxiv.org/abs/2304.05216

[388] Ensheng Shi, Fengji Zhang, Yanlin Wang, Bei Chen, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. SoTaNa: The open-source software development assistant. arXiv:2308.13416. Retrieved from https://arxiv.org/abs/2308.13416

[389] Jieke Shi, Zhou Yang, Bowen Xu, Hong Jin Kang, and David Lo. 2023. Compressing pre-trained models of code into 3 mb. In*ASE '22*. ACM, New York, NY, Article 24, 12 pages. DOI: https://doi.org/10.1145/3551349.3556964

[390] Zejian Shi, Yun Xiong, Xiaolong Zhang, Yao Zhang, Shanshan Li, and Yangyong Zhu. 2022. Cross-modal contrastive learning for code search. In *Proceedings of the 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME '22)*. IEEE, 94–105.

[391] Jiho Shin, Sepehr Hashtroudi, Hadi Hemmati, and Song Wang. 2023. Domain adaptation for deep unit test case generation. arXiv:2308.08033. Retrieved from https://arxiv.org/abs/2308.08033

[392] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. 2023. Prompt engineering or fine tuning: An empirical assessment of large language models in automated software engineering tasks. arXiv:2310.10508. Retrieved from https://arxiv.org/abs/2310.10508

[393] Atsushi Shirafuji, Yutaka Watanobe, Takumi Ito, Makoto Morishita, Yuki Nakamura, Yusuke Oda, and Jun Suzuki. 2023. Exploring the robustness of large language models for solving programming problems. arXiv:2306.14583. Retrieved from https://arxiv.org/abs/2306.14583

[394] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2023. Learning performance-improving code edits. arXiv:2302.07867. Retrieved from https://arxiv.org/abs/2302.07867

[395] Mohammed Latif Siddiq, Beatrice Casey, and Joanna Santos. 2023. A lightweight framework for high-quality code generation. arXiv:2307.08220. Retrieved from https://arxiv.org/abs/2307.08220

[396] Mohammed Latif Siddiq, Joanna Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2023. Exploring the effectiveness of large language models in generating unit tests. arXiv:2305.00418. Retrieved from https://arxiv.org/abs/2305.00418

[397] André Silva, Sen Fang, and Martin Monperrus. 2023. Repairllama: Efficient representations and fine-tuned adapters for program repair. arXiv:2312.15698. Retrieved from https://arxiv.org/abs/2312.15698

[398] Adish Singla. 2023. Evaluating ChatGPT and GPT-4 for visual programming. arXiv:2308.02522. Retrieved from https://arxiv.org/abs/2308.02522

[399] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of chatgpt. arXiv:2301.08653. Retrieved from https://arxiv.org/abs/2301.08653

[400] Giriprasad Sridhara, Ranjani H. G., and Sourav Mazumdar. 2023. ChatGPT: A study on its utility for ubiquitous software engineering tasks. arXiv:2305.16837. Retrieved from https://arxiv.org/abs/2305.16837

[401] Saurabh Srivastava, Sumit Gulwani, and Jeffrey S. Foster. 2010. From program verification to program synthesis. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 313–326.
[402] Benjamin Steenhoek, Hongyang Gao, and Wei Le. 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
[403] Benjamin Steenhoek, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Reinforcement learning from automatic feedback for high-quality unit test generation. arXiv:2310.02368. Retrieved from https://arxiv.org/abs/2310.02368
[404] Hongjin Su, Jungo Kasai, Chen Henry Wu, Weijia Shi, Tianlu Wang, Jiayi Xin, Rui Zhang, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2022. Selective annotation makes language models better few-shot learners. arXiv:2209.01975. Retrieved from https://arxiv.org/abs/2209.01975
[405] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2023. Clover: Closed-loop verifiable code generation. arXiv:2310.17807. Retrieved from https://arxiv.org/abs/2310.17807
[406] Jiamou Sun, Zhenchang Xing, Qinghua Lu, Xiwei Xu, Liming Zhu, Thong Hoang, and Dehai Zhao. 2023. Silent vulnerable dependency alert prediction with vulnerability key aspect explanation. arXiv:2302.07445. Retrieved from https://arxiv.org/abs/2302.07445
[407] Tiezhu Sun, Kevin Allix, Kisub Kim, Xin Zhou, Dongsun Kim, David Lo, Tegawendé F. Bissyandé, and Jacques Klein. 2023. Dexbert: Effective, task-agnostic and fine-grained representation learning of android bytecode. *IEEE Transactions on Software Engineering* 49, 10 (2023), 4691–4706. DOI: https://doi.org/10.1109/TSE.2023.3310874
[408] Weisong Sun, Chunrong Fang, Yudu You, Yuchen Chen, Yi Liu, Chong Wang, Jian Zhang, Quanjun Zhang, Hanwei Qian, Wei Zhao, Yang Liu, and Zhenyu Chen. 2023. A prompt learning framework for source code summarization. arXiv:2312.16066. Retrieved from https://arxiv.org/abs/2312.16066
[409] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Quanjun Zhang, Hanwei Qian, Yang Liu, and Zhenyu Chen. 2023. Automatic code summarization via ChatGPT: How far are we? arXiv:2305.12865. Retrieved from https://arxiv.org/abs/2305.12865
[410] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Miaolei Shi, and Yang Liu. 2024. LLM4Vuln: A unified evaluation framework for decoupling and enhancing LLMS' vulnerability reasoning. arXiv:2401.16185. Retrieved from https://arxiv.org/abs/2401.16185
[411] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2023. When GPT meets program analysis: Towards intelligent detection of smart contract logic vulnerabilities in gptscan. arXiv:2308.03314. Retrieved from https://arxiv.org/abs/2308.03314
[412] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, and Li Li. 2024. When neural code completion models size up the situation: Attaining cheaper and faster completion through dynamic model inference. arXiv:2401.09964. Retrieved from https://arxiv.org/abs/2401.09964
[413] Zhensu Sun, Li Li, Yan Liu, Xiaoning Du, and Li Li. 2022. On the importance of building high-quality training datasets for neural code search. In *Proceedings of the 44th International Conference on Software Engineering*, 1609–1620.
[414] Jeffrey Svajlenko, Judith F. Islam, Iman Keivanloo, Chanchal K. Roy, and Mohammad Mamun Mia. 2014. Towards a big data curated benchmark of inter-project code clones. In *Proceedings of the 2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 476–480.
[415] Jeniya Tabassum, Mounica Maddela, Wei Xu, and Alan Ritter. 2020. Code and named entity recognition in stackoverflow. arXiv:2005.01634. Retrieved from https://arxiv.org/abs/2005.01634
[416] Chee Wei Tan, Shangxin Guo, Man Fai Wong, and Ching Nam Hang. 2023. Copilot for xcode: Exploring AI-assisted programming by prompting cloud-based large language models. arXiv:2307.14349. Retrieved from https://arxiv.org/abs/2307.14349
[417] Wei Tang, Mingwei Tang, Minchao Ban, Ziguo Zhao, and Mingjun Feng. 2023. CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection. *Journal of Systems and Software* 199 (2023), 111623.
[418] Xunzhu Tang, Zhenghan Chen, Kisub Kim, Haoye Tian, Saad Ezzini, and Jacques Klein. 2023. Just-in-time security patch detection–LLM at the rescue for data augmentation. arXiv:2312.01241. Retrieved from https://arxiv.org/abs/2312.01241
[419] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2023. ChatGPT vs SBST: A comparative assessment of unit test suite generation. arXiv:2307.00588. Retrieved from https://arxiv.org/abs/2307.00588
[420] Ze Tang, Jidong Ge, Shangqing Liu, Tingwei Zhu, Tongtong Xu, Liguo Huang, and Bin Luo. 2023. Domain adaptive code completion via language models and decoupled domain databases. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 421–433.
[421] Artur Tarassow. 2023. The potential of LLMS for coding with low-resource and domain-specific programming languages. arXiv:2307.13018. Retrieved from https://arxiv.org/abs/2307.13018

[422] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A large language model for science. arXiv:2211.09085. Retrieved from https://arxiv.org/abs/2211.09085

[423] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2023. VeriGen: A large language model for verilog code generation. arXiv:2308.00708. Retrieved from https://arxiv.org/abs/2308.00708

[424] Chandra Thapa, Seung Ick Jang, Muhammad Ejaz Ahmed, Seyit Camtepe, Josef Pieprzyk, and Surya Nepal. 2022. Transformer-based language models for software vulnerability detection. In *Proceedings of the 38th Annual Computer Security Applications Conference*, 481–496.

[425] Haoye Tian, Kui Liu, Abdoul Kader Kaboré, Anil Koyuncu, Li Li, Jacques Klein, and Tegawendé F. Bissyandé. 2020. Evaluating representation learning of code changes for predicting patch correctness in program repair. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 981–992.

[426] Haoye Tian, Kui Liu, Yinghua Li, Abdoul Kader Kaboré, Anil Koyuncu, Andrew Habib, Li Li, Junhao Wen, Jacques Klein, and Tegawendé F. Bissyandé. 2023. The best of both worlds: Combining learned embeddings with engineered features for accurate prediction of correct patches. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–34.

[427] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tegawendé F. Bissyandé. 2023. Is chatgpt the ultimate programming assistant–How far is it? arXiv:2304.11938. Retrieved from https://arxiv.org/abs/2304.11938

[428] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. DebugBench: Evaluating debugging capability of large language models. arXiv:2401.04621. Retrieved from https://arxiv.org/abs/2401.04621

[429] Zhao Tian and Junjie Chen. 2023. Test-case-driven programming understanding in large language models for better code generation. arXiv:2309.16120. Retrieved from https://arxiv.org/abs/2309.16120

[430] Norbert Tihanyi, Tamas Bisztray, Ridhi Jain, Mohamed Amine Ferrag, Lucas C. Cordeiro, and Vasileios Mavroeidis. 2023. The formai dataset: Generative AI in software security through the lens of formal verification. arXiv:2307.02192. Retrieved from https://arxiv.org/abs/2307.02192

[431] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from https://arxiv.org/abs/2302.13971

[432] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288. Retrieved from https://arxiv.org/abs/2307.09288

[433] Haoxin Tu, Zhide Zhou, He Jiang, Imam Nur Bani Yusuf, Yuxian Li, and Lingxiao Jiang. 2023. LLM4CBI: Taming Llms to generate effective test programs for compiler bug isolation. arXiv:2307.00593. Retrieved from https://arxiv.org/abs/2307.00593

[434] Michele Tufano, Shubham Chandel, Anisha Agarwal, Neel Sundaresan, and Colin Clement. 2023. Predicting code coverage without execution. arXiv:2307.13383. Retrieved from https://arxiv.org/abs/2307.13383

[435] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *Proceedings of the 44th International Conference on Software Engineering*, 2291–2302.

[436] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates Inc., Red Hook, NY, 6000–6010.

[437] Vasudev Vikram, Caroline Lemieux, and Rohan Padhye. 2023. Can large language models write good property-based tests? arXiv:2307.04346. Retrieved from https://arxiv.org/abs/2307.04346

[438] Julian Von der Mosel, Alexander Trautsch, and Steffen Herbold. 2022. On the validity of pre-trained transformers for natural language processing in the software engineering domain. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1487–1507.

[439] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. 2023. Frustrated with code quality issues? LLMS can help! arXiv:2309.12938. Retrieved from https://arxiv.org/abs/2309.12938

[440] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip Yu. 2019. Multi-modal attention network learning for semantic source code retrieval. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE '19)*. IEEE, 13–25.

[441] Yao Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Dezhong Yao, Hai Jin, and Lichao Sun. 2022. You see what I want you to see: Poisoning vulnerabilities in neural code search. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, New York, NY, 1233–1245. DOI: https://doi.org/10.1145/3540250.3549153

[442] Yao Wan, Wei Zhao, Hongyu Zhang, Yulei Sui, Guandong Xu, and Hai Jin. 2022. What do they capture? A structural analysis of pre-trained language models for source code. In *Proceedings of the 44th International Conference on Software Engineering*, 2377–2388.

[443] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S. Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 397–407.

[444] Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 billion parameter autoregressive language model. Retrieved from https://github.com/kingoflolz/mesh-transformer-jax

[445] Chong Wang, Jianan Liu, Xin Peng, Yang Liu, and Yiling Lou. 2023. Boosting static resource leak detection via LLM-based resource-oriented intention inference. arXiv:2311.04448. Retrieved from https://arxiv.org/abs/2311.04448

[446] Chong Wang, Jian Zhang, Yebo Feng, Tianlin Li, Weisong Sun, Yang Liu, and Xin Peng. 2024. Teaching code LLMS to use autocompletion tools in repository-level code generation. arXiv:2401.06391. Retrieved from https://arxiv.org/abs/2401.06391

[447] Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. 2023. One adapter for all programming languages? Adapter tuning for code search and summarization. arXiv:2303.15822. Retrieved from https://arxiv.org/abs/2303.15822

[448] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2023. Software testing with large language model: Survey, landscape, and vision. arXiv:2307.07221. Retrieved from https://arxiv.org/abs/2307.07221

[449] Jian Wang, Shangqing Liu, Xiaofei Xie, and Yi Li. 2023. Evaluating aigc detectors on code content. arXiv:2304.05193. Retrieved from https://arxiv.org/abs/2304.05193

[450] Shuai Wang, Liang Ding, Li Shen, Yong Luo, Bo Du, and Dacheng Tao. 2024. OOP: Object-oriented programming evaluation benchmark for large language models. arXiv:2401.06628. Retrieved from https://arxiv.org/abs/2401.06628

[451] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F. Bissyandé, and Xiaoguang Mao. 2023. Natural language to code: How far are we?. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 375–387.

[452] Simin Wang, Liguo Huang, Amiao Gao, Jidong Ge, Tengfei Zhang, Haitao Feng, Ishna Satyarth, Ming Li, He Zhang, and Vincent Ng. 2022. Machine/deep learning for software engineering: A systematic literature review. *IEEE Transactions on Software Engineering* 49, 3 (2022), 1188–1231.

[453] Shufan Wang, Sebastien Jean, Sailik Sengupta, James Gung, Nikolaos Pappas, and Yi Zhang. 2023. Measuring and mitigating constraint violations of in-context learning for utterance-to-API semantic parsing. arXiv:2305.15338. Retrieved from https://arxiv.org/abs/2305.15338

[454] Shiqi Wang, Zheng Li, Haifeng Qian, Chenghao Yang, Zijian Wang, Mingyue Shang, Varun Kumar, Samson Tan, Baishakhi Ray, Parminder Bhatia, Ramesh Nallapati, Murali Krishna Ramanathan, Dan Roth, and Bing Xiang. 2022. ReCode: Robustness evaluation of code generation models. arXiv:2212.10264. Retrieved from https://arxiv.org/abs/2212.10264

[455] Wenhan Wang, Ge Li, Bo Ma, Xin Xia, and Zhi Jin. 2020. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In *Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER '20)*. IEEE, 261–271.

[456] Wenhan Wang, Ge Li, Sijie Shen, Xin Xia, and Zhi Jin. 2020. Modular tree network for source code representation learning. *ACM Transactions on Software Engineering and Methodology* 29, 4 (2020), 1–23.

[457] Weishi Wang, Yue Wang, Shafiq Joty, and Steven C. H. Hoi. 2023. RAP-Gen: Retrieval-augmented patch generation with codet5 for automatic program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 146–158.

[458] Xingyao Wang, Hao Peng, Reyhaneh Jabbarvand, and Heng Ji. 2023. LeTI: Learning to generate from textual interactions. arXiv:2305.10314. Retrieved from https://arxiv.org/abs/2305.10314

[459] Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. 2021. SynCoBERT: Syntax-guided multi-modal contrastive pre-training for code representation. arXiv:2108.04556. Retrieved from https://arxiv.org/abs/2108.04556

[460] Yanlin Wang, Yanxian Huang, Daya Guo, Hongyu Zhang, and Zibin Zheng. 2024. Sparsecoder: Identifier-aware sparse transformer for file-level code summarization. arXiv:2401.14727. Retrieved from https://arxiv.org/abs/2401.14727

[461] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. arXiv:2305.07922. Retrieved from https://arxiv.org/abs/2305.07922

[462] Yawen Wang, Lin Shi, Mingyang Li, Qing Wang, and Yun Yang. 2020. A deep context-wise method for coreference detection in natural language requirements. In *Proceedings of the 2020 IEEE 28th International Requirements Engineering Conference (RE '20)*. IEEE, 180–191.

[463] Yawen Wang, Junjie Wang, Hongyu Zhang, Xuran Ming, Lin Shi, and Qing Wang. 2022. Where is your app frustrating users? In *Proceedings of the 44th International Conference on Software Engineering*, 2427–2439.

[464] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. arXiv:2109.00859. Retrieved from https://arxiv.org/abs/2109.00859

[465] Zejun Wang, Jia Li, Ge Li, and Zhi Jin. 2023. Chatcoder: Chat-based refine requirement improves LLMs' code generation. arXiv:2311.00272. Retrieved from https://arxiv.org/abs/2311.00272

[466] Cody Watson, Nathan Cooper, David Nader Palacio, Kevin Moran, and Denys Poshyvanyk. 2022. A systematic literature review on the use of deep learning in software engineering research. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 1–58.

[467] Huihui Wei and Ming Li. 2017. Supervised deep features for software functional clone detection by exploiting lexical and syntactical information in source code. In *IJCAI*, 3034–3040.

[468] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, Vol. 35, 24824–24837.

[469] Moshi Wei, Nima Shiri Harzevili, Yuchao Huang, Junjie Wang, and Song Wang. 2022. Clear: Contrastive learning for API recommendation. In *Proceedings of the 44th International Conference on Software Engineering*, 376–387.

[470] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Source code is all you need. arXiv:2312.02120. Retrieved from https://arxiv.org/abs/2312.02120

[471] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 172–184.

[472] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. 2023. Exploring parameter-efficient fine-tuning techniques for code generation with large language models. arXiv:2308.10462. Retrieved from https://arxiv.org/abs/2308.10462

[473] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. 2023. On the usage of continual learning for out-of-distribution generalization in pre-trained language models of code. arXiv:2305.04106. Retrieved from https://arxiv.org/abs/2305.04106

[474] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with ChatGPT. arXiv:2302.11382. Retrieved from https://arxiv.org/abs/2302.11382

[475] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. arXiv:2303.07839. Retrieved from https://arxiv.org/abs/2303.07839

[476] Patricia Widjojo and Christoph Treude. 2023. Addressing compiler errors: Stack overflow or large language models? arXiv:2307.10793. Retrieved from https://arxiv.org/abs/2307.10793

[477] Ratnadira Widyasari, Ting Zhang, Abir Bouraffa, and David Lo. 2023. Explaining explanation: An empirical study on explanation in code reviews. arXiv:2311.09020. Retrieved from https://arxiv.org/abs/2311.09020

[478] Man-Fai Wong, Shangxin Guo, Ching-Nam Hang, Siu-Wai Ho, and Chee-Wei Tan. 2023. Natural language generation and understanding of big code for AI-assisted programming: A review. *Entropy* 25, 6 (2023), 888.

[479] Di Wu, Yang Feng, Hongyu Zhang, and Baowen Xu. 2024. Automatic recognizing relevant fragments of APIs using API references. *Automated Software Engineering* 31, 1 (2024), 3.

[480] Fangzhou Wu, Xiaogeng Liu, and Chaowei Xiao. 2023. Deceptprompt: Exploiting LLM-driven code generation via adversarial natural language instructions. arXiv:2312.04730. Retrieved from https://arxiv.org/abs/2312.04730

[481] Yueqi Xie, Jiawei Shao, Justin Curl, Lingjuan Lyu, Qifeng Chen, Xing Xie and Xing Xie. 2023. Defending ChatGPT against jailbreak attack via self-reminders. *Nature Machine Intelligence* 5 (2023), 1486–1496.

[482] Qianou Ma, Tongshuang Wu, and Kenneth Koedinger. 2023. Is AI the better programming partner? Human-human pair programming vs. human-AI pair programming. arXiv:2306.05153. Retrieved from https://arxiv.org/abs/2306.05153

[483] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How effective are neural networks for fixing security vulnerabilities. arXiv:2305.18607. Retrieved from https://arxiv.org/abs/2305.18607

[484] Yonghao Wu, Zheng Li, Jie M Zhang, Mike Papadakis, Mark Harman, and Yong Liu. 2023. Large language models in fault localisation. arXiv:2308.15276. Retrieved from https://arxiv.org/abs/2308.15276

[485] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. arXiv:2308.04748. Retrieved from https://arxiv.org/abs/2308.04748

[486] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2022. Practical program repair in the era of large pre-trained language models. arXiv:2210.14179. Retrieved from https://arxiv.org/abs/2210.14179

[487] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. DOI: https://doi.org/10.1109/ICSE48619.2023.00129

[488] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational automated program repair. arXiv:2301.13246. Retrieved from https://arxiv.org/abs/2301.13246

[489] Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the conversation going: Fixing 162 out of 337 bugs for 0.42 each using ChatGPT. arXiv:2304.00385. Retrieved from https://arxiv.org/abs/2304.00385

[490] Danning Xie, Byungwoo Yoo, Nan Jiang, Mijung Kim, Lin Tan, Xiangyu Zhang, and Judy S. Lee. 2023. Impact of large language models on generating software specifications. arXiv:2306.03324. Retrieved from https://arxiv.org/abs/2306.03324

[491] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. Chatunitest: A ChatGPT-based automated unit test generation tool. arXiv:2305.04764. Retrieved from https://arxiv.org/abs/2305.04764

[492] Weimin Xiong, Yiwen Guo, and Hao Chen. 2023. The program testing ability of large language models for code. arXiv:2310.05727. Retrieved from https://arxiv.org/abs/2310.05727

[493] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 1–10.

[494] Junjielong Xu, Ziang Cui, Yuan Zhao, Xu Zhang, Shilin He, Pinjia He, Liqun Li, Yu Kang, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2024. UniLog: Automatic logging via LLM and in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–12.

[495] Xiangzhe Xu, Zhuo Zhang, Shiwei Feng, Yapeng Ye, Zian Su, Nan Jiang, Siyuan Cheng, Lin Tan, and Xiangyu Zhang. 2023. LMPA: Improving decompilation by synergy of large language model and program analysis. arXiv:2306.02546. Retrieved from https://arxiv.org/abs/2306.02546

[496] Zhuolin Xu, Yuanzhang Lin, Qiushi Li, and Shin Hwei Tan. 2023. Guiding chatgpt to fix web UI tests via explanation-consistency checking. arXiv:2312.05778. Retrieved from https://arxiv.org/abs/2312.05778

[497] Dapeng Yan, Zhipeng Gao, and Zhiming Liu. 2023. A closer look at different difficulty levels code generation abilities of chatgpt. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 1887–1898.

[498] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. 2023. Codetransocean: A comprehensive multilingual benchmark for code translation. arXiv:2310.04951. Retrieved from https://arxiv.org/abs/2310.04951

[499] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent Hellendoorn. 2024. Large language models for test-free fault localization. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–12.

[500] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2023. White-box compiler fuzzing empowered by large language models. arXiv:2310.15991. Retrieved from https://arxiv.org/abs/2310.15991

[501] Chengran Yang, Jiakun Liu, Bowen Xu, Christoph Treude, Yunbo Lyu, Ming Li, and David Lo. 2023. APIDoc-Booster: An extract-then-abstract framework leveraging large language models for augmenting API documentation. arXiv:2312.10934. Retrievedf from https://arxiv.org/abs/2312.10934

[502] Chengran Yang, Bowen Xu, Junaed Younus Khan, Gias Uddin, Donggyun Han, Zhou Yang, and David Lo. 2022. Aspect-based api review classification: How far can pre-trained transformer model go?. In *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '22)*. IEEE, 385–395.

[503] Di Yang, Aftab Hussain, and Cristina Videira Lopes. 2016. From query to usable code: An analysis of stack overflow code snippets. In *Proceedings of the 13th International Conference on Mining Software Repositories*, 391–402.

[504] Guang Yang, Yu Zhou, Xiang Chen, Xiangyu Zhang, Yiran Xu, Tingting Han, and Taolue Chen. 2023. A syntax-guided multi-task learning approach for turducken-style code generation. arXiv:2303.05061. Retrieved from https://arxiv.org/abs/2303.05061

[505] Guang Yang, Yu Zhou, Xiangyu Zhang, Xiang Chen, Tingting Han, and Taolue Chen. 2023. Assessing and improving syntactic adversarial robustness of pre-trained models for code translation. arXiv:2310.18587. Retrieved from https://arxiv.org/abs/2310.18587

[506] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Bing Yin, and Xia Hu. 2023. Harnessing the power of LLMs in practice: A survey on chatgpt and beyond. arXiv:2304.13712. Retrieved from https://arxiv.org/abs/2304.13712

[507] Kang Yang, Xinjun Mao, Shangwen Wang, Tanghaoran Zhang, Bo Lin, Yanlin Wang, Yihao Qin, Zhang Zhang, and Xiaoguang Mao. 2023. Enhancing code intelligence tasks with ChatGPT. arXiv:2312.15202. Retrieved from https://arxiv.org/abs/2312.15202

[508] Lanxin Yang, He Zhang, Haifeng Shen, Xin Huang, Xin Zhou, Guoping Rong, and Dong Shao. 2021. Quality assessment in systematic literature reviews: A software engineering perspective. *Information and Software Technology* 130 (2021), 106397.

[509] Yanming Yang, Xin Xia, David Lo, and John Grundy. 2022. A survey on deep learning for software engineering. *ACM Computing Surveys* 54, 10s (2022), 1–73.

[510] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural attack for pre-trained models of code. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 1482–1493. DOI: https://doi.org/10.1145/3510003.3510146

[511] Zhou Yang, Zhensu Sun, Terry Yue Zhuo, Premkumar T. Devanbu, and David Lo. 2024. Robustness, security, privacy, explainability, efficiency, and usability of large language models for code. arXiv:2403.07506. DOI: https://doi.org/10.48550/ARXIV.2403.07506

[512] Zhou Yang, Bowen Xu, Jie M. Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2023. Stealthy backdoor attack for code models. DOI: https://doi.org/10.48550/ARXIV.2301.02496

[513] Jiacheng Ye, Chengzu Li, Lingpeng Kong, and Tao Yu. 2023. Generating data for symbolic language with large language models. arXiv:2305.13917. Retrieved from https://arxiv.org/abs/2305.13917

[514] Ryan Yen, Jiawen Zhu, Sangho Suh, Haijun Xia, and Jian Zhao. 2023. Coladder: Supporting programmers with hierarchical code generation in multi-level abstraction. arXiv:2310.08699. Retrieved from https://arxiv.org/abs/2310.08699

[515] Burak Yetiştiren, Işik Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778. Retrieved from https://arxiv.org/abs/2304.10778

[516] Pengcheng Yin and Graham Neubig. 2017. A syntactic neural model for general-purpose code generation. arXiv:1704.01696. Retrieved from https://arxiv.org/abs/1704.01696

[517] ymcui. 2023. Chinese LLaMA & Alpaca large language models. Retrieved from https://github.com/ymcui/Chinese-LLaMA-Alpaca-2/blob/main/README_EN.md

[518] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous large language model agents enabling intent-driven mobile GUI testing. arXiv:2311.08649. Retrieved from https://arxiv.org/abs/2311.08649

[519] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Tao Xie, and Qianxiang Wang. 2023. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. arXiv:2302.00288. Retrieved from https://arxiv.org/abs/2302.00288

[520] Siyu Yu, Yifan Wu, Zhijing Li, Pinjia He, Ningjiang Chen, and Changjian Liu. 2023. Log parsing with generalization ability under new log types. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 425–437.

[521] Wei Yuan, Quanjun Zhang, Tieke He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. Circle: Continual repair across programming languages. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 678–690.

[522] Zhiqiang Yuan, Junwei Liu, Qiancheng Zi, Mingwei Liu, Xin Peng, and Yiling Lou. 2023. Evaluating instruction-tuned large language models on code comprehension and generation. arXiv:2308.01240. Retrieved from https://arxiv.org/abs/2308.01240

[523] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No more manual tests? Evaluating and improving chatgpt for unit test generation. arXiv:2305.04207. Retrieved from https://arxiv.org/abs/2305.04207

[524] Daoguang Zan, Bei Chen, Yongshun Gong, Junzhi Cao, Fengji Zhang, Bingchao Wu, Bei Guan, Yilong Yin, and Yongji Wang. 2023. Private-library-oriented code generation with large language models. arXiv:2307.15370. Retrieved from https://arxiv.org/abs/2307.15370

[525] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Yongji Wang, and Jian-Guang Lou. 2022. When language model meets private library. arXiv:2210.17236. Retrieved from https://arxiv.org/abs/2210.17236

[526] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: Continual pre-training on sketches for library-oriented code generation. arXiv:2206.06888. Retrieved from https://arxiv.org/abs/2206.06888

[527] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2023. Large language models meet NL2Code: A survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Long papers)*, Vol. 1, 7443–7464.

[528] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2023. Self-taught optimizer (stop): Recursively self-improving code generation. arXiv:2310.02304. Retrieved from https://arxiv.org/abs/2310.02304

[529] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 39–51.

[530] Cen Zhang, Mingqiang Bai, Yaowen Zheng, Yeting Li, Xiaofei Xie, Yuekang Li, Wei Ma, Limin Sun, and Yang Liu. 2023. Understanding large language model based fuzz driver generation. arXiv:2307.12469. Retrieved from https://arxiv.org/abs/2307.12469

[531] Chenyuan Zhang, Hao Liu, Jiutian Zeng, Kejing Yang, Yuhong Li, and Hui Li. 2023. Prompt-enhanced software vulnerability detection using chatgpt. arXiv:2308.12697. Retrieved from https://arxiv.org/abs/2308.12697

[532] He Zhang, Muhammad Ali Babar, and Paolo Tell. 2011. Identifying relevant studies in software engineering. *Information and Software Technology* 53, 6 (2011), 625–637.

[533] Jingxuan Zhang, Siyuan Liu, Lina Gong, Haoxiang Zhang, Zhiqiu Huang, and He Jiang. 2023. Beqain: An effective and efficient identifier normalization approach with bert and the question answering system. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2597–2620. DOI: https://doi.org/10.1109/TSE.2022.3227559

[534] Jialu Zhang, Todd Mytkowicz, Mike Kaufman, Ruzica Piskac, and Shuvendu K. Lahiri. 2022. Using pre-trained language models to resolve textual and semantic merge conflicts (experience paper). In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 77–88.

[535] Jiyang Zhang, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2023. Multilingual code co-evolution using large language models. arXiv:2307.14991. Retrieved from https://arxiv.org/abs/2307.14991

[536] Jiyang Zhang, Sheena Panthaplackel, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2022. CoditT5: Pretraining for source code and natural language editing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–12.

[537] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2020. Retrieval-based neural source code summarization. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 1385–1397.

[538] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. 2019. A novel neural source code representation based on abstract syntax tree. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE '19)*. IEEE, 783–794.

[539] Kechi Zhang, Ge Li, Jia Li, Zhuo Li, and Zhi Jin. 2023. Toolcoder: Teach code generation models to use apis with search tools. arXiv:2305.04032. Retrieved from https://arxiv.org/abs/2305.04032

[540] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. arXiv:2401.07339. Retrieved from https://arxiv.org/abs/2401.07339

[541] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-edit: Fault-aware code editor for code generation. arXiv:2305.04087. Retrieved from https://arxiv.org/abs/2305.04087

[542] Kexun Zhang, Danqing Wang, Jingtao Xia, William Yang Wang, and Lei Li. 2023. Algo: Synthesizing algorithmic programs with generated oracle verifiers. arXiv:2305.14591. Retrieved from https://arxiv.org/abs/2305.14591

[543] Lichen Zhang, Shuai Lu, and Nan Duan. 2024. Selene: Pioneering automated proof in software verification. arXiv:2401.07663. Retrieved from https://arxiv.org/abs/2401.07663

[544] Quanjun Zhang, Chunrong Fang, Weisong Sun, Yan Liu, Tieke He, Xiaodong Hao, and Zhenyu Chen. 2023. Boosting automated patch correctness prediction via pre-trained language model. arXiv:2301.12453. Retrieved from https://arxiv.org/abs/2301.12453

[545] Quanjun Zhang, Chunrong Fang, Weisong Sun, Yan Liu, Tieke He, Xiaodong Hao, and Zhenyu Chen. 2024. APPT: Boosting automated patch correctness prediction via fine-tuning pre-trained models. *IEEE Transactions on Software Engineering* (2024).

[546] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2023. A survey on large language models for software engineering. arXiv:2312.15223. Retrieved from https://arxiv.org/abs/2312.15223

[547] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting template-based automated program repair via mask prediction. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 535–547.

[548] Simiao Zhang, Jiaping Wang, Guoliang Dong, Jun Sun, Yueling Zhang, and Geguang Pu. 2024. Experimenting a new programming practice with LLMs. arXiv:2401.01062. Retrieved from https://arxiv.org/abs/2401.01062

[549] Ting Zhang, DongGyun Han, Venkatesh Vinayakarao, Ivana Clairine Irsan, Bowen Xu, Ferdian Thung, David Lo, and Lingxiao Jiang. 2023. Duplicate bug report detection: How far are we? *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–32.

[550] Ting Zhang, Ivana Clairine Irsan, Ferdian Thung, and David Lo. 2023. Cupid: Leveraging chatgpt for more accurate duplicate bug report detection. arXiv:2308.10022. Retrieved from https://arxiv.org/abs/2308.10022

[551] Ting Zhang, Ivana Clairine Irsan, Ferdian Thung, and David Lo. 2023. Revisiting sentiment analysis for software engineering in the era of large language models. arXiv:2310.11113. Retrieved from https://arxiv.org/abs/2310.11113.

[552] Ting Zhang, Ivana Clairine Irsan, Ferdian Thung, David Lo, Asankhaya Sharma, and Lingxiao Jiang. 2023. Evaluating pre-trained language models for repairing api misuses. arXiv:2310.16390. Retrieved from https://arxiv.org/abs/2310.16390

[553] Ting Zhang, Bowen Xu, Ferdian Thung, Stefanus Agus Haryono, David Lo, and Lingxiao Jiang. 2020. Sentiment analysis for software engineering: How far can pre-trained transformer models go? In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME '20)*. IEEE, 70–80.

[554] Tianyi Zhang, Tao Yu, Tatsunori Hashimoto, Mike Lewis, Wen-tau Yih, Daniel Fried, and Sida Wang. 2023. Coder reviewer reranking for code generation. In *Proceedings of the International Conference on Machine Learning*. PMLR, 41832–41846.

[555] Yuwei Zhang, Zhi Jin, Ying Xing, and Ge Li. 2023. Steam: Simulating the interactive behavior of programmers for automatic bug fixing. arXiv:2308.14460. Retrieved from https://arxiv.org/abs/2308.14460

[556] Yuwei Zhang, Ge Li, Zhi Jin, and Ying Xing. 2023. Neural program repair with program dependence analysis and effective filter mechanism. arXiv:2305.09315. Retrieved from https://arxiv.org/abs/2305.09315

[557] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2022. Automatic chain of thought prompting in large language models. arXiv:2210.03493. Retrieved from https://arxiv.org/abs/2210.03493

[558] Jianyu Zhao, Yuyang Rong, Yiwen Guo, Yifeng He, and Hao Chen. 2023. Understanding programs by exploiting (fuzzing) test cases. arXiv:2305.13592. Retrieved from https://arxiv.org/abs/2305.13592

[559] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2023. A survey of large language models. arXiv:2303.18223. Retrieved from https://arxiv.org/abs/2303.18223

[560] Xu Zhao, Yuxi Xie, Kenji Kawaguchi, Junxian He, and Qizhe Xie. 2023. Automatic model selection with large language models for reasoning. arXiv:2305.14333. Retrieved from https://arxiv.org/abs/2305.14333

[561] Yanjie Zhao, Li Li, Haoyu Wang, Haipeng Cai, Tegawendé F Bissyandé, Jacques Klein, and John Grundy. 2021. On the impact of sample duplication in machine-learning-based android malware detection. *ACM Transactions on Software Engineering and Methodology* 30, 3 (2021), 1–38.

[562] Zelin Zhao, Zhaogui Xu, Jialong Zhu, Peng Di, Yuan Yao, and Xiaoxing Ma. 2023. The right prompts for the job: Repair code-review defects with large language model. arXiv:2312.17485. Retrieved from https://arxiv.org/abs/2312.17485

[563] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A pre-trained model for code generation with multilingual evaluations on humaneval-x. arXiv:2303.17568. Retrieved from https://arxiv.org/abs/2303.17568

[564] Wenqing Zheng, S. P. Sharan, Ajay Kumar Jaiswal, Kevin Wang, Yihan Xi, Dejia Xu, and Zhangyang Wang. 2023. Outline, then details: Syntactically guided coarse-to-fine code generation. arXiv:2305.00909. Retrieved from https://arxiv.org/abs/2305.00909

[565] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2023. A survey of large language models for code: Evolution, benchmarking, and future trends. arXiv:2311.10372. Retrieved from https://arxiv.org/abs/2311.10372

[566] Li Zhong and Zilong Wang. 2023. A study on robustness and reliability of large language model code generation. arXiv:2308.10335. Retrieved from https://arxiv.org/abs/2308.10335

[567] Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. Codebertscore: Evaluating code generation with pretrained models of code. arXiv:2302.05527. Retrieved from https://arxiv.org/abs/2302.05527

[568] Shufan Zhou, Beijun Shen, and Hao Zhong. 2019. Lancer: Your code tell me what you need. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE '19)*. IEEE, 1202–1205.

[569] Wenxuan Zhou, Sheng Zhang, Yu Gu, Muhao Chen, and Hoifung Poon. 2023. Universalner: Targeted distillation from large language models for open named entity recognition. arXiv:2308.03279. Retrieved from https://arxiv.org/abs/2308.03279

[570] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. 2024. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM, 88:1–88:13.

[571] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. arXiv:2211.01910. Retrieved from https://arxiv.org/abs/2211.01910

[572] Jie Zhu, Lingwei Li, Li Yang, Xiaoxiao Ma, and Chun Zuo. 2023. Automating method naming with context-aware prompt-tuning. arXiv:2303.05771. Retrieved from https://arxiv.org/abs/2303.05771

[573] Jianfei Zhu, Guanping Xiao, Zheng Zheng, and Yulei Sui. 2022. Enhancing traceability link recovery with unlabeled data. In *Proceedings of the 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE '22)*. IEEE, 446–457.

[574] Terry Yue Zhuo. 2023. Large language models are state-of-the-art evaluators of code generation. arXiv:2304.14317. Retrieved from https://arxiv.org/abs/2304.14317

[575] Terry Yue Zhuo, Xiaoning Du, Zhenchang Xing, Jiamou Sun, Haowei Quan, Li Li, and Liming Zhu. 2023. Pop quiz! Do pre-trained code models possess knowledge of correct API names? arXiv:2309.07804. Retrieved from https://arxiv.org/abs/2309.07804

Appendices

A Data Types

We classified the data types of all datasets into five categories: code-based, text-based, graph-based, software repository-based, and combined data types, as shown in Table A1.

Table A1. Data Types of Datasets Involved in Prior Studies

Category	Data type	# Studies	References
Text-based datasets	Programming tasks/problems	42	[32, 38, 43, 45, 59, 133, 138, 142, 161, 166, 176, 207–209, 213, 222, 224, 225, 236, 252, 259, 311, 329, 356, 368, 371, 387, 392, 399, 429, 450, 458, 465, 470, 492, 514, 524, 526, 540, 554, 567, 574]
	Prompts	33	[34, 73, 76, 107, 169, 170, 188, 189, 193, 196, 219, 231, 240, 253, 266, 273, 309, 374, 395, 398, 416, 419, 421, 430, 433, 437, 449, 453, 454, 474, 495, 513, 558]
	SO posts	12	[26, 129, 130, 132, 147, 204, 271, 298, 350, 388, 469, 553]
	Bug reports	11	[55, 56, 64, 91, 110, 132, 152, 158, 173, 185, 216]
	Requirements documentation	9	[79, 83, 132, 135, 203, 272, 292, 341, 462]
	APIs/API documentation	8	[63, 148, 190, 330, 479, 501, 502, 525]
	Q&A pairs	6	[180, 345, 377, 438, 449, 571]
	Vulnerability descriptions	4	[333, 410, 424, 483]
	Reviews	4	[268, 296, 477, 551]
	Logs	3	[263, 275, 520]
	Methods	3	[283, 286, 523]
	Project issues	3	[98, 411, 550]
	Code comments	2	[342, 493]
	Theorems	2	[95, 543]
	Buggy text	1	[265]
	Dockerfiles	1	[134]
	Outage descriptions	1	[174]
	Semantic merge conflicts	1	[534]
	Site text	1	[201]
	Software development tasks	1	[548]
	User intents	1	[162]
	Software specifications	1	[279]
	User reviews	1	[463]

(Continued)

Table A1. Continued

Category	Data type	# Studies	References
Code-based datasets	Source code	60	[1, 10, 19, 31, 37, 40, 53, 57, 86, 102, 104, 141, 151, 160, 165, 179, 181, 223, 242, 243, 249, 251, 255, 256, 274, 276, 282, 299, 306, 316, 324, 342, 344, 366, 381, 382, 384, 388, 390, 391, 392, 396, 403, 409, 412, 417, 442, 461, 478, 491, 494, 498, 504, 505, 519, 529, 539, 556, 564, 572]
	Bugs/Buggy code	16	[33, 59, 77, 75, 125, 183, 184, 194, 332, 336, 397, 457, 471, 486, 488, 555]
	Vulnerable source code	8	[35, 48, 101, 114, 195, 346, 402, 531]
	Patches	4	[215, 425, 426, 544]
	Code changes	3	[106, 230, 535]
	Test suites/cases	3	[164, 493, 542]
	Bug-fix pairs	2	[84, 521]
	Error code	2	[150, 476]
	Error-fix pairs	1	[54]
	Flaky test cases	1	[90]
	Identifiers	1	[533]
	Labeled clone pairs	1	[74]
	Packages	1	[375]
Graph-based datasets	GUI images	1	[203]
Software repository-based datasets	Code repository	9	[21, 44, 78, 122, 172, 290, 420, 446, 540]
	Android apps	3	[264, 407, 518]
	Issues and commits	3	[11, 246, 553]
	Pull-requests	2	[379, 553]
	Industrial projects	1	[239]
	Open-source projects	1	[227]
	Web applications	1	[496]
Combined datasets	Programming tasks and test suites/cases	17	[72, 89, 120, 144, 145, 163, 240, 297, 317, 322, 339, 357, 376, 393, 394, 427, 497]
	Source code and comments	12	[115, 121, 210, 285, 314, 342, 367, 435, 472, 507, 536, 541]
	Programming tasks and solutions	8	[67, 70, 124, 192, 205, 335, 370, 428]
	Source code and description	3	[248, 405, 447]
	Code-text pairs	2	[280, 354]
	Source code and API usage sequences	2	[473, 575]
	Source code and test suites/cases	2	[365, 434]
	Bug report and test suites/cases	1	[340]
	Buggy code and comments	1	[562]
	Buggy code and solutions	1	[293]
	Code files and summaries	1	[460]
	Binary code and related annotations	1	[9]
	Failing test code and error messages	1	[489]
	Source code and Q&A pairs	1	[372]
	Source code, methods, and logs	1	[238]
	Vulnerable code and description	1	[480]

B Input Forms

In LLM4SE research, data are often transformed into specific formats to be used as input for LLMs. Table B1 illustrates four input formats, namely token-based input, tree/graph-based input, pixel-based input, and hybrid-based input, along with all the papers that utilize each type.

Table B1. The Various Input Forms of LLMs Proposed in Prior Studies

Category	Input forms	# Studies	References
Token-based input	Text in tokens	150	[11, 26, 32, 34, 38, 43, 45, 55, 56, 63, 64, 72, 73, 76, 79, 83, 91, 94, 95, 98, 107, 110, 129, 130, 132–135, 138, 142, 145–148, 152, 158, 162, 163, 166, 170]
Token-based input	Text in tokens		[169, 173, 174, 176, 180, 185, 188–191, 193, 196, 201, 203, 204, 207, 208, 213, 216, 219, 222, 224, 225, 231, 236, 240, 246, 252, 253, 259, 263, 265, 266, 268, 271–273, 275, 279, 283, 287, 288, 291, 292, 296, 298, 308, 309, 322, 329, 330, 333, 334, 340, 341, 345, 356, 374, 377, 379, 387, 388, 392, 395, 398, 399, 411, 416, 419, 421, 424, 429, 430, 433, 437, 438, 450, 451, 453, 454, 458, 462, 463, 465, 469, 470, 477, 479, 483, 490, 492, 495, 501, 502, 513, 520, 523, 525, 526, 534, 543, 548, 550, 551, 553, 554, 558, 567, 571, 574]
	Code in tokens	118	[1, 7, 10, 19, 31, 35, 37, 40, 48, 49, 53, 54, 57, 60, 74, 75, 84, 86, 90, 101–104, 106, 114, 119, 150, 151, 153, 157, 160, 164, 165, 167, 168, 179, 181–184, 194, 195, 209, 210, 215, 221, 223, 226, 242, 249, 251, 254–256, 274, 282, 284, 286, 306, 315, 324, 325, 332, 336, 344, 346, 354, 365, 366, 375, 381, 382, 384, 386, 388, 390–392, 396, 397, 402, 403, 408, 409, 412, 417, 418, 425, 426, 434, 439, 442, 445, 457, 471, 476, 478, 484, 486, 488, 491, 493, 494, 498, 499, 504, 505, 519, 529, 535, 539, 542, 544, 545, 552, 555, 564, 572]
	Code and text in tokens	78	[21, 44, 59, 67, 69, 70, 78, 89, 115, 120, 121, 124, 141, 144, 156, 161, 172, 175, 192, 205, 227, 230, 233, 238, 243, 248, 280, 285, 290, 293, 297, 307, 311, 317, 335, 339, 342, 343, 350, 357, 367, 368, 370–372, 376, 393, 394, 405, 406, 410, 420, 427, 428, 435, 446, 447, 460, 461, 472, 473, 480, 485, 489, 497, 507, 514, 515, 521, 524, 531, 533, 536, 540, 541, 562, 563, 575]
Tree/Graph-based input	Code in tree structure	2	[316, 547]
	Code in graph structure	3	[77, 276, 556]
Pixel-based input	Pixel	1	[302]
Hybrid-based input	Hybrid input forms	2	[9, 314]

C Prompt Engineering

Table C1 showcases eight prompt engineering techniques mentioned in 395 studies: Zero-shot prompting, Few-shot prompting, CoT prompting, APE, CoC prompting, Auto-CoT prompting, MoT prompting, and SCoT prompting.

Table C1. Prompt Engineering Techniques for SE Tasks

Prompt engineering	# Studies	References
Zero-shot prompting	79	[7, 21, 32, 35, 49, 59, 63, 64, 72, 73, 84, 86, 89, 102, 115, 120–122, 145, 153, 161, 175, 179, 180, 182, 183, 192, 204, 221, 225–227, 236, 238, 240, 252, 256, 259, 272, 285, 290, 311, 322, 332–335, 365, 374, 376, 384]
		[393, 408, 427, 428, 430, 434, 439, 450, 461, 465, 473, 480, 483, 486, 497, 501, 505, 513, 541, 542, 547, 550, 551, 554, 555, 562, 563, 566]

(Continued)

Table C1. Continued

Prompt engineering	# Studies	References
Few-shot prompting	88	[7, 10, 19, 21, 33, 35, 37, 38, 43, 59, 60, 64, 70, 73, 74, 78, 84, 91, 95, 103, 104, 120, 124, 125, 132, 133, 142, 144–147, 163, 166, 168, 170, 183–185, 189, 195, 207, 213, 225, 231, 238, 252, 274, 278, 290, 293, 297, 298, 307, 309, 311, 317, 327, 354, 363, 374, 377, 394, 404, 405, 408, 409, 428, 434, 450, 453, 461, 465, 469, 488, 490, 494, 495, 498, 500, 501, 513, 535, 548, 551, 560, 563, 566, 575]
CoT prompting	18	[64, 91, 150, 151, 225, 233, 263, 297, 345, 377, 394, 429, 450, 458, 498, 504, 531, 548]
APE	2	[408, 571]
CoC prompting	2	[144, 213]
Auto-CoT prompting	1	[327]
MoT prompting	1	[222]
SCoT prompting	1	[225]
Others	76	[7, 20, 34, 51, 59, 69, 75, 76, 82, 99, 107, 138, 141, 167, 169, 173, 175, 176, 188, 201, 209, 216, 219, 224, 233, 240, 243, 248, 254, 263, 264, 266, 276, 300, 302, 325, 332, 335, 339, 356, 368, 388, 392, 394, 396, 398, 400, 408, 410, 416, 418, 419, 433, 445, 449, 470, 474–476, 478, 484, 485, 489, 491, 492, 496, 504, 514, 515, 519, 521, 523–525, 534, 558]

D Evaluation Metrics

We categorize the types of tasks that LLMs address in SE into four categories: regression, classification, recommendation, and generation. Each task has commonly used evaluation metrics, as shown in Table D1.

Table D1. Evaluation Metrics for Different Types of Tasks

Problem type	Metric	# Studies	References
Regression	MAE	1	[98]
Classification	Precision	35	[10, 26, 35, 48, 53, 74, 77, 83, 90, 94, 130, 135, 147, 190, 191, 195, 201, 203, 215, 298, 335, 341, 346, 379, 382, 387, 402, 407, 424, 426, 502, 529, 531, 545, 553]
	Recall	34	[10, 26, 35, 48, 53, 74, 83, 90, 94, 130, 135, 147, 190, 191, 195, 201, 203, 215, 298, 335, 346, 379, 382, 387, 402, 407, 418, 424, 426, 529, 531, 545, 550, 553]
	F1-score	33	[26, 35, 48, 90, 94, 114, 130, 135, 147, 190, 191, 195, 201, 203, 215, 254, 272, 298, 335, 346, 379, 382, 386, 387, 402, 418, 424, 502, 529, 531, 545, 553, 569]
	Accuracy	23	[48, 110, 114, 165, 181, 182, 190, 191, 195, 201, 215, 216, 233]
	Accuracy		[254, 280, 307, 335, 346, 367, 426, 529, 531, 545]
	AUC	9	[11, 79, 386, 418, 426, 449, 499, 502, 545]
	ROC	4	[11, 77, 79, 386]
	FPR	4	[48, 411, 424, 449]
	FNR	3	[411, 424, 449]
	MCC	2	[94, 502]
Recommendation	MRR	15	[49, 55, 86, 160, 221, 223, 242, 246, 271, 282, 350, 372, 390, 447, 469]
	Precision/Precision@k	6	[49, 129, 246, 469, 479, 572]
	MAP/MAP@k	6	[49, 55, 158, 246, 469, 573]
	F-score/F-score@k	5	[129, 246, 479, 572, 573]
	Recall/Recall@k	4	[129, 469, 479, 572]
	Accuracy	3	[160, 223, 372]

(Continued)

Table D1. Continued

Problem type	Metric	# Studies	References
Generation	BLEU/BLEU-4/BLEU-DC	62	[1, 7, 9, 19, 40, 44, 57, 67, 102, 104, 115, 156, 157, 164, 170, 174, 207, 226, 238, 240, 243, 248, 255, 262, 268, 283–285, 288, 299, 314, 332, 365, 366, 381, 387, 388, 391–393, 408, 435, 446, 447, 451, 457, 460, 461, 473, 504, 505, 507, 513, 524, 529, 535, 536, 555, 563, 567, 574]
	Pass@k	54	[1, 31, 34, 38, 43, 69, 70, 73, 76, 84, 120, 124, 138, 144, 145, 161, 163, 169, 170, 208, 213, 222, 224, 225, 227, 236, 240, 259, 273, 293, 297, 309, 317, 357, 368, 388, 392, 393, 423, 429, 450, 458, 465, 470, 497, 505, 519, 524–526, 540–542, 563]
	Accuracy/Accuracy@k	38	[91, 142, 150, 151, 153, 162, 164, 173, 184, 185, 208, 249, 251, 255, 275, 284, 288, 290, 308, 314, 329, 334, 344, 354, 365, 377, 381, 392, 393, 412, 425, 490, 494, 513, 520, 529, 533, 543]
	EM	36	[1, 9, 54, 78, 102, 107, 119, 120–122, 164, 175, 226, 240, 251, 256, 285, 299, 332, 334, 344, 365, 381, 387, 393, 420, 457, 461, 472, 473, 477, 505, 513, 536, 556, 541]
	CodeBLEU	29	[1, 19, 44, 102, 121, 164, 170, 240, 248, 287, 324, 332, 365, 381, 391–393, 446, 451, 461, 472, 473, 505, 524, 529, 535, 563, 567, 574]
	ROUGE/ROUGE-L	22	[7, 9, 102, 104, 115, 156, 157, 174, 230, 238, 288, 314, 388, 408, 409, 412, 460, 501, 507, 524, 567, 574]
	Precision	18	[57, 101, 153, 179, 204, 268, 290, 387, 412, 425, 463, 473, 490, 501, 529, 536, 551, 575]
	METEOR	16	[7, 9, 40, 102, 104, 115, 174, 314, 388, 408, 409, 460, 507, 536, 567, 574]
	Recall	15	[101, 153, 179, 204, 227, 268, 290, 387, 412, 425, 463, 490, 501, 529, 551]
	F1-score	15	[101, 153, 179, 204, 227, 268, 290, 387, 412, 425, 463, 490, 501, 529, 551]
	MRR	6	[255, 314, 387, 447, 507, 529]
	ES	6	[78, 119, 243, 256, 420, 446]
	ED	5	[78, 226, 251, 322, 520]
	MAR	4	[365, 433, 447, 529]
	ChrF	3	[381, 567, 574]
	CrystalBLEU	3	[226, 567, 574]
	CodeBERTScore	2	[567, 574]
	MFR	1	[433]
	PP	1	[493]

E SE Tasks

According to the software development lifecycle, we have categorized the SE tasks mentioned in 395 studies into six categories: Requirements engineering, Software design, Software development, Software quality assurance, Software maintenance, and Software management. Table E1 presents all the papers that apply LLMs to these tasks.

Table E1. Distribution of SE Tasks over Six Activities

SE activity	SE task	# Studies	References
Requirements engineering	Anaphoric ambiguity treatment	4	[83, 291, 292, 400]
	Requirements classification	4	[79, 132, 135, 272]
	Requirement analysis and evaluation	2	[341, 363]
	Specification generation	2	[274, 490]
	Coreference detection	1	[462]
	Requirements elicitation	1	[475]
	Specification formalization	1	[82]
	Traceability automation	1	[246]
	Use cases generation	1	[548]

(Continued)

Table E1. Continued

SE activity	SE task	# Studies	References
Software design	GUI retrieval	1	[203]
	Rapid prototyping	1	[279]
	Software specification synthesis	1	[475]
	System design	1	[548]
Software development	Code generation	118	[20, 22, 32, 34, 38, 43–45, 67, 71–73, 76, 84, 107, 111, 120, 133, 138, 142, 144–146, 148, 161, 163, 166, 169, 170, 172, 176, 188, 192, 193, 205, 208, 209, 212, 213, 222, 224, 225, 227, 231, 236, 240, 247, 248, 252, 253, 259, 262, 266, 273, 278, 287, 297, 299, 301, 306, 308, 309, 317, 322, 329, 334, 354, 356, 357, 368, 371, 377, 381, 387, 388, 392, 395, 404, 405, 416, 421, 423, 427, 429, 430, 446, 449, 450, 451, 454, 458, 465, 470, 472, 478, 480, 482, 497, 504, 507, 514, 515, 519, 524–526, 528, 529, 535, 540, 541, 548, 554, 563, 564, 566, 567, 574]
	Code completion	22	[57, 68–70, 78, 119, 160, 179, 191, 223, 243, 249, 256, 316, 333, 342, 343, 387, 412, 420, 478, 493]
	Code summarization	21	[7, 9, 19, 40, 101, 102, 115, 175, 284, 288, 366, 368, 387, 388, 392, 408, 409, 427, 447, 460, 507]
	Code search	12	[86, 219, 221, 241, 255, 282, 370, 372, 387, 390, 447, 451]
	Code translation	12	[164, 168, 255, 262, 324, 325, 344, 392, 478, 498, 505, 507]
	Code understanding	8	[181, 276, 280, 300, 384, 461, 478, 558]
	API inference	5	[148, 330, 453, 473, 575]
	Program synthesis	6	[99, 124, 162, 207, 393, 398]
	API recommendation	5	[49, 242, 469, 479, 539]
	Code editing	5	[21, 122, 226, 293, 394]
	Code representation	3	[1, 314, 442]
	Code comment generation	2	[104, 283]
	Method name generation	2	[400, 572]
	Code recommendation	2	[271, 350]
	Agile story point estimation	1	[98]
	API documentation augment	1	[501]
	API documentation smells	1	[190]
	API entity and relation extraction	1	[147]
	Data analysis	1	[51]
	Fuzz driver generation	1	[530]
	Control flow graph generation	2	[151]
	Identifier normalization	1	[533]
	Instruction generation	1	[571]
	Type inference	1	[165]
	Others	14	[31, 129, 189, 298, 302, 311, 327, 339, 345, 374, 385, 513, 560, 569]

(Continued)

Table E1. Continued

SE activity	SE task	# Studies	References
Software quality assurance	Vulnerability detection	18	[35, 48, 101, 103, 114, 195, 201, 255, 315, 386, 402, 406, 410, 411, 417, 424, 500, 531]
	Test generation	17	[20, 59, 101, 196, 340, 375, 376, 391, 396, 403, 419, 437, 485, 491, 492, 523, 542]
	Bug localization	5	[55, 56, 77, 91, 182]
	Verification	5	[37, 95, 307, 430, 543]
	Testing automation	4	[63, 64, 141, 194]
	Fault localization	3	[233, 484, 499]
	Defect detection	2	[407, 478]
	GUI testing	2	[264, 518]
	Static analysis	2	[125, 290]
	Binary taint analysis	1	[254]
	Compiler fuzzing	1	[346]
	Decompilation	1	[495]
	Invariant prediction	1	[335]
	Malicious code localization	1	[407]
	Mobile app crash detection	1	[265]
	Resource leak detection	1	[445]
	Test prediction	1	[90]
Software maintenance	Program repair	35	[33, 37, 60, 89, 102, 150, 153, 157, 167, 173, 210, 215, 262, 284, 332, 336, 365, 397, 399, 425–427, 457, 471, 474, 476, 483, 486, 488, 489, 521, 544, 547, 555, 556]
	Code clone detection	8	[10, 53, 74, 168, 255, 367, 382, 387]
	Code review	7	[121, 230, 262, 268, 379, 435, 536]
	Debugging	4	[183, 371, 428, 433]
	Bug reproduction	3	[152, 184, 185]
	Review/commit/code classification	3	[106, 204, 502]
	Duplicate bug report detection	3	[132, 158, 550]
	Logging	3	[238, 286, 494]
	Log parsing	3	[263, 275, 520]
	Sentiment analysis	3	[26, 551, 553]
	Code revision	2	[180, 439]
	Vulnerability repair	2	[156, 333]
	API misuses repair	1	[552]
	Bug prediction	1	[110]
	Bug triage	1	[216]
	Code coverage prediction	1	[434]
	Code review explained	1	[477]
	Code-Review defects repair	1	[562]
	Crash bug repair	1	[75]
	Crash bug repair	1	[75]
	Dockerfile Repair	1	[134]
	Patch correctness prediction	1	[545]
	Patch detection	1	[418]
	Program merge conflicts repair	1	[534]
	Rename Refactoring	1	[251]
	Tag recommendation	1	[130]
	Technical debt payback	1	[285]
	Traceability recovery	1	[573]
	Web test repair	1	[496]
	Type error repair	1	[54]
	Others	5	[174, 296, 378, 438, 463]
Software management	Effort estimation	2	[11, 239]
	Software tool configuration	1	[186]

Received 3 September 2023; revised 9 August 2024; accepted 27 August 2024